



DEPARTMENT OF HEALTH AND CLINICAL OUTCOMES RESEARCH

Health Data Science Capstone

HDS-5960-02 (Summer 2025)

INSTRUCTORS

Divya S. Subramaniam, PhD, MPH

Dipti P. Subramaniam, PhD, MPH

**PREDICTING MENTAL HEALTH SEVERITY AMONG U.S. COLLEGE
STUDENTS WHO USE FACEBOOK**

By

Supraja Medicherla

Sashi Priya Koka

Anusha Reddy Mula

Jayalakshmi Jawahar Nesan

REPORT

ABSTRACT

Anxiety and depression remain widespread among U.S. college students, particularly those who actively engage with social media platforms such as Facebook. This study investigates the use of supervised machine learning models to classify anxiety severity based on demographic, clinical, and emotional distress indicators collected through self-reported survey data from 579 college students across 48 U.S. states. After comprehensive data cleaning, feature engineering, and statistical analysis, seven machine learning algorithms were evaluated, including logistic regression, random forest, XGBoost, support vector machine, artificial neural network (ANN), and K-Nearest Neighbors (KNN). The final tuned KNN classifier demonstrated the best overall performance with an F1 score of 0.879, recall of 92.7%, and AUC of 0.781, effectively identifying individuals with mild to severe anxiety symptoms. Emotional distress variables-particularly “Exhausted (not physical),” “Hopeless,” and “Very sad”-were found to be the most influential predictors, while demographic and general health indicators showed limited relevance. A user-friendly web application was also developed using Streamlit to demonstrate real-world applicability of the model. The findings highlight the feasibility of using interpretable, data-driven tools for scalable mental health screening, and support further integration of such models into campus wellness initiatives and digital health platforms. Future research should incorporate behavioral data, multimodal sources, and longitudinal designs to enhance predictive accuracy and generalizability.

Keywords: Anxiety Prediction, Machine Learning, K-Nearest Neighbors (KNN), College Students, Facebook, Mental Health Screening, GAD-7, Emotional Distress, Streamlit App, Social Media Analytics

INTRODUCTION

Mental health disorders are a growing concern in the United States, particularly among young adults-one of the most vulnerable populations.^{1,2,3} Globally, anxiety and depression are the most prevalent mental health conditions, affecting an estimated 264 million and 280 million individuals, respectively.⁴ In the U.S., approximately one in five adults experiences some form of mental illness annually, with one in twenty facing more severe conditions.³ Among adolescents, the prevalence of mental health disorders rose sharply from 33% to 44% between 2011 and 2021, with the steepest increases observed among females and sexual minority youth.^{4,5}

Numerous studies have found that individuals with mental health concerns engage with social media at rates equal to or higher than their peers, with usage among young adults reaching as high as 97%.^{6,7,8} Given Facebook’s widespread popularity among college-aged populations, concerns have emerged about its potential impact on mental health.^{6,8,9} With over 2.8 billion users worldwide, Facebook remains one of the most commonly used platforms-especially among young adults.¹⁰ Research has increasingly linked prolonged or excessive Facebook use to elevated symptoms of depression, anxiety, and stress.^{9,10} These mental health conditions, in turn, are major contributors to reduced quality of life globally.¹ Anxiety disorders in young people are frequently accompanied by psychiatric or medical comorbidities, which may severely impact social functioning and academic performance.^{1,2,5,6,10} As social media adoption surged-from 12% to 90% among U.S. adults aged 18-29 between 2005 and 2015-college students became

increasingly exposed to online risks such as cyberbullying and social comparison.^{2,7} These behaviors can exacerbate symptoms of anxiety and depression.^{4,5,6,9} Notably, this demographic has also seen rising rates of suicidal ideation and behavior during this same period.³

While platforms like Facebook offer benefits such as connectivity and social support, excessive or unregulated use has been associated with poor sleep quality, reduced life satisfaction, academic difficulties, anxiety, depression, and suicidality.^{1,3,4,10,11} Although prior studies have examined the broader relationship between social media and mental health among college students, few have identified specific demographic and health-related predictors of anxiety severity among Facebook users.^{3,5,6,7,8} Given the rising demand for timely mental health support and limited clinical resources on college campuses, there is a growing need for scalable, technology-enabled screening tools that can identify at-risk individuals early and efficiently.^{5,10,11}

To address this gap, this study investigates how individual demographics and prior mental health history relate to the severity of anxiety symptoms among U.S. college students who use Facebook. Using supervised machine learning algorithms, the goal is to identify individuals at higher risk for severe anxiety and support early intervention strategies.

The results are intended to inform the development of scalable mental health screening tools that can assist college wellness centers, university counseling programs, and digital health platforms. By uncovering predictors of anxiety severity from self-reported survey data, this project contributes to the growing field of data-driven mental health support tailored to digitally engaged populations.

METHODS

Study Sample and Data Source:

The dataset used in this project was derived from a national mental health survey aimed at assessing emotional well-being and psychological health among U.S. college students who are active Facebook users. It comprises anonymous survey responses from 579 college students collected during the 2021-2022 academic year across 48 U.S. states. Participants were born between 1971 and 2003. Although specific Facebook usage patterns were not collected, participants completed standardized assessments such as the Patient Health Questionnaire-9 (PHQ-9) for depression and the Generalized Anxiety Disorder-7 (GAD-7) for anxiety. Additional data included demographics (sex, age, race, state), general health status, emotional distress indicators (e.g., hopelessness, sadness, exhaustion), suicidal ideation, and mental health history (e.g., prior depression diagnosis, therapy, medication use), as well as the number of self-reported medical and psychological conditions. Composite scores for depression and anxiety severity were calculated using PHQ-9 and GAD-7 responses.¹²

Participants were eligible if they reported active Facebook use, were currently enrolled in a U.S. college or university during the 2021-2022 academic year, and completed both psychological assessments. Cases with incomplete or invalid demographic information, missing responses on PHQ-9 or GAD-7, or no reported Facebook use were excluded. After applying these criteria, 579 valid responses were retained for analysis.¹²

Data Cleaning and Preparation:

The original dataset contained 118 variables, including numeric scores, categorical responses, and derived indicators related to mental health. Initial inspection was performed using descriptive statistics and summary functions to understand data distribution, identify missing values, and verify variable types. Subsequently, several data cleaning steps were then applied. Redundant or highly granular features-such as individual PHQ-9 and GAD-7 question responses, binary flags for medical conditions, and administrative fields (e.g., full-time status, international student status, and unused service history fields)-were removed to reduce dimensionality and noise. Only the composite mental health scores and core predictor variables were retained for modeling. Columns were also renamed for clarity; for example, `acha_depression` was renamed to `Previously_Diagnosed_Depression`, and emotional distress variables like `acha_12months_times_1` were renamed to clearer labels such as `Hopeless`, `Overwhelmed`, and `Exhausted`.

Feature Engineering:

New variables were derived to enhance modeling effectiveness. Age was calculated by subtracting birth year from 2022. General Health responses were mapped from categorical descriptors (e.g., Poor, Fair) to an ordinal numeric scale from 1 to 5. Emotional distress variables (e.g., Hopeless, Overwhelmed, Exhausted) were transformed from categorical frequency responses (e.g., “Never,” “1-2 times,” “11 or more times”) into ordered numeric values ranging from 0 to 6. Sex was encoded as a binary variable (Male = 0, Female = 1). The PHQ-9 total score (range 0-27) was categorized into five clinical depression severity levels and stored as `Depression_Severity_Score`. The GAD-7 total score (range 0-21) was categorized into four anxiety severity levels and stored as `Anxiety_Severity_Score`.¹³

To improve classification performance, the original four-level anxiety severity variable was consolidated into a binary target variable, `Anxiety_Severity`, with 0 representing minimal anxiety (no clinically significant symptoms) and 1 representing mild to severe anxiety (elevated symptoms meeting clinical thresholds).¹³ Race variables (from `race_1` to `race_6`) were combined into a single categorical variable, `Race`, to simplify analysis.

Missing values in key binary variables such as `Previously_Diagnosed_Depression` and `In_Therapy_for_Depression` were encoded as 2, representing unknown status. This approach preserved data while acknowledging uncertainty and avoided potential bias from imputation or exclusion, which was particularly important given the dataset’s limited size.

Exploratory Data Analysis (EDA):

Exploratory analyses were conducted to examine variable distributions, identify patterns, and explore associations between predictors and anxiety severity. Data processing and visualization were performed using Python libraries, including pandas, seaborn, and matplotlib. Descriptive statistics summarized participant characteristics such as age, sex, race, general health, emotional distress indicators, and depression and anxiety severity scores. Chi-square tests were used to assess associations between categorical variables (e.g., sex, prior depression diagnosis, therapy status, and emotional distress symptoms) and anxiety severity. For continuous variables such as age and general health, one-way ANOVA was conducted to determine whether there were

significant differences in mean values across anxiety severity groups. A significance threshold of $p < 0.05$ was applied to all statistical tests. In addition, 95% confidence intervals (CIs) were calculated for mean age and general health scores within each anxiety group to quantify the precision of group mean estimates and aid in interpreting the clinical relevance of any observed differences.¹⁴

Pearson correlation matrices assessed linear relationships among numeric variables to detect multicollinearity and associations with mental health outcomes. Emotional distress variables were reshaped into long format and visualized with grouped bar plots stratified by frequency to compare distress indicators clearly. Racial representation was displayed using a pie chart derived from consolidated race variables. Violin plots examined depression severity by gender, and boxplots depicted anxiety severity distribution by age. General health scores were compared across anxiety severity categories using grouped count plots. State-level average PHQ-9 and GAD-7 scores were calculated and presented via grouped bar charts to visualize geographic variation in depression and anxiety.

Feature Selection:

Feature selection was guided primarily by statistical results from EDA, supported by established literature and mental health domain expertise. The goal was to identify predictors with statistically significant associations with anxiety severity and ensure their relevance to modeling. A total of 13 predictor variables-comprising both categorical and numerical data-were selected for inclusion, with Anxiety_Severity defined as the binary classification target (0 = no anxiety, 1 = mild to severe anxiety). The selected features spanned multiple key domains. Demographic variables included sex (coded as Male = 0, Female = 1), age, and race.^{3,4,5} Mental health history was represented by Previously_Diagnosed_Depression, In_Therapy_for_Depression, and Depression_Severity_Score. Emotional distress indicators included Hopeless, Overwhelmed, Exhausted (non-physical), Very Sad, Depressed - hard to function, Seriously Considered Suicide, and Attempted Suicide.^{1,2,3,5,6,9}

General health was captured via an ordinal scale (1 = Poor to 5 = Excellent), along with a Total_Health_Issues_Score summarizing self-reported medical and psychological conditions.^{1,3,6,10} These variables were selected due to their established clinical relevance and significant patterns observed within the dataset.

Data Transformation and Preprocessing:

To prepare the dataset for modeling, several transformation and preprocessing steps were applied. Categorical variables such as sex and race were converted into binary indicator variables using one-hot encoding. To avoid multicollinearity, the first category was dropped from each encoded feature. Numerical variables were standardized using z-score normalization to ensure that all features were on a consistent scale, which is particularly important for distance-based models like K-Nearest Neighbors.

The dataset was then split into training and testing subsets using an 80/20 stratified split to preserve the original distribution of the target classes (minimal vs. mild-to-severe anxiety). Because the target variable showed class imbalance, with fewer instances of minimal anxiety, we applied the Synthetic Minority Oversampling Technique (SMOTE) to the training data.¹⁵

SMOTE creates synthetic samples of the minority class to help balance class representation and reduce bias during model training. Importantly, SMOTE was applied only to the training set to prevent data leakage and ensure a fair and unbiased evaluation on the test set.^{15,16} All preprocessing steps-including encoding, scaling, and SMOTE resampling-were implemented using a unified pipeline constructed with scikit-learn and imblearn libraries. This ensured consistent application of transformations and reproducibility of results throughout the modeling workflow.

Modeling Techniques:

Seven supervised machine learning algorithms were developed to predict anxiety severity as a binary classification task. All models were trained on the SMOTE-resampled training set and evaluated on the holdout test set.^{15,17} The models included:

The logistic regression model was trained using the 'lbfgs' solver and class weights were balanced to address initial class imbalance. Hyperparameter tuning was performed using GridSearchCV with 5-fold cross-validation, optimizing the regularization parameter C over a logarithmic range [0.01, 0.1, 1, 10], and evaluated based on the weighted F1 score. XGBoost was first trained with default parameters and then tuned with a learning rate of 0.05, 500 estimators, a max depth of 3, and regularization parameters (reg_alpha = 10, reg_lambda = 20). Log-loss was used as the evaluation metric, and cross-validation was conducted to assess model performance.¹⁹

The random forest model started with 100 estimators and was tuned to 250 trees, with a maximum depth of 6, minimum samples per split of 10, and minimum samples per leaf of 11.¹⁸ Balanced class weights and the 'sqrt' feature selection strategy were used. Similarly, the decision tree model was tuned via GridSearchCV over parameters such as max depth (7), min samples split (11), and max_features = 'sqrt'.^(20,21) Both tree-based models were evaluated on training and test sets with 5-fold cross-validation to ensure stability.^{19,21}

The support vector machine was tested with various kernels, with the best-performing kernel selected based on accuracy and F1 score.^{17,19} GridSearchCV was applied to tune the regularization parameter C and kernel-specific hyperparameters.²¹ A validation curve was plotted to examine sensitivity to different C values. The final SVM model was evaluated comprehensively across all key metrics.^{19,21}

The KNN classifier was initially trained with k=5 using Euclidean distance. GridSearchCV was used to identify the best k (ranging from 1-20), distance metrics (e.g., Manhattan distance), and weight functions (uniform vs. distance). The final model with k=9 was evaluated using 5-fold cross-validated F1 scores.^{19,21}

The artificial neural network was constructed using TensorFlow and Keras. The baseline architecture consisted of one hidden dense layer with 32 ReLU units and a sigmoid output layer. The model was compiled with the Adam optimizer and binary cross-entropy loss, and trained for 20 epochs with a batch size of 32. Hyperparameter tuning was performed using Keras Tuner's RandomSearch, optimizing hidden layer size (16-128 units) and learning rate (0.01-0.0001). All models were evaluated with standard classification metrics including accuracy, precision, recall, F1 score, and confusion matrices.^{18,19,21}

Model Performance Evaluation:

A comprehensive evaluation was conducted on the holdout test dataset using multiple metrics. Predicted probabilities for the positive class (mild to severe anxiety) were used to generate Receiver Operating Characteristic (ROC) curves and calculate Area Under the Curve (AUC) scores.^{20,21} ROC curves illustrate each model's discrimination ability across classification thresholds.

Additional metrics - accuracy, precision, recall (sensitivity), and F1 score - were computed. Accuracy reflects overall correct prediction proportion; precision and recall quantify the model's capacity to minimize false positives and false negatives, respectively. The F1 score balances precision and recall, especially relevant for imbalanced classes.^{20,21}

Performance metrics were calculated before and after hyperparameter tuning. Optimization utilized GridSearchCV or Keras Tuner with weighted F1 score as the primary objective.²¹ To ensure fairness and robustness, all models were evaluated on the same test set, and results were summarized in tabular form for direct comparison. Models were ranked primarily by weighted F1 score to emphasize balanced performance.²⁰ This framework facilitated identification of the best-performing models for anxiety severity classification and demonstrated the beneficial impact of tuning on accuracy and reliability.

To better understand model decision-making and identify key predictors influencing classification, permutation feature importance analysis was conducted on the final tuned model. This approach quantified the decrease in model performance (F1 score) when individual features were randomly permuted, thereby highlighting features most critical to accurate anxiety severity prediction. Such interpretability analysis provides valuable insights into the underlying relationships captured by the model and supports transparency in mental health prediction.

Model Selection:

Based on the comprehensive evaluation of all models using multiple performance metrics and ROC-AUC analysis, the tuned K-Nearest Neighbors (KNN) model demonstrated the best overall balance of accuracy, precision, recall, F1 score, and AUC.^{20,21} Its superior performance on the holdout test set, along with consistent cross-validation results, supported its selection as the final predictive model for classifying anxiety severity among U.S. college students.

App Development:

To enhance the accessibility and usability of the predictive model, a user-friendly web application was developed using Streamlit.²² The app, titled "Anxiety Screener for Facebook Users," allows individuals to input relevant demographic and health-related information and receive a personalized prediction of their anxiety severity level. The application integrates the final trained K-Nearest Neighbors (KNN) classifier along with the preprocessing pipeline used during model training.

User inputs are automatically transformed using the saved preprocessing object-which includes feature scaling and categorical encoding-before being passed to the KNN model. The app then predicts both the class label (minimal vs. mild to severe anxiety) and the associated probability

score. Results are presented with confidence levels and supportive messaging to promote mental health awareness and guide users toward appropriate resources when needed.

The model and web application were first tested locally, then deployed for public use. The live app is publicly accessible at:

<https://anxiety-screener-app.streamlit.app/>

RESULTS

Descriptive Statistics and Association Analysis:

To explore associations between key variables and anxiety severity levels (as measured by GAD-7), both numerical and categorical predictors were tested using appropriate statistical methods.

ANOVA for Continuous Variables:

A one-way Analysis of Variance (ANOVA) was performed to determine whether the means of continuous variables significantly differed across the four levels of anxiety severity (Minimal, Mild, Moderate, and Severe).

Table 1:

ANOVA Results for Numerical Variables (Age, General Health)

	Variable	P-value	Significant(<0.05)
1	Age	0.618836	False
2	General_Health	0.984618	False

Neither age nor general health status showed statistically significant differences across anxiety severity groups ($p > 0.05$), suggesting these continuous variables do not differ significantly across anxiety levels in this dataset.

Table 2:

Confidence Intervals for Numerical Variables by Anxiety Severity Group

	Variable	GAD_Severity	Mean	95% CI Lower	95% CI Upper	N
1	Age	0	23.56	22.77	24.35	172
2	Age	1	23.62	23.09	24.15	407
3	General_Health	0	3.19	3.04	3.33	172

4	General_Health	1	3.17	3.07	3.26	407
---	----------------	---	------	------	------	-----

Table 2 presents the mean values and 95% confidence intervals for Age and General Health, stratified by anxiety severity (GAD_Severity: 0 = minimal anxiety, 1 = mild to severe anxiety). For Age, the mean values were nearly identical between the two groups (23.56 for minimal anxiety vs. 23.62 for mild to severe anxiety), with overlapping confidence intervals (22.77-24.35 and 23.09-24.15, respectively), indicating no significant difference. Similarly, General Health scores showed comparable means (3.19 vs. 3.17) and overlapping confidence intervals (3.04-3.33 and 3.07-3.26), reinforcing the ANOVA findings that neither Age nor General Health significantly varies by anxiety severity.

Chi-Square Tests for Categorical Variables:

Chi-square tests of independence were conducted to assess the association between various categorical and ordinal predictors and anxiety severity (GAD-7 severity scale: minimal, mild, moderate, severe). The table below summarizes the p-values and statistical significance for each variable:

Table 3:

Chi-Square Test Results for Categorical Variables Associated with Anxiety Severity

	Variable	P-Value	Significant (<0.05)
1	Depressed - hard to function*	2.675725e-47	True
2	Very sad*	1.140630e-41	True
3	Exhausted (not physical)*	1.484494e-38	True
4	Hopeless*	5.617337e-37	True
5	Overwhelmed*	3.403522e-34	True
6	Total_Health_Issues_Score*	4.244773e-20	True
7	Seriously considered suicide*	4.971903e-13	True
8	Sex*	2.559304e-10	True
9	Previously_Diagnosed_Depression*	8.383576e-08	True
10	In_Therapy_for_Depression*	2.329720e-03	True
11	Attempted suicide*	1.702317e-02	True
12	Race	6.173724e-02	False

13	Depression_Severity_Score	1.829259e-01	False
----	---------------------------	--------------	-------

* Indicates statistical significance at $p < 0.05$.

Chi-Square Test Results for Categorical Variables Most emotional distress indicators, along with demographic and health history variables, demonstrated statistically significant associations with anxiety severity ($p < 0.05$). However, Race and Depression Severity Score were not significantly associated in this dataset.

Following the identification of statistically significant predictors through ANOVA and Chi-square tests, multiple machine learning models were developed to classify anxiety severity based on these features.

Model Development and Performance Comparison:

Multiple machine learning models were trained to predict anxiety severity (minimal vs. mild-to-severe) using an 80/20 stratified train-test split. Synthetic Minority Oversampling Technique (SMOTE) addressed class imbalance during training. Models included Logistic Regression, XGBoost, Random Forest, Decision Tree, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Artificial Neural Network (ANN).

Performance Before Hyperparameter Tuning:

Table 4:

Model Performance Before Hyperparameter Tuning

 Model Performance Comparison Table:

	Model	Accuracy	Precision	Recall	F1 Score	AUC
6	ANN	0.8103	0.8488	0.8902	0.8690	0.8264
5	KNN	0.7931	0.8372	0.8780	0.8571	0.7676
2	Random Forest	0.7931	0.8372	0.8780	0.8571	0.7884
0	Logistic Regression	0.7931	0.8718	0.8293	0.8500	0.8225
4	SVM	0.7759	0.8415	0.8415	0.8415	0.7744
1	XGBoost	0.7500	0.7978	0.8659	0.8304	0.7231
3	Decision Tree	0.7414	0.7889	0.8659	0.8256	0.6535

Before tuning, the ANN achieved the highest F1 score (0.8690) and area under the ROC curve (AUC = 0.8264), closely followed by KNN (F1 = 0.8571, AUC = 0.7676) and Random Forest (F1 = 0.8571, AUC = 0.7884). Logistic Regression also performed well, with an F1 score of 0.85 and AUC of 0.8225. Compared to ANN and KNN, the XGBoost and Decision Tree models

demonstrated lower F1 scores and AUC values, indicating reduced effectiveness in classifying anxiety severity.

Performance After Hyperparameter Tuning:

Table 5:

Model Performance After Hyperparameter Tuning

Tuned Models Performance Comparison Table:						
	Model	Accuracy	Precision	Recall	F1 Score	AUC
5	Tuned KNN	0.8190	0.8352	0.9268	0.8786	0.7807
6	Tuned ANN	0.8190	0.8588	0.8902	0.8743	0.8350
2	Tuned Random Forest	0.7931	0.8372	0.8780	0.8571	0.7884
3	Tuned Decision Tree	0.7845	0.8202	0.8902	0.8538	0.7816
0	Tuned Logistic Regression	0.7931	0.8718	0.8293	0.8500	0.8225
4	Tuned SVM	0.7845	0.8800	0.8049	0.8408	0.8250
1	Tuned XGBoost	0.7500	0.7978	0.8659	0.8304	0.7231

Hyperparameter tuning via GridSearchCV (for traditional models) and Keras Tuner (for ANN) yielded significant improvements. The tuned KNN model stood out, achieving an F1 score of 0.8786, recall of 0.927, and an AUC of 0.7807, reflecting excellent sensitivity for detecting elevated anxiety symptoms. The tuned ANN also delivered strong results (F1 = 0.8743, AUC = 0.8350). Tuned Random Forest and Decision Tree models showed moderate gains, while tuned Logistic Regression and SVM maintained stable, slightly lower performance. Although XGBoost improved marginally, it remained the least effective among the evaluated models. These findings underscore the critical value of hyperparameter tuning, particularly for KNN and ANN, which benefited most in terms of sensitivity and generalization. Given its overall balance of recall, F1 score, and accuracy, the tuned KNN was selected as the final predictive model.

Receiver Operating Characteristic (ROC) Analysis:

Before Hyperparameter Tuning:

Figure 1:

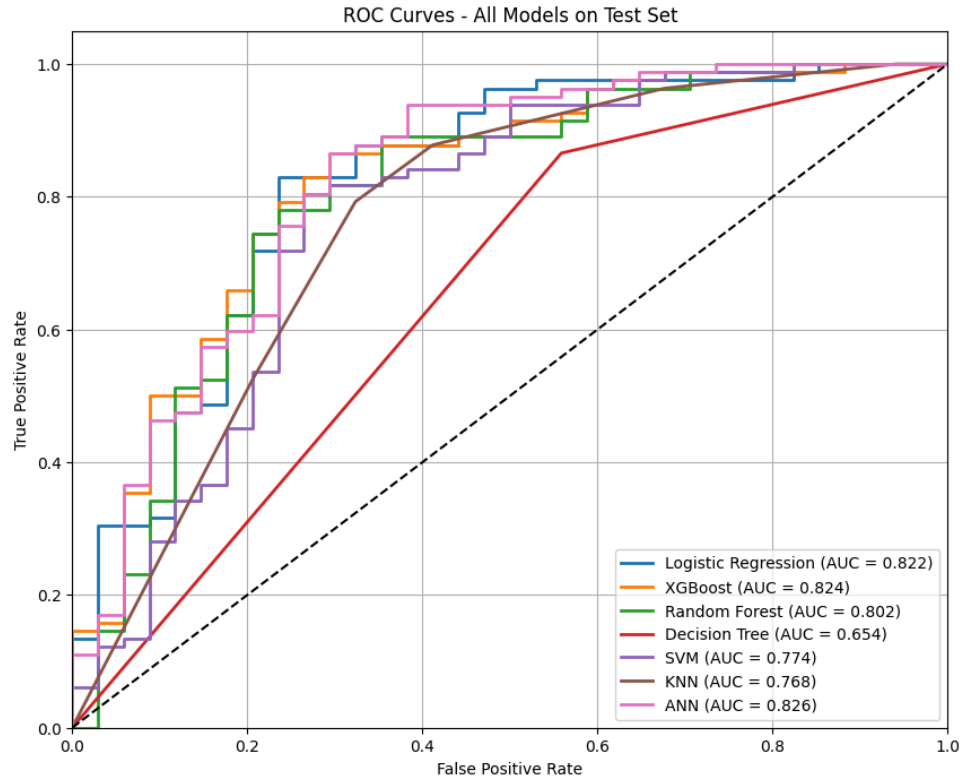


Figure 1 shows ROC curves for all models on the test set before hyperparameter tuning.

The ROC curves illustrate baseline model performance before hyperparameter tuning. Among the models, the ANN maintained the highest AUC (0.826), demonstrating its strong generalization ability in distinguishing between minimal and mild-to-severe anxiety. XGBoost (AUC = 0.824) and Logistic Regression (AUC = 0.822) followed closely, indicating that both complex and simpler models can capture meaningful patterns in the data. Random Forest (AUC = 0.802) and K-Nearest Neighbors (KNN, AUC = 0.786) also showed solid performance. In contrast, the Decision Tree model performed the worst, with an AUC of 0.654, reflecting limited predictive capability at baseline.

After Hyperparameter Tuning:

Figure 2:

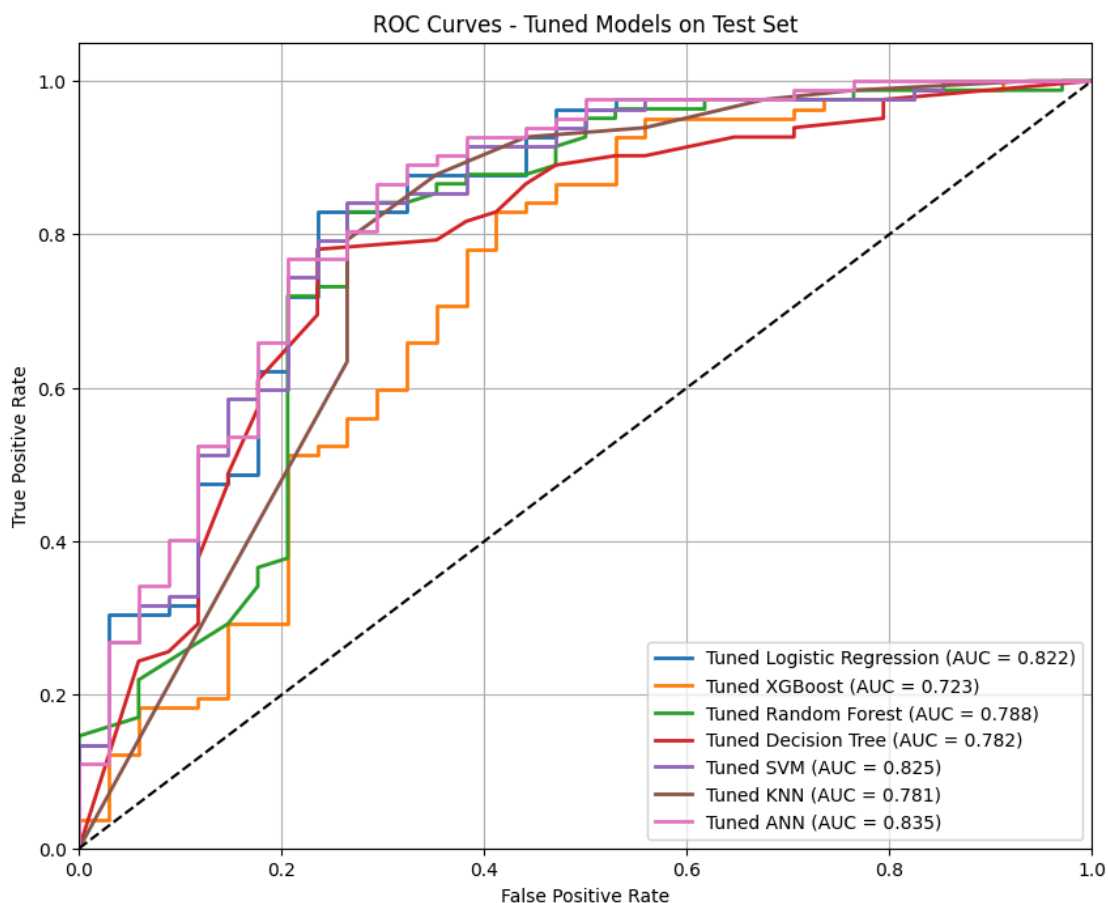


Figure 2 displays ROC curves for all models after hyperparameter tuning.

The ROC curves show enhanced discriminative performance for most models following hyperparameter tuning. The tuned ANN displayed the highest AUC (0.835), reaffirming its robustness and superior generalization capacity. Logistic Regression maintained a stable AUC of 0.823, suggesting that its performance was already near optimal, with regularization tuning contributing to consistent results. Support Vector Machine (SVM) showed a notable improvement, increasing its AUC to 0.825 after tuning its kernel and regularization parameters. Decision Tree and Random Forest models also improved slightly, with AUCs rising to 0.782 and 0.788, respectively, but continued to underperform compared to ANN, SVM, and Logistic Regression. The K-Nearest Neighbors KNN model improved to an AUC of 0.781, reflecting better sensitivity. In contrast, XGBoost showed only slight progress (AUC = 0.723), suggesting limited alignment with the dataset's structure. Overall, ANN, SVM, and Logistic Regression emerged as the top-performing classifiers after tuning.

Final Model Selection and Performance:

Evaluation based on classification metrics and ROC-AUC revealed important differences. Although the tuned ANN achieved the highest AUC (0.835), the tuned KNN consistently outperformed other models in F1 Score (0.879), Accuracy (81.9%), and Recall (92.7%). These metrics are especially important in mental health contexts where minimizing false negatives is critical. Logistic Regression and SVM also showed balanced, stable performance, while XGBoost underperformed despite its popularity. Overall, the tuned KNN offers the best combination of discrimination and predictive power for classifying anxiety severity.

Table 5:

Performance Metrics for Tuned K-Nearest Neighbors Model on Train and Test Sets

Tuned KNN Model Performance Metrics				
	Accuracy	Precision	Recall	F1 Score
Train	0.8661	0.879	0.9385	0.9077
Test	0.819	0.8352	0.9268	0.8786

Figure 3:

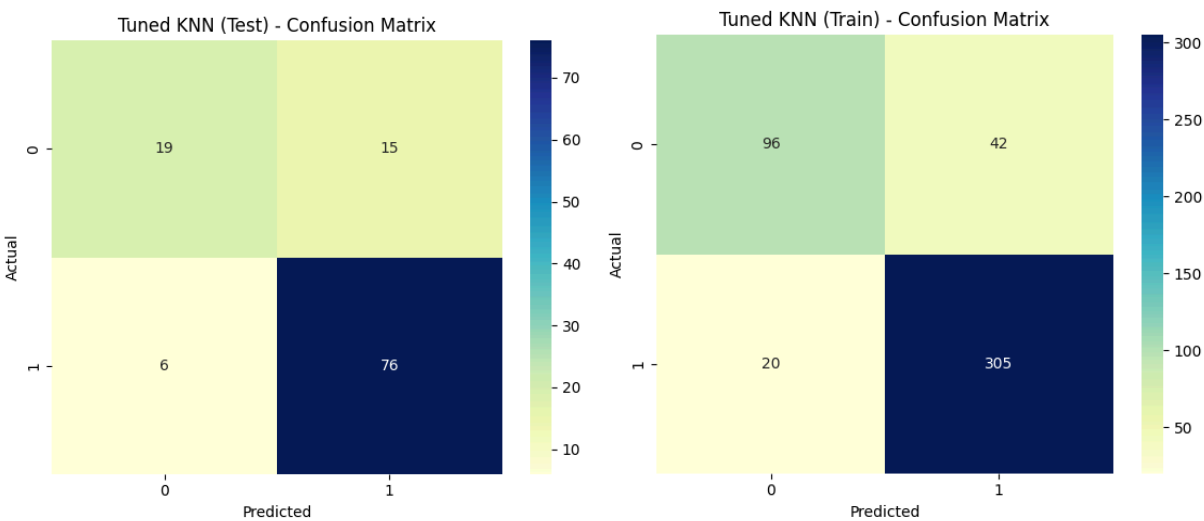
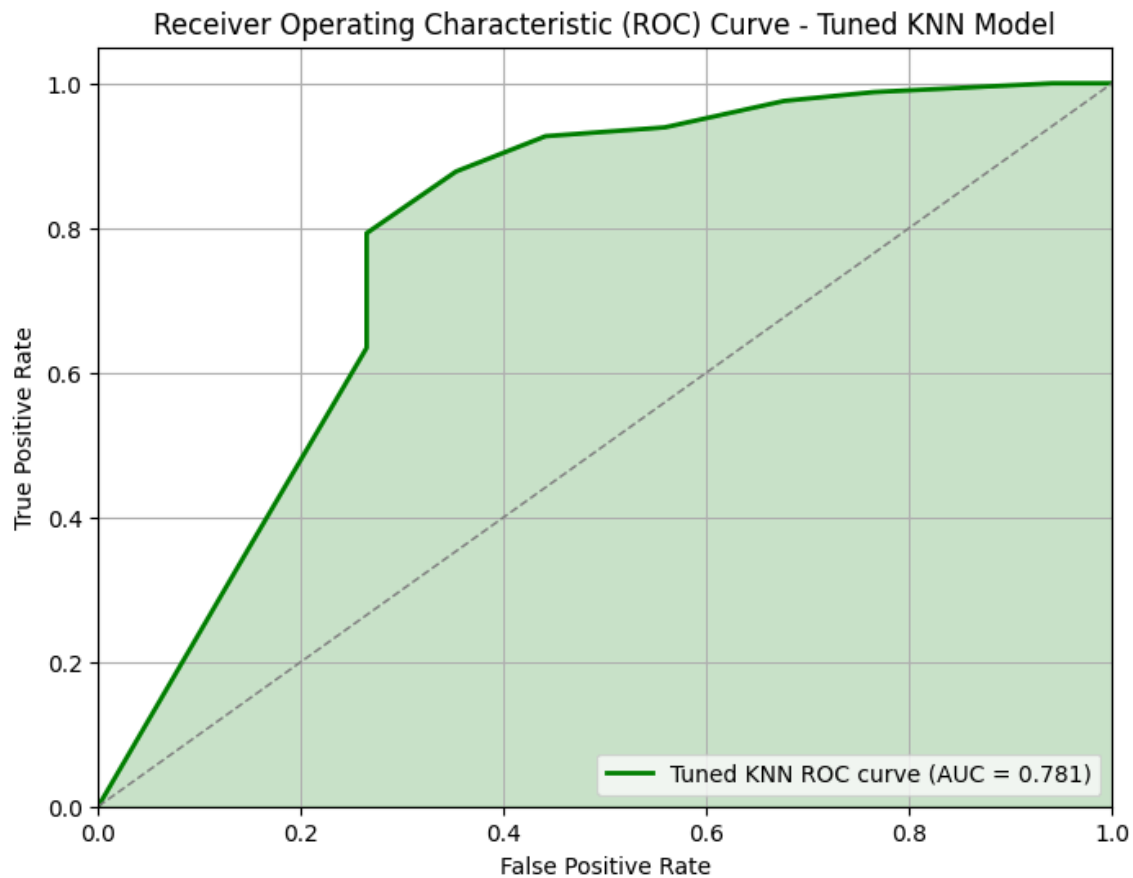


Figure 3 presents the confusion matrices for the tuned KNN model on both training and test sets.

The confusion matrices for the tuned KNN model on the training and test sets demonstrate its strong ability to classify anxiety severity. On the test set, the model correctly identified 76 out of 82 individuals with mild-to-severe anxiety, achieving a recall of 92.7% and an overall accuracy of 81.9%. Only 6 false negatives were observed, indicating high sensitivity, which is especially important in mental health prediction where failing to detect elevated symptoms can have serious

consequences. The model also produced 15 false positives, where individuals with minimal anxiety were incorrectly classified. On the training set, the model achieved even higher performance, with a recall of 93.8% and an accuracy of 86.6%, correctly identifying 305 out of 325 individuals with mild-to-severe anxiety.

Figure 4:



As shown in **Figure 4**, the ROC curve for the tuned KNN model confirms its strong discriminative ability, with an AUC of 0.78.

Additionally, feature importance analysis using permutation importance (Figure) highlights the most influential predictors in the model. This interpretability insight aids in understanding which variables contribute most to anxiety severity classification.

Figure 5:

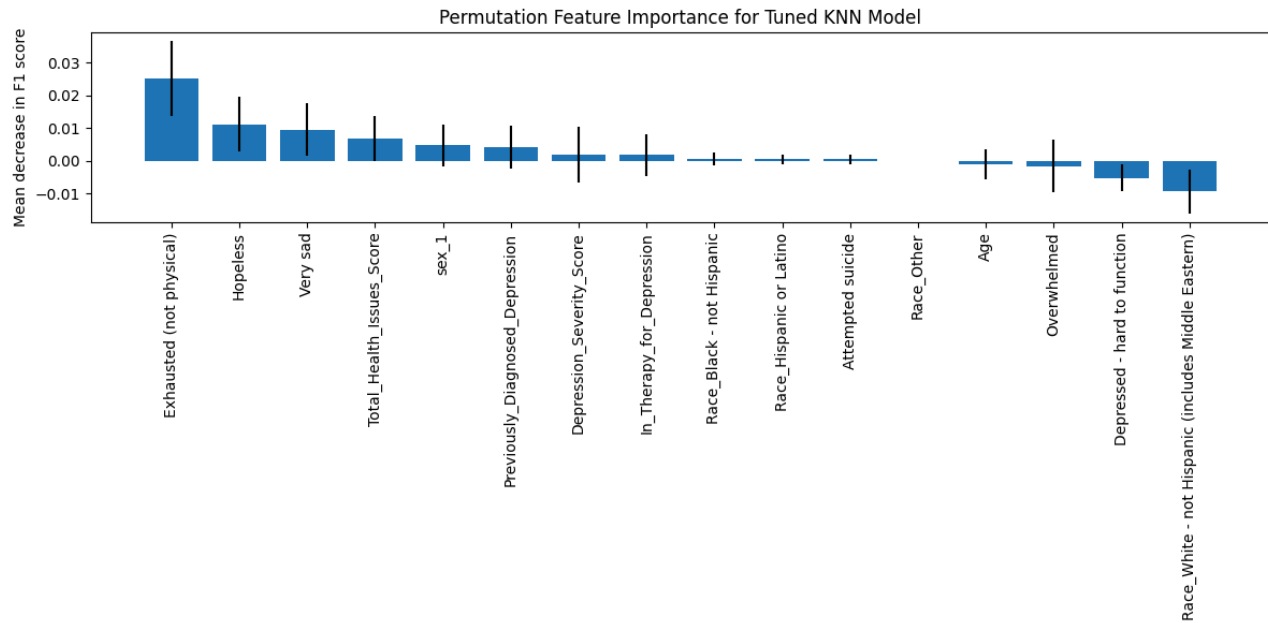


Figure 5 shows the permutation importance plot, highlighting the most influential predictors in the tuned KNN model.

Permutation feature importance analysis was conducted on the tuned K-Nearest Neighbors model to identify the predictors most influential in classifying anxiety severity. The results showed that “Exhausted (not physical)” was the most important feature, as permuting its values led to the largest decrease in the F1 score. Other key emotional distress indicators, including “Hopeless” and “Very sad,” also significantly impacted model performance. In contrast, variables such as “Depressed - hard to function” and “Overwhelmed” had minimal or negative importance, suggesting they contribute little unique information for anxiety classification. Demographic variables, including age and race categories, demonstrated low importance, indicating a lesser role in predicting anxiety severity within this dataset. Overall, the model relies primarily on emotional distress-related features to distinguish between different anxiety severity levels.

DISCUSSION

This study developed and evaluated multiple supervised machine learning models to classify anxiety severity among U.S. college students who actively use Facebook. By integrating demographic, clinical, and emotional distress indicators, the tuned K-Nearest Neighbors (KNN) classifier demonstrated strong predictive performance. For the high-risk group (mild to severe anxiety), the KNN model achieved a recall of 92.7% and an F1 score of 0.879, reflecting excellent sensitivity in identifying students experiencing elevated anxiety symptoms. These findings underscore the potential of machine learning techniques in enhancing mental health screening, particularly for digitally engaged populations such as college students.^{6,7,8}

Consistent with both prior literature and this study's statistical analyses, emotional distress indicators emerged as the most influential predictors in the model. Features such as "Exhausted (not physical)," "Hopeless," and "Very sad" had the highest impact on classification accuracy, reinforcing their clinical importance in identifying anxiety.^{1,2,3,5,6,9} These findings align with results from the Chi-square tests, which revealed significant associations between emotional distress variables and anxiety severity ($p < 0.001$). In contrast, demographic factors such as age and race had minimal predictive value, which corresponds with the ANOVA results and confidence interval analyses showing no significant differences in mean age or general health scores across anxiety levels.^{3,4,5}

The inclusion of diverse predictors-ranging from prior mental health history to current emotional states contributed to the model's robustness. Participants with a history of depression or current therapy engagement were more likely to report severe anxiety, confirming well-established comorbidities between anxiety and depression.^{1,2} Permutation feature importance further validated that emotional and psychological symptoms, rather than demographic variables, were the primary drivers of predictive performance. These insights emphasize the importance of incorporating both historical and current mental health indicators in future screening tools to improve diagnostic accuracy.^{1,3,5} Confidence intervals calculated for continuous variables added an additional layer of interpretability, showing that the mean values of age and general health across anxiety severity groups were tightly clustered and statistically indistinguishable. This reinforces their limited role in anxiety classification, despite being commonly collected variables in health assessments.^{3,4}

Despite promising outcomes, several limitations must be acknowledged. The reliance on self-reported survey data introduces potential recall and response bias.^{5,11} The absence of behavioral metrics from Facebook such as time spent, interaction frequency, or post content restricts opportunities for richer, behavior-based modeling.^{6,9} Additionally, while SMOTE was used to address class imbalance, synthetic oversampling may not fully represent the true data distribution.¹⁵ The modest sample size ($n = 579$) and limited demographic granularity, particularly in race and gender diversity, may constrain the generalizability of the results to broader populations.^{4,5} Ethical considerations are also essential. While the Streamlit application, "Anxiety Screener for Facebook Users," provides an accessible tool for early screening, it is not intended for clinical use.^{11,22}

FUTURE DIRECTIONS

To enhance the generalizability and practical utility of these findings, future research should adopt longitudinal study designs to capture how anxiety severity evolves over time. Tracking individuals across different time points would offer valuable insights into the temporal dynamics of mental health and improve predictive accuracy for future risk.^{4,5} Additionally, incorporating behavioral metrics from Facebook-such as time spent on the platform, posting frequency, and interaction patterns-could enrich the feature set and enable the modeling of digital phenotypes that better reflect real-world anxiety-related behaviors.^{6,8,9}

It is also important to expand the sample to include greater ethnic, socioeconomic, and geographic diversity, especially given the uneven burden of mental health conditions across demographic groups.^{1,3,5} A more representative dataset would improve external validity and

ensure that the model performs well across diverse subpopulations. Future studies may also explore multimodal data integration by combining survey responses with other sources, such as social media text analysis, wearable device outputs, or physiological markers, which have shown potential in emerging mental health research.^{6,7,10} This comprehensive approach could yield more nuanced and accurate assessments of mental health.

IMPLICATIONS

This study demonstrates the feasibility of employing machine learning models to identify college students at risk of anxiety using self-reported survey data. The final tuned KNN model, with its high recall and balanced classification metrics, is particularly well-suited for early identification in preventive and screening contexts.^{6,15} Such models could be integrated into campus wellness platforms, mental health apps, or telehealth services to assist healthcare professionals and counselors in prioritizing outreach to high-risk individuals.^{10,12} While these tools are not intended to replace clinical diagnoses, they offer cost-effective, scalable, and interpretable supplements to traditional screening methods.^{14,18} As mental health challenges continue to grow on college campuses, integrating data-driven approaches like this one could meaningfully enhance early detection, support, and intervention outcomes.^{1,5,11}

CONCLUSION

This study successfully demonstrated the application of supervised machine learning models-particularly a tuned K-Nearest Neighbors (KNN) classifier-for predicting anxiety severity among U.S. college students who use Facebook. By leveraging self-reported demographic, clinical, and emotional distress data, the tuned KNN model achieved high recall (92.7%) and an F1 score of 0.879, making it a strong candidate for scalable mental health screening. Emotional distress indicators such as “Exhausted (not physical),” “Hopeless,” and “Very sad” emerged as the most significant predictors, whereas demographic variables like age and race contributed minimally.

The deployment of the predictive model through a user-friendly Streamlit application highlights its real-world applicability in digital health environments. Although the study faced limitations such as reliance on self-reported data and lack of behavioral metrics, it lays a solid foundation for integrating machine learning into mental health screening initiatives on college campuses. Future research should focus on incorporating longitudinal data, digital behavior, and multimodal inputs to enhance model accuracy and generalizability. Overall, this work contributes to the growing field of data-driven mental health solutions and underscores the importance of early, accessible, and interpretable interventions for at-risk student populations.

REFERENCES

1. Zubair U, Khan MK, Albashari M. Link between excessive social media use and psychiatric disorders. *Ann Med Surg (Lond)*. 2023;85(4):875-878. Published 2023 Mar 27. doi:10.1097/MS9.0000000000000112
2. Lin LY, Sidani JE, Shensa A, et al. ASSOCIATION BETWEEN SOCIAL MEDIA USE AND DEPRESSION AMONG U.S. YOUNG ADULTS. *Depress Anxiety*. 2016;33(4):323-331. doi:10.1002/da.22466
3. Braghieri L, Levy R, Makarin A. Social Media and Mental Health. *American Economic Review*. Accessed July 30, 2025. <https://www.aeaweb.org/articles?id=10.1257%2Faer.20211218>.
4. Beyari H. The Relationship between Social Media and the Increase in Mental Health Problems. *Int J Environ Res Public Health*. 2023;20(3):2383. Published 2023 Jan 29. doi:10.3390/ijerph20032383
5. Journal of Adolescent Health. Associations between social media use and anxiety among adolescents: A systematic review study. *Journal of Adolescent Health* website. Published 2024. Accessed July 30, 2025. [https://www.jahonline.org/article/S1054-139X\(24\)00433-6/fulltext](https://www.jahonline.org/article/S1054-139X(24)00433-6/fulltext).
6. JAMA Network Open. Association between social media use and self-reported symptoms of depression in US adults. *JAMA Network Open* website. Published November 23, 2021. Accessed July 30, 2025. <https://jamanetwork.com/journals/jamanetworkopen/fullarticle/2786464>.
7. Woodward MJ, McGettrick CR, Dick OG, Ali M, Teeters JB. Time spent on social media and associations with mental health in young adults: examining TikTok, Twitter, Instagram, Facebook, YouTube, Snapchat, and Reddit. *J Technol Behav Sci*. Published January 13, 2025. Accessed July 30, 2025. <https://link.springer.com/article/10.1007/s41347-024-00474-y>
8. Naslund JA, Bondre A, Torous J, Aschbrenner KA. Social Media and Mental Health: Benefits, Risks, and Opportunities for Research and Practice. *J Technol Behav Sci*. 2020;5(3):245-257. doi:10.1007/s41347-020-00134-x
9. Khalaf AM, Alubied AA, Khalaf AM, Rifaey AA, Alubied A, Rifaey A. The impact of social media on the mental health of adolescents and young adults: a systematic review. *Cureus*. Published August 5, 2023. Accessed July 30, 2025. <https://www.cureus.com/articles/176889-the-impact-of-social-media-on-the-mental-health-of-adolescents-and-young-adults-a-systematic-review#!/>.
10. Alenezi A, Hamed W, Elhehe I, El-Etreby R. Association between Facebook Addiction, Depression, and Emotional Regulation among Women. *Healthcare (Basel)*. 2023;11(12):1701. Published 2023 Jun 9. doi:10.3390/healthcare11121701

11. Salma N, Bhuiyan F. Facebook addiction and its impact on depression: a cross-sectional study. *J Health Popul Nutr*. Published June 6, 2024. Accessed July 30, 2025. <https://jhpn.biomedcentral.com/articles/10.1186/s41043-024-00556-w>.
12. Braghieri L, Levy R, Makarin A. *Data and Code for: Social Media and Mental Health* [dataset]. Nashville, TN: American Economic Association; 2022. Distributed by: Inter-university Consortium for Political and Social Research; October 19, 2022. doi:10.3886/E175582V1
13. Stocker R, Tran T, Hammarberg K, Nguyen H, Rowe H, Fisher J. Patient Health Questionnaire 9 (PHQ-9) and General Anxiety Disorder 7 (GAD-7) data contributed by 13,829 respondents to a national survey about COVID-19 restrictions in Australia. *Psychiatry Res*. 2021;298:113792. doi:10.1016/j.psychres.2021.113792
14. Shensa A, Sidani JE, Dew MA, Escobar-Viera CG, Primack BA. Social Media Use and Depression and Anxiety Symptoms: A Cluster Analysis. *Am J Health Behav*. 2018;42(2):116-128. doi:10.5993/AJHB.42.2.11
15. Blagus R, Lusa L. SMOTE for high-dimensional class-imbalanced data. *BMC Bioinformatics*. 2013;14:106. Published 2013 Mar 22. doi:10.1186/1471-2105-14-106
16. Abraham A, Pedregosa F, Eickenberg M, et al. Machine learning for neuroimaging with scikit-learn. *Front Neuroinform*. 2014;8:14. Published 2014 Feb 21. doi:10.3389/fninf.2014.00014
17. Cho G, Yim J, Choi Y, Ko J, Lee SH. Review of Machine Learning Algorithms for Diagnosing Mental Illness. *Psychiatry Investig*. 2019;16(4):262-269. doi:10.30773/pi.2018.12.21.2
18. Iyortsuun NK, Kim SH, Jhon M, Yang HJ, Pant S. A Review of Machine Learning and Deep Learning Approaches on Mental Health Diagnosis. *Healthcare (Basel)*. 2023;11(3):285. Published 2023 Jan 17. doi:10.3390/healthcare11030285
19. Sarker IH. Machine Learning: Algorithms, Real-World Applications and Research Directions. *SN Comput Sci*. 2021;2(3):160. doi:10.1007/s42979-021-00592-x
20. Mansoor MA, Ansari KH. Early Detection of Mental Health Crises through Artificial-Intelligence-Powered Social Media Analysis: A Prospective Observational Study. *J Pers Med*. 2024;14(9):958. Published 2024 Sep 9. doi:10.3390/jpm14090958
21. Chadha A, Kaushik B. A Hybrid Deep Learning Model Using Grid Search and Cross-Validation for Effective Classification and Prediction of Suicidal Ideation from Social Network Data. *New Gener Comput*. 2022;40(4):889-914. doi:10.1007/s00354-022-00191-1
22. Streamlit. *A faster way to build and share data apps*. <https://streamlit.io>. Accessed July 30, 2025.

APPENDIX

Link to Code Repository:

The complete source code for data preprocessing, machine learning modeling, and the Streamlit application is available at: <https://github.com/suprajamedicherla/Anxiety-Screener-App/tree/main>

Import necessary libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency
from scipy.stats import f_oneway
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split, learning_curve, cross_val_score, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, roc_auc_score, roc_curve, RocCurveDisplay, auc
)
```

```
,
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import make_pipeline
from imblearn.over_sampling import SMOTE, SMOTENC
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import validation_curve
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import tensorflow as tf
import os
import random
import sys
import joblib
```

Data Loading

```
[ ] df = pd.read_csv("mental_health2425v8in.csv")
df.head()
```

	year_1	state_1	general_health	phq9_1	phq9_2	phq9_3	phq9_4	phq9_5	phq9_6	phq9_7	...	acha_12months_any_comp	sex	fulltime	international	race_1	race_2	race_3	race
0	2000	MD	Very Good	Several days	Several days	Not at all	Several days	Not at all	Not at all	More than half of the days	...		2 Female	Yes	No	NaN	NaN	NaN	Asi
1	2001	SC	Good	More than half of the days	More than half of the days	Nearly every day	Nearly every day	Nearly every day	Several days	Nearly every day	...		0 Female	Yes	No	NaN	NaN	NaN	Asi
2	1999	NJ	Good	Several days	Several days	More than half of the days	More than half of the days	Several days	Several days	Several days	...		2 Female	Yes	No	NaN	NaN	Hispanic or Latino	Asi
3	1998	NY	Good	Several days	Several days	Nearly every day	Nearly every day	Not at all	Several days	Several days	...		2 Female	Yes	No	White - not Hispanic (includes Middle Eastern)	NaN	NaN	White -

```
df.describe()
```

	year_1	phq9_1NUM	phq9_2NUM	phq9_3NUM	phq9_4NUM	phq9_5NUM	phq9_6NUM	phq9_7NUM	phq9_8NUM	phq9_9NUM	...	acha_12months_any_21NUM	acha_12months_any_22NUM	acha_12months_any_23NUM
count	579.000000	579.000000	579.000000	579.000000	579.000000	579.000000	579.000000	579.000000	579.000000	579.000000	...	579.000000	579.000000	579.000000
mean	1998.398964	1.02418	1.065630	1.343696	1.568221	1.226252	1.122625	1.188256	0.438687	0.395509	...	0.041451	0.017271	0.017271
std	5.403260	0.90334	0.935189	0.998792	0.968407	1.043484	1.042599	1.017562	0.765354	0.767448	...	0.199503	0.130393	0.130393
min	1971.000000	0.00000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000
25%	1998.000000	0.00000	0.000000	1.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000
50%	2000.000000	1.00000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.000000	0.000000	...	0.000000	0.000000	0.000000
75%	2001.000000	2.00000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	1.000000	1.000000	...	0.000000	0.000000	0.000000
max	2003.000000	3.00000	3.000000	3.000000	3.000000	3.000000	3.000000	3.000000	3.000000	3.000000	...	1.000000	1.000000	1.000000

8 rows x 49 columns

The df.describe() output provides a summary of 49 numerical columns from a dataset of 579 U.S. college students, showing descriptive statistics for key mental health and demographic variables. Overall, the statistics provide insight into the distribution, central tendency, and variability of depression-related symptoms and other health indicators in the dataset.

```
[ ] df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 579 entries, 0 to 578
Columns: 118 entries, year_1 to race_6
dtypes: int64(49), object(69)
memory usage: 533.9+ KB
```

The output of df.info() reveals that the dataset comprises 579 records and 118 columns. Out of these, 49 columns are of numerical type (int64), which likely represent encoded survey responses, psychological scores (such as PHQ-9 and GAD-7), and other binary or ordinal variables. The remaining 69 columns are of type object, typically indicating text-based data, including categorical survey responses, condition names, or other non-numeric inputs that have not yet been encoded. The dataset occupies approximately 534 KB of memory.

Data Cleaning and Preparation

```
# Dropping Unnecessary Columns
data= df.drop(['phq9_1', 'phq9_2', 'phq9_3', 'phq9_4', 'phq9_5', 'phq9_6', 'phq9_7', 'phq9_8', 'phq9_9', 'phq9_1NUM', 'phq9_2NUM', 'phq9_3NUM',
'phq9_4NUM', 'phq9_5NUM', 'phq9_6 NUM', 'phq9_7NUM', 'phq9_8NUM', 'phq9_9NUM', 'gad7_1', 'gad7_2', 'gad7_3', 'gad7_4', 'gad7_5', 'gad7_6',
'gad7_7', 'gad7_1NUM', 'gad7_2NUM', 'gad7_3NUM', 'gad7_4NUM', 'gad7_5NUM', 'gad7_6NUM', 'gad7_7NUM', 'acha_12months_any_1',
'acha_12months_any_2', 'acha_12months_any_3', 'acha_12months_any_4', 'acha_12months_any_5', 'acha_12months_any_6',
'acha_12months_any_7', 'acha_12months_any_8', 'acha_12months_any_9', 'acha_12months_any_10', 'acha_12months_any_11',
'acha_12months_any_12', 'acha_12months_any_13', 'acha_12months_any_14', 'acha_12months_any_15', 'acha_12months_any_16',
'acha_12months_any_17', 'acha_12months_any_18', 'acha_12months_any_19', 'acha_12months_any_20', 'acha_12months_any_21',
'acha_12months_any_22', 'acha_12months_any_23', 'acha_12months_any_24', 'acha_12months_any_25',
'acha_12months_any_26', 'acha_12months_any_27', 'acha_12months_any_28', 'acha_12months_any_29', 'acha_12months_any_1NUM',
'acha_12months_any_2NUM', 'acha_12months_any_3NUM', 'acha_12months_any_4NUM', 'acha_12months_any_5NUM', 'acha_12months_any_6NUM',
'acha_12months_any_7NUM', 'acha_12months_any_8NUM', 'acha_12months_any_9NUM', 'acha_12months_any_10NUM', 'acha_12months_any_11NUM',
'acha_12months_any_12NUM', 'acha_12months_any_13NUM', 'acha_12months_any_14NUM', 'acha_12months_any_15NUM', 'acha_12months_any_16NUM',
'acha_12months_any_17NUM', 'acha_12months_any_18NUM', 'acha_12months_any_19NUM', 'acha_12months_any_20NUM', 'acha_12months_any_21NUM',
'acha_12months_any_22NUM', 'acha_12months_any_23NUM', 'acha_12months_any_24NUM', 'acha_12months_any_25NUM', 'acha_12months_any_26NUM',
'acha_12months_any_27NUM', 'acha_12months_any_28NUM', 'acha_12months_any_29NUM', 'international', 'fulltime',
'acha_services_1', 'acha_services_3', ], axis=1)
```

To simplify the dataset and enhance model efficiency, numerous columns were removed during preprocessing. This included individual question responses from the PHQ-9 and GAD-7 scales, as their total severity scores were already computed and used as consolidated predictors. Similarly, the `acha_12months_any_*` variables, which captured detailed individual health conditions and their numeric equivalents, were dropped to reduce dimensionality, since their information was either redundant or represented in summary features. Additional columns such as `international`, `fulltime`, and underutilized service fields like `acha_services_1` and `acha_services_3` were excluded due to limited relevance, low variance, or high rates of missing data.

```
# Rename column names
data.rename(columns={
    'acha_depression': 'Previously_Diagnosed_Depression',
    'acha_services_2': 'In_Therapy_for_Depression',
    'acha_12months_any_comp': 'Total_Health_Issues_Score',
    'acha_12months_times_1': 'Hopeless',
    'acha_12months_times_2': 'Overwhelmed',
    'acha_12months_times_3': 'Exhausted (not physical)',
    'acha_12months_times_4': 'Very sad',
    'acha_12months_times_5': 'Depressed - hard to function',
    'acha_12months_times_6': 'Seriously considered suicide',
    'acha_12months_times_7': 'Attempted suicide'
}, inplace=True)
```

In this step, several column names were renamed to improve clarity and interpretability of the dataset.

Feature Engineering and Encoding

```
# Calculate Age
data['Age'] = 2022 - data['year_1']
```

The Age of each respondent is calculated by subtracting the value in the `year_1` column from the year 2022. The `year_1` variable represents the respondent's year of birth, so this operation estimates their age at the time of data analysis or survey administration. This derived Age variable is important for subsequent analyses, as age can be a significant factor in understanding

trends related to mental health severity, anxiety, and other health-related variables among college students.

```
# The column was mapped to numeric values as General_Health for analysis
df_data2 = {
    'Poor': 1, 'Fair': 2, 'Good': 3, 'Very Good': 4, 'Excellent': 5
}
data['General_Health'] = data['general_health'].map(df_data2)
data.drop(columns=['general_health'], inplace=True)
```

In this step, the `general_health` column which originally contained categorical text responses such as "Poor," "Fair," and "Excellent" was converted into a numeric format for analysis. Each health category was mapped to an ordinal scale from 1 to 5, with 1 representing the lowest health status ("Poor") and 5 the highest ("Excellent"). This transformation, stored in the new column `General_Health`.

```
# Columns were transformed from categorical frequency responses into a numeric
hopeless = {
    "Never": 0,
    "1-2 times": 1,
    "3-4 times": 2,
    "5-6 times": 3,
    "7-8 times": 4,
    "9-10 times": 5,
    "11 or more times": 6
}
data['Hopeless'] = data['Hopeless'].map(hopeless)
```

```
overwhelmed = {
    "Never": 0,
    "1-2 times": 1,
    "3-4 times": 2,
    "5-6 times": 3,
    "7-8 times": 4,
    "9-10 times": 5,
    "11 or more times": 6
}
data['Overwhelmed'] = data['Overwhelmed'].map(overwhelmed)
```

```
exhausted = {
    "Never": 0,
    "1-2 times": 1,
    "3-4 times": 2,
    "5-6 times": 3,
    "7-8 times": 4,
    "9-10 times": 5,
    "11 or more times": 6
}

data['Exhausted (not physical)'] = data['Exhausted (not physical)'].map(exhausted)
```



```

very_sad = {
  "Never": 0,
  "1-2 times": 1,
  "3-4 times": 2,
  "5-6 times": 3,
  "7-8 times": 4,
  "9-10 times": 5,
  "11 or more times": 6
}
data['Very sad'] = data['Very sad'].map(very_sad)

```

```

depressed = {
  "Never": 0,
  "1-2 times": 1,
  "3-4 times": 2,
  "5-6 times": 3,
  "7-8 times": 4,
  "9-10 times": 5,
  "11 or more times": 6
}
data['Depressed - hard to function'] = data['Depressed - hard to function'].map(depressed)

```

```

considered_suicide = {
  "Never": 0,
  "1-2 times": 1,
  "3-4 times": 2,
  "5-6 times": 3,
  "7-8 times": 4,
  "9-10 times": 5,
  "11 or more times": 6
}
data['Seriously considered suicide'] = data['Seriously considered suicide'].map(considered_suicide)

```

```

attempted_suicide = {
  "Never": 0,
  "1-2 times": 1,
  "3-4 times": 2,
  "5-6 times": 3,
  "7-8 times": 4,
  "9-10 times": 5,
  "11 or more times": 6
}
data['Attempted suicide'] = data['Attempted suicide'].map(attempted_suicide)

```

To prepare the emotional distress indicators for quantitative analysis and machine learning modeling, categorical frequency responses in multiple columns were converted into ordinal numeric values. Specifically, the variables Hopeless, Overwhelmed, Exhausted (not physical), Very sad, Depressed - hard to function, Seriously considered suicide, and Attempted suicide were all mapped to a consistent numerical scale ranging from 0 ("Never") to 6 ("11 or more times"). This transformation enabled the model to interpret the frequency of distress symptoms in a structured way, preserving the ordinal nature of the data while making it suitable for correlation analysis, statistical testing, and predictive modeling.

```
# Convert 'sex' column to : Male as 0 and Female as 1
data['sex'] = data['sex'].map({'Male': 0, 'Female': 1})
```

To facilitate binary classification, the sex column was converted from categorical labels to numeric values. Specifically, 'Male' was mapped to 0 and 'Female' to 1. This encoding allows the model to interpret sex as a binary predictor without introducing unnecessary complexity, supporting its use in statistical tests and predictive analysis related to anxiety severity.

```
# Mapping the PHQ-9 severity categories to numerical scores for easier analysis and modeling
phq9_severity_score = {
    'Minimal depression': 0,
    'Mild depression': 1,
    'Moderate depression': 2,
    'Moderately severe depression': 3,
    'Severe depression': 4
}
data['Depression_Severity_Score'] = data['phq9_severity'].map(phq9_severity_score)
```

To streamline analysis and enable effective use in machine learning models, the PHQ-9 depression severity categories were mapped to numerical scores. The categorical levels ranging from “Minimal depression” to “Severe depression” were assigned values from 0 to 4 in ascending order of severity. This transformation created a new variable, Depression_Severity_Score.

```
# Mapping the GAD-7 anxiety severity categories to numerical scores
Anxiety_severity_score = {
    'minimal anxiety': 0,
    'mild anxiety': 1,
    'moderate anxiety': 2,
    'severe anxiety': 3
}
data['Anxiety_Severity_Score'] = data['gad7_severity'].map(Anxiety_severity_score)
```

To facilitate quantitative analysis and model training, the GAD-7 anxiety severity categories were converted into numerical scores. Each severity level from “minimal anxiety” to “severe anxiety” was mapped to an integer value from 0 to 3. This transformation resulted in a new column, Anxiety_Severity_Score, enabling easier computation and interpretation within statistical analyses and machine learning algorithms.

```
# Convert 'Previously_Diagnosed_Depression' from categorical 'Yes'/'No' to numerical values
# 'No': 0, 'Yes': 1
# Fill any missing values with 2 to indicate unknown or missing diagnosis status

data['Previously_Diagnosed_Depression'] = data['Previously_Diagnosed_Depression'].map({
    'No': 0,
    'Yes': 1
})
data['Previously_Diagnosed_Depression'] = data['Previously_Diagnosed_Depression'].fillna(2)
```

To prepare the `Previously_Diagnosed_Depression` variable for modeling, categorical responses were converted to numeric format: "No" was mapped to 0 and "Yes" to 1. Any missing values in this column were filled with 2, representing an unknown or unreported diagnosis status. This approach preserves all available data while distinguishing respondents who did not disclose their depression history, reducing potential bias from data omission.

```
# Map 'In_Therapy_for_Depression' from categorical to numeric: 'No' = 0, 'Yes' = 1
# Fill missing values with 2 (indicating unknown/missing)

data['In_Therapy_for_Depression'] = data['In_Therapy_for_Depression'].map({'No': 0, 'Yes': 1}).fillna(2).astype(int)
```

To enable analysis and modeling, the `In_Therapy_for_Depression` column was encoded by mapping “No” responses to 0 and “Yes” responses to 1. Missing or unreported values were assigned a value of 2 to indicate an unknown therapy status. This approach maintains the completeness of the dataset while accounting for ambiguity in participant responses, ensuring that all records could be used in the modeling phase.

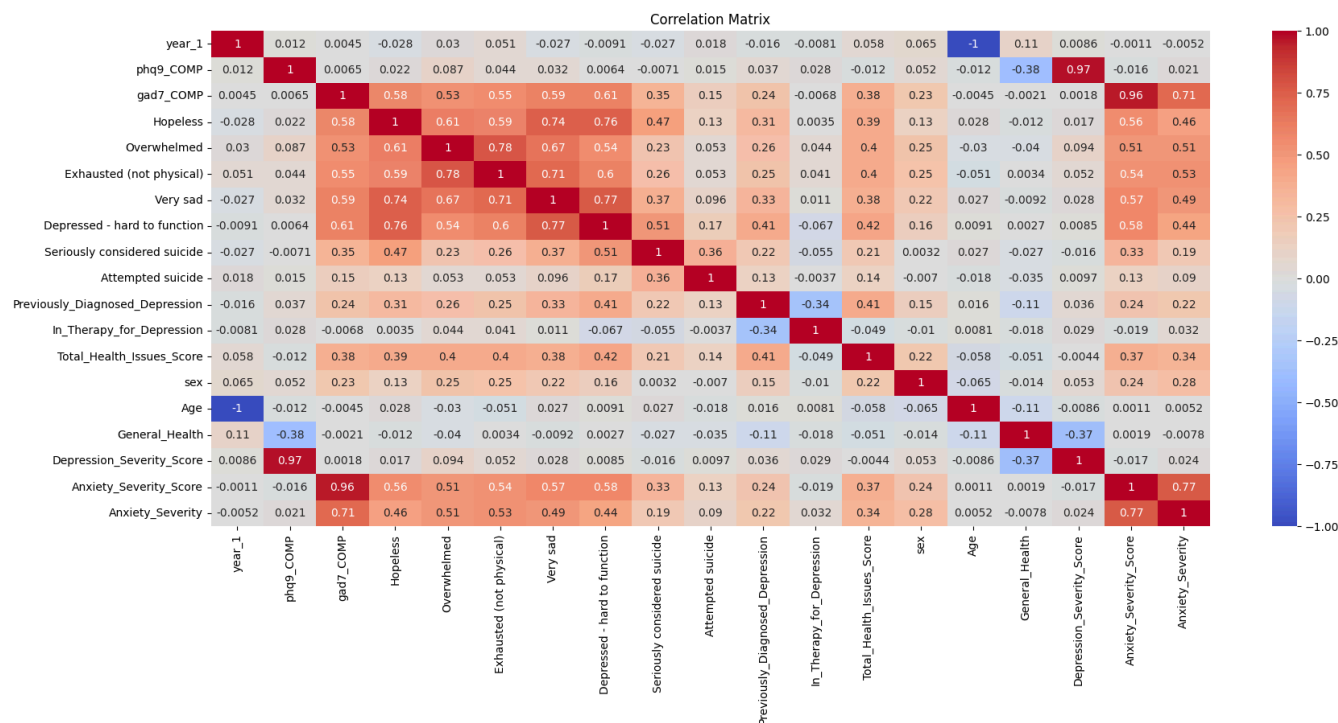
```
# Create binary target variable from Anxiety Severity Score
# 0 = No Anxiety, 1 = Mild to Severe Anxiety
data['Anxiety_Severity'] = data['Anxiety_Severity_Score'].apply(lambda x: 0 if x == 0 else 1)
```

To address the model’s limited ability to distinguish between adjacent anxiety severity levels particularly between mild (1) and moderate (2) the `Anxiety_Severity_Score` was simplified into a binary target variable called `Anxiety_Severity`. Individuals with a score of 0 (minimal anxiety) were labeled as 0, while those with scores from 1 to 3 (mild to severe anxiety) were grouped and labeled as 1. This binary classification approach improved the model’s performance by focusing on a clear distinction between having anxiety and not having anxiety, instead of trying to separate between mild, moderate, and severe levels.

Exploratory Data Analysis (EDA)

```
plt.figure(figsize=(20, 8))
sns.heatmap(data.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```

To examine the relationships between numerical variables in the dataset, a correlation matrix heatmap was generated.



The correlation matrix represents the strength and direction of relationships between variables in the dataset. Mental health symptoms such as feeling "Depressed - hard to function", "Overwhelmed", and "Hopeless" are highly correlated with both depression and anxiety scores, suggesting they are key indicators of psychological distress. Strong correlations are also observed among suicide-related variables like "Seriously considered suicide", "Attempted suicide", and "Depressed - hard to function", reinforcing their close association.

General_Health is negatively correlated with both Depression_Severity_Score or phq9_COMP, indicating that individuals with poorer mental health perceive their general health as worse. Lastly, sex and age show very weak or no meaningful correlation with mental health indicators, implying these demographic factors may not directly influence the severity scores.

```

emotions = [
    'Hopeless',
    'Overwhelmed',
    'Exhausted (not physical)',
    'Very sad',
    'Depressed - hard to function',
    'Seriously considered suicide',
    'Attempted suicide'
]

long_df = pd.melt(data, value_vars=emotions,
                  var_name='Emotion', value_name='Frequency')

freq_counts = (long_df
               .groupby(['Emotion', 'Frequency'])
               .size()
               .reset_index(name='Count'))

freq_labels = {
    0: '0 = Never',
    1: '1 = 1-2 times',
    2: '2 = 3-4 times',
    3: '3 = 5-6 times',
    4: '4 = 7-8 times',
    5: '5 = 9-10 times',
    6: '6 = 11+ times'
}
freq_counts['Frequency_Label'] = freq_counts['Frequency'].map(freq_labels)

```

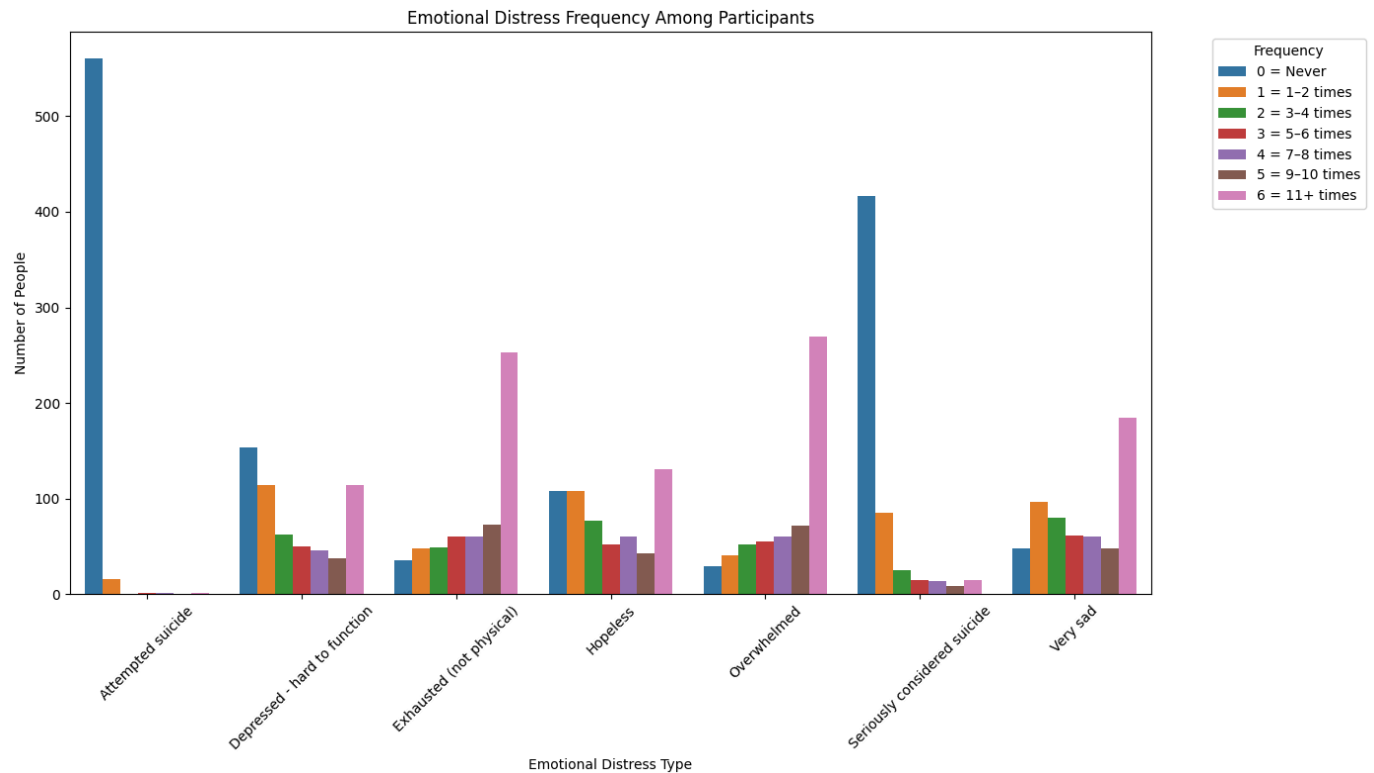
```

freq_order = [
    '0 = Never', '1 = 1-2 times', '2 = 3-4 times', '3 = 5-6 times',
    '4 = 7-8 times', '5 = 9-10 times', '6 = 11+ times'
]

plt.figure(figsize=(14, 8))
sns.barplot(
    data=freq_counts,
    x='Emotion',
    y='Count',
    hue='Frequency_Label',
    hue_order=freq_order,
    ci=None
)

plt.title('Emotional Distress Frequency Among Participants')
plt.xlabel('Emotional Distress Type')
plt.ylabel('Number of People')
plt.legend(title='Frequency', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



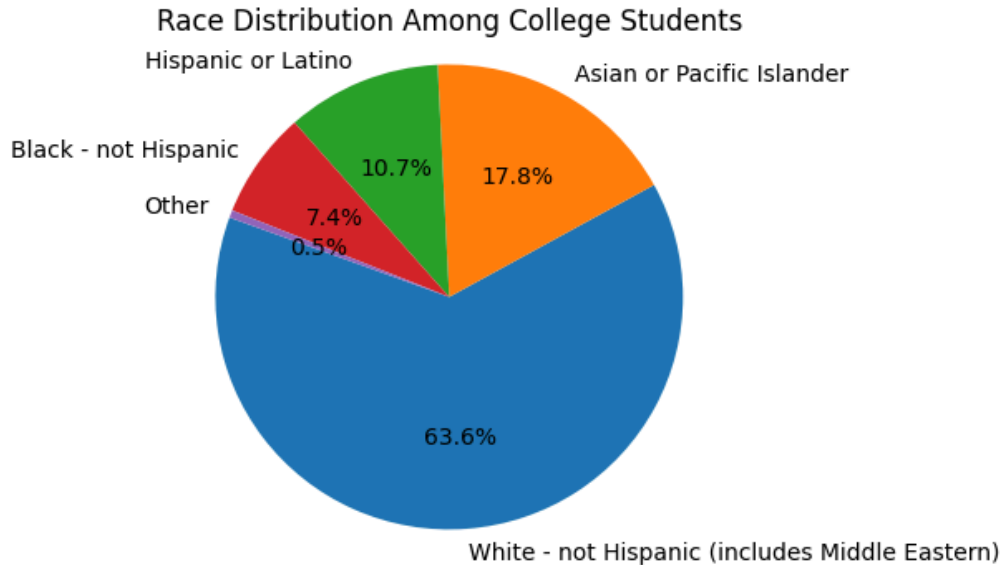
The bar plot illustrates the frequency of various emotional distress experiences among participants. Notably, the majority reported never attempting suicide or seriously considering suicide, indicating these severe responses were less common. However, high frequencies (11+ times) were reported for feelings of exhaustion (not physical), overwhelm, and sadness, suggesting these are more prevalent and persistent emotional struggles. A significant number also reported experiencing feelings like hopelessness and difficulty functioning due to depression multiple times. This distribution highlights that while extreme behaviors like suicidal actions are less frequent, chronic emotional distress is widespread among participants and warrants attention in mental health assessments and interventions.

```
data['Race'] = data[['race_1', 'race_2', 'race_3', 'race_4', 'race_5', 'race_6']].bfill(axis=1).iloc[:, 0]
data.drop(columns=['race_1', 'race_2', 'race_3', 'race_4', 'race_5', 'race_6'], inplace=True)
data['Race'] = data['Race'].astype('category')
```

```
race_counts = data['Race'].value_counts()

plt.figure(figsize=(4, 4))
plt.pie(race_counts.values, labels=race_counts.index, autopct='%1.1f%%', startangle=160)
plt.title('Race Distribution Among College Students')
plt.axis('equal')
plt.show()
```

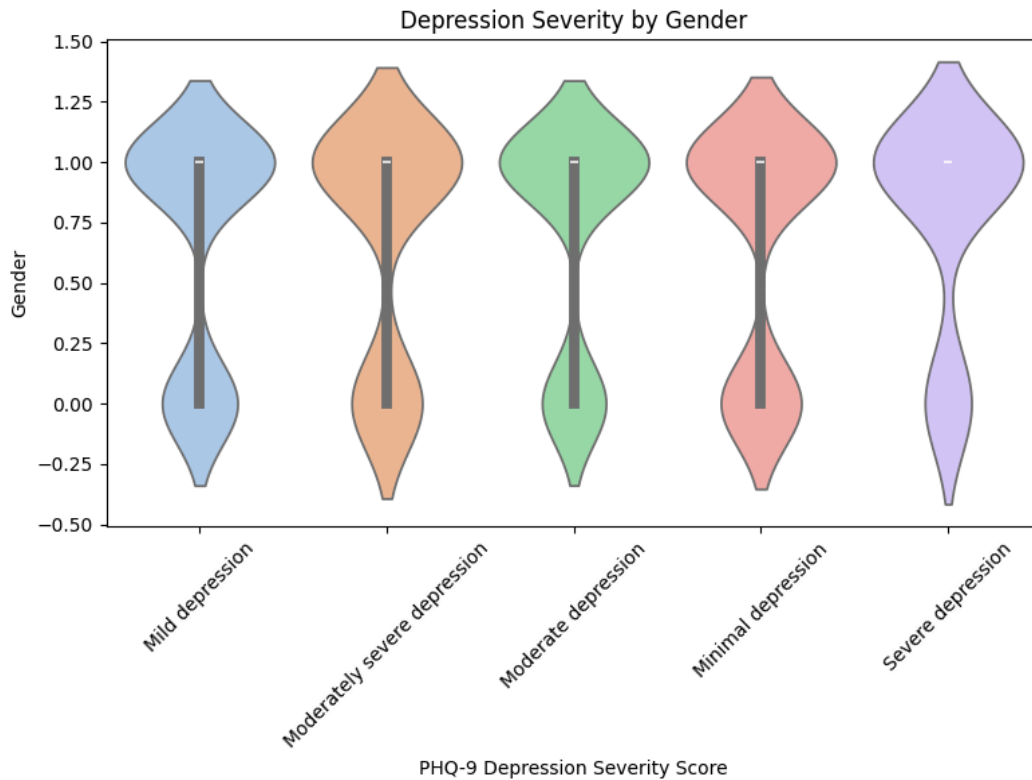
To streamline race-related data for analysis, the values from six separate race columns (race_1 to race_6) were consolidated into a single Race column. This was achieved using backward fill (bfill), which selects the first non-null value across the columns for each participant—ensuring each individual is assigned a single race category



The pie chart illustrates the racial distribution among college students in the dataset. The majority of students identify as White - not Hispanic (including Middle Eastern), comprising 63.6% of the total population. This is followed by Asian or Pacific Islander students at 17.8%, and Hispanic or Latino students at 10.7%. Black - not Hispanic students make up 7.4%, while those identifying as “Other” represent only 0.5%. This distribution indicates a predominance of White students, with comparatively lower representation from other racial and ethnic groups.

```
plt.figure(figsize=(8,6))
sns.violinplot(x='phq9_severity', y='sex', data=data, palette='pastel')
plt.title('Depression Severity by Gender')
plt.xlabel('PHQ-9 Depression Severity Score')
plt.ylabel('Gender')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

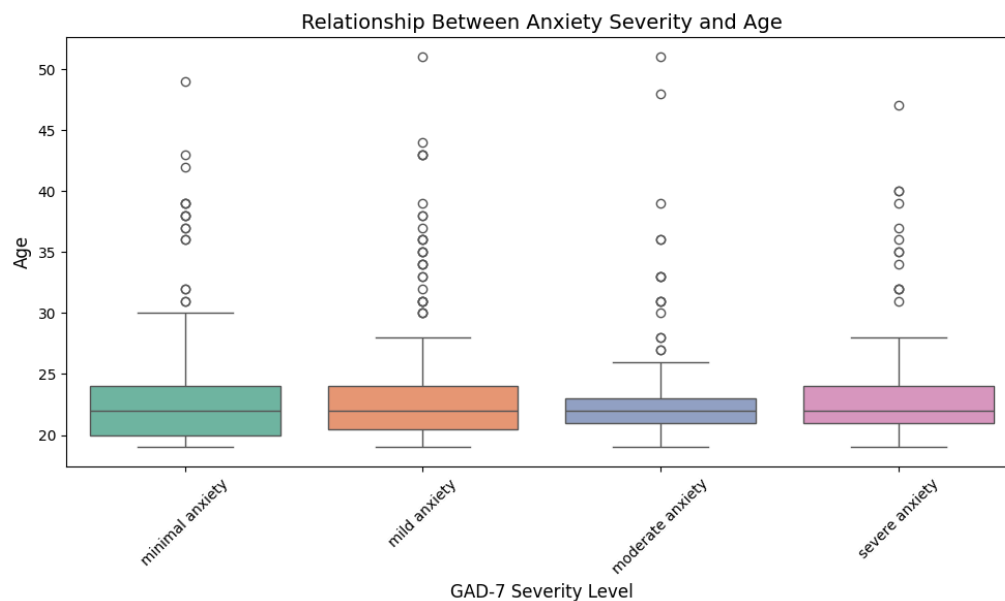
To examine the relationship between depression severity and gender, a violin plot was generated using the phq9_severity scores and the sex variable. This plot visualizes the distribution of depression levels separately for males and females.



The violin plot displays the distribution of PHQ-9 (depression severity scores) across genders. Each depression category-ranging from minimal to severe-is represented by a violin shape, showing the density of participants for each gender (where 0 likely represents males and 1 represents females). Overall, the distribution appears relatively balanced across genders for each severity level, though the female group shows slightly higher density in categories like moderate and moderately severe depression. This suggests that while both genders experience various levels of depression, females may be more represented in higher severity categories. The plot provides a visual cue that depression severity is prevalent across both genders but may impact females more frequently or intensely in certain categories.

```
plt.figure(figsize=(10, 6))
sns.boxplot(
    x='gad7_severity',
    y='Age',
    hue='gad7_severity',
    data=data,
    palette='Set2',
    legend=False
)
plt.title('Relationship Between Anxiety Severity and Age', fontsize=14)
plt.xlabel('GAD-7 Severity Level', fontsize=12)
plt.ylabel('Age', fontsize=12)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```


A boxplot was used to explore the relationship between participants' age and their GAD-7 anxiety severity levels. This visualization displays the distribution, median, and spread of ages within each anxiety category (minimal, mild, moderate, severe).

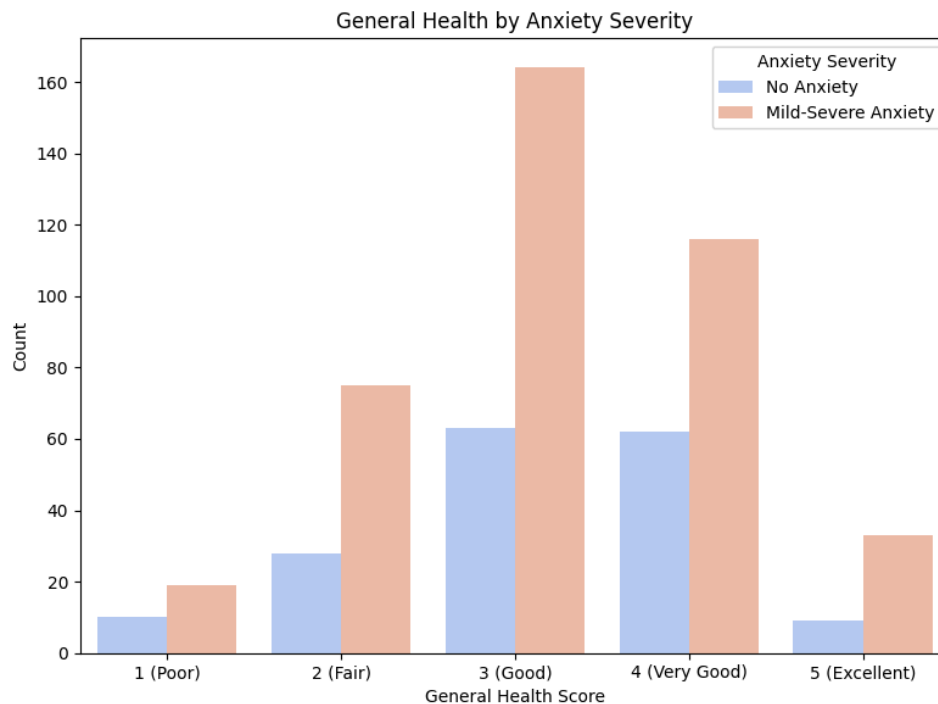


The boxplot illustrates the relationship between age and GAD-7 (anxiety severity) levels-ranging from minimal to severe anxiety. Across all categories, the age distribution remains fairly consistent, with the median age hovering around 21 to 22 years. The interquartile ranges (IQRs) are also similar, indicating that the majority of respondents in each anxiety category fall within the same age range, typically early adulthood. There are several outliers above age 30 in all severity groups, suggesting that while most participants are college-aged, a small number of older individuals also experience varying levels of anxiety. Notably, no clear trend emerges between age and increasing anxiety severity; that is, age does not appear to be a strong differentiator across anxiety categories. This suggests that within this college population, anxiety severity is relatively independent of age.

```
health = {
    1: 'Poor',
    2: 'Fair',
    3: 'Good',
    4: 'Very Good',
    5: 'Excellent'
}

plt.figure(figsize=(8, 6))
sns.countplot(data=data, x='General_Health', hue='Anxiety_Severity', palette='coolwarm')
plt.title('General Health by Anxiety Severity')
plt.xlabel('General Health Score')
plt.ylabel('Count')
plt.legend(title='Anxiety Severity', labels=['No Anxiety', 'Mild-Severe Anxiety'])
tick_labels = [f'{i} ({health[i]})' for i in sorted(health.keys())]
plt.xticks(ticks=range(len(tick_labels)), labels=tick_labels)
plt.tight_layout()
plt.show()
```

A count plot was generated to examine the relationship between self-reported general health and anxiety severity. General health scores ranged from 1 (Poor) to 5 (Excellent), and participants were grouped by binary anxiety status (No Anxiety vs. Mild-to-Severe Anxiety).



The bar plot shows the distribution of general health scores among individuals with and without mild to severe anxiety. Overall, individuals with mild-severe anxiety are more prevalent across all health categories compared to those with no anxiety. Notably, the highest number of mild-severe anxiety cases are seen in participants who rated their health as “Good” and “Very Good”, with “Good” showing the peak count.

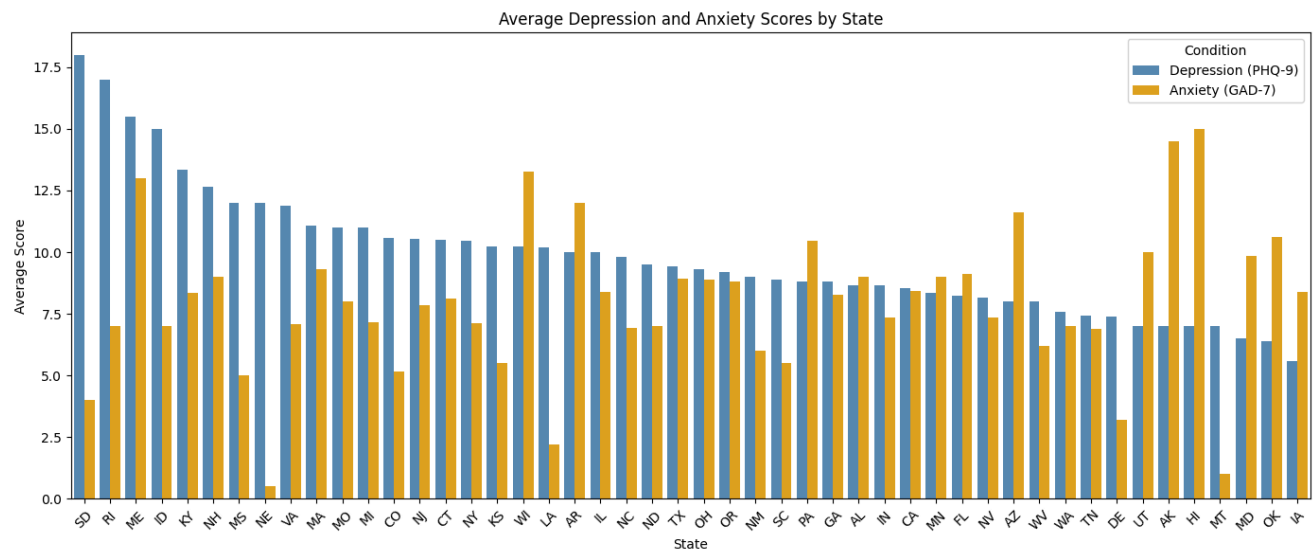
Interestingly, anxiety is not confined to participants with poor health. Even among those who rate their health as “Very Good” or “Excellent”, many still report experiencing anxiety. Meanwhile, those reporting no anxiety are mostly concentrated in the “Good” and “Very Good” categories as well, though in significantly smaller numbers. This suggests that self-perceived general health alone does not strongly protect against anxiety. While poorer health aligns with increased anxiety risk, many individuals with average or even high health ratings also experience significant anxiety symptoms.

```

state_avg = data.groupby('state_1')[['phq9_COMP', 'gad7_COMP']].mean().reset_index()
state_avg = state_avg.sort_values(by='phq9_COMP', ascending=False)
state_melted = pd.melt(state_avg, id_vars='state_1', value_vars=['phq9_COMP', 'gad7_COMP'],
                      var_name='Condition', value_name='Average Score')
state_melted['Condition'] = state_melted['Condition'].replace({
    'phq9_COMP': 'Depression (PHQ-9)',
    'gad7_COMP': 'Anxiety (GAD-7)'
})
custom_palette = {
    'Depression (PHQ-9)': '#488BBE',
    'Anxiety (GAD-7)': '#FFB000'
}
# Plot
plt.figure(figsize=(14, 6))
sns.barplot(data=state_melted, x='state_1', y='Average Score', hue='Condition', palette=custom_palette)
plt.title('Average Depression and Anxiety Scores by State')
plt.xlabel('State')
plt.ylabel('Average Score')
plt.xticks(rotation=45)
plt.legend(title='Condition')
plt.tight_layout()
plt.show()

```

This visualization compares the average PHQ-9 (Depression) and GAD-7 (Anxiety) scores across U.S. states represented in the dataset. By calculating mean composite scores for each mental health condition at the state level, the chart highlights geographic variation in reported mental health symptoms.



The bar chart displays the average depression (PHQ-9) and anxiety (GAD-7) scores by U.S. state, offering a comparative view of mental health conditions across different regions. Overall, depression scores are generally higher than anxiety scores in most states. States like SD (South Dakota), IN (Indiana), and ME (Maine) show the highest average depression scores, with values nearing or exceeding 15. In contrast, states such as WY (Wyoming), MS (Mississippi), and WV (West Virginia) exhibit significantly higher anxiety scores compared to their depression scores. On the other end, states like HI (Hawaii) and MT (Montana) show relatively low scores for both conditions.

Descriptive statistics

Chi-Square test:

```
# For Anxiety using chi2 for categorical values
categorical_vars = [
    'sex',
    'Previously_Diagnosed_Depression',
    'In_Therapy_for_Depression', 'Total_Health_Issues_Score', 'Depression_Severity_Score', 'Hopeless', 'Overwhelmed', 'Exhausted (not physical)',
    'Very sad', 'Depressed - hard to function',
    'Seriously considered suicide', 'Attempted suicide', 'Race'
]
results = []
for var in categorical_vars:
    contingency = pd.crosstab(data[var], data['gad7_severity'])
    chi2, p, dof, expected = chi2_contingency(contingency)
    results.append({'Variable': var, 'p-value': p})
chi_square_results = pd.DataFrame(results)
chi_square_results = chi_square_results.sort_values(by='p-value')
chi_square_results['Significant (<0.05)'] = chi_square_results['p-value'] < 0.05
chi_square_results
```

The chi-square test of independence was conducted to examine the relationship between various categorical variables and anxiety severity levels, as measured by the GAD-7 scale. The analysis included demographic factors (such as sex and race), clinical history (like previous diagnosis or therapy for depression), and emotional or psychological indicators (such as feelings of hopelessness, exhaustion, sadness, or suicidal thoughts).

	Variable	p-value	Significant (<0.05)
9	Depressed - hard to function	2.675725e-47	True
8	Very sad	1.140630e-41	True
7	Exhausted (not physical)	1.484494e-38	True
5	Hopeless	5.617337e-37	True
6	Overwhelmed	3.403522e-34	True
3	Total_Health_Issues_Score	4.244773e-20	True
10	Seriously considered suicide	4.971903e-13	True
0	sex	2.559304e-10	True
1	Previously_Diagnosed_Depression	8.383576e-08	True
2	In_Therapy_for_Depression	2.329720e-03	True
11	Attempted suicide	1.702317e-02	True
12	Race	6.173724e-02	False
4	Depression_Severity_Score	1.829259e-01	False

The chi-square tests were conducted to examine the association between categorical variables and anxiety severity levels (GAD-7 severity). The chi-square analysis revealed that several emotional distress variables—including “Depressed - hard to function,” “Very sad,” “Exhausted (not physical),” “Hopeless,” and “Overwhelmed”—were highly significantly associated with

anxiety severity levels ($p < 0.001$), suggesting a strong link between these emotional states and elevated anxiety. Additionally, health-related variables such as Total Health Issues Score and a history of depression diagnosis were also significantly related to anxiety severity. Demographic and treatment-related variables like sex and being in therapy for depression showed significant associations as well, indicating potential gender and care-seeking differences in anxiety outcomes. On the other hand, Race and Depression Severity Score did not show statistically significant associations ($p > 0.05$), suggesting that these factors may not have a strong influence on anxiety severity.

One-Way ANOVA:

```
# For Anxiety using Anova for numerical values
numeric_vars = [
    'Age',
    'General_Health'
]
target = 'gad7_severity'
results = []
for var in numeric_vars:
    # Drop missing values
    subset = data[[var, target]].dropna()
    # Group values by each category of target
    groups = [group[var].values for _, group in subset.groupby(target)]
    # Run ANOVA
    if len(groups) > 1:
        f_stat, p = f_oneway(*groups)
        results.append({'Variable': var, 'p-value': p, 'Significant (<0.05)': p < 0.05})
    else:
        results.append({'Variable': var, 'p-value': None, 'Significant (<0.05)': False})
anova_results = pd.DataFrame(results).sort_values(by='p-value', na_position='last').reset_index(drop=True)
anova_results
```

A one-way ANOVA was conducted to determine whether there were significant differences in the mean values of numerical variables across different levels of anxiety severity (as measured by GAD-7). The analysis included two continuous predictors: Age and General Health.

	Variable	p-value	Significant (<0.05)
0	Age	0.618836	False
1	General_Health	0.984618	False

The results indicated that neither variable showed a statistically significant relationship with anxiety severity (p-values of 0.6188 and 0.9846, respectively). This suggests that, within this dataset, participants' age and their self-reported general health do not significantly influence or vary by the severity of anxiety symptoms. Therefore, these numerical factors may not be strong predictors of anxiety levels in this college student population.

```

import scipy.stats as stats

ci_results = []

for var in numeric_vars:
    for level in data[target].dropna().unique():
        group_data = data[data[target] == level][var].dropna()
        n = len(group_data)
        mean = np.mean(group_data)
        std = np.std(group_data, ddof=1)
        stderr = std / np.sqrt(n)
        ci_low, ci_high = stats.t.interval(0.95, df=n-1, loc=mean, scale=stderr)
        ci_results.append({
            'Variable': var,
            'GAD7_Severity': level,
            'Mean': round(mean, 2),
            '95% CI Lower': round(ci_low, 2),
            '95% CI Upper': round(ci_high, 2),
            'N': n
        })

ci_df = pd.DataFrame(ci_results)
ci_df = ci_df.sort_values(by=['Variable', 'GAD7_Severity']).reset_index(drop=True)
ci_df

```

The confidence interval analysis revealed minimal differences in mean age and general health between students with minimal anxiety and those experiencing mild to severe anxiety. The average age was 23.56 for the minimal anxiety group and 23.62 for the higher anxiety group, with overlapping 95% confidence intervals ([22.77, 24.35] and [23.09, 24.15], respectively). Similarly, general health scores averaged 3.19 and 3.17 for the two groups, again with overlapping confidence intervals ([3.04, 3.33] and [3.07, 3.26]). These overlaps suggest that neither age nor general health differs significantly between anxiety severity groups, supporting earlier ANOVA findings and indicating that these continuous variables are not strong predictors of anxiety severity in this dataset.

Model Development

Feature Selection and Data Transformation

```

# predictors and target
predictors = [
    'sex', 'Age', 'Race', 'Previously_Diagnosed_Depression',
    'Total_Health_Issues_Score', 'In_Therapy_for_Depression',
    'Hopeless', 'Very_sad', 'Depression_Severity_Score',
    'Overwhelmed', 'Exhausted (not physical)',
    'Depressed - hard to function', 'Attempted suicide'
]
target = 'Anxiety_Severity'

```

```

# Select data and drop missing rows
model_input_df = data[predictors + [target]].dropna()

X = model_input_df[predictors]
y = model_input_df[target]

# Define categorical and numerical columns properly
categorical_cols = ['sex', 'Race']
numeric_cols = [
    'Age', 'Previously_Diagnosed_Depression', 'In_Therapy_for_Depression',
    'Total_Health_Issues_Score', 'Hopeless', 'Very_sad', 'Depression_Severity_Score',
    'Overwhelmed', 'Exhausted (not physical)', 'Depressed - hard to function', 'Attempted suicide'
]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

```

```

# Create preprocessing pipeline
preprocessor = ColumnTransformer([
    ("cat", OneHotEncoder(drop='first', handle_unknown='ignore'), categorical_cols),
    ("num", StandardScaler(), numeric_cols)
])

```

To model anxiety severity, a set of predictors was selected based on demographic, clinical, and psychological features. The predictors include both categorical variables (e.g., sex and race) and numerical variables such as age, previously diagnosed depression, depressive symptoms, and emotional distress indicators (e.g., hopeless, very sad, exhausted). The target variable for prediction is Anxiety Severity. Prior to modeling, the dataset was cleaned by dropping any rows with missing values for the selected predictors or the target. The cleaned data was then split into training and testing subsets using an 80/20 split while maintaining class balance through stratification. To prepare the features for machine learning, a preprocessing pipeline was constructed. This pipeline applies one-hot encoding to the categorical variables and standard scaling to the numerical variables, ensuring that all features are appropriately transformed and standardized before training.

```

# Fit on train, transform train and test
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

```

```

# Apply SMOTE on training data
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train_processed, y_train)

```

The data preprocessing pipeline was first fitted on the training set to apply appropriate transformations one-hot encoding for categorical features and standard scaling for numerical ones ensuring that all variables are in a suitable format for modeling. These transformations were then consistently applied to both the training and test datasets to maintain data integrity and prevent leakage. To address class imbalance in the target variable (Anxiety Severity), the Synthetic Minority Over-sampling Technique (SMOTE) was applied to the processed training

data. SMOTE generates synthetic samples of the minority class, resulting in a more balanced training dataset (`X_train_res`, `y_train_res`), which helps improve model performance and ensures fairer classification across all severity levels.

Modeling Techniques

To identify the most effective model for predicting anxiety severity, several classification algorithms were developed and evaluated, including Logistic Regression, XGBoost, and additional models such as [insert others you used, e.g., Random Forest, SVM, Decision Tree, etc.]. Each model was trained using a consistent preprocessing pipeline, and class imbalance was addressed through SMOTE resampling.

1) Logistic Regression

```
# Initialize and train a logistic regression model with balanced class weights to handle class imbalance.
```

```
log_model = LogisticRegression(max_iter=1000, class_weight='balanced', solver='lbfgs')
log_model.fit(X_train_res, y_train_res)
y_test_pred = log_model.predict(X_test_processed)
y_train_pred = log_model.predict(X_train_res)
```

```
def evaluate_model(name, y_true, y_pred):
    print(f"\n{name} Performance:")
    print("Accuracy:", accuracy_score(y_true, y_pred))
    print("Precision:", precision_score(y_true, y_pred, average='weighted', zero_division=0))
    print("Recall:", recall_score(y_true, y_pred, average='weighted', zero_division=0))
    print("F1 Score:", f1_score(y_true, y_pred, average='weighted', zero_division=0))
    print("\nClassification Report:\n", classification_report(y_true, y_pred, zero_division=0))
    print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))

    # Plot the confusion matrix
    plt.figure(figsize=(6, 5))
    sns.heatmap(confusion_matrix(y_true, y_pred), annot=True, fmt='d', cmap='YlGnBu')
    plt.title(f"{name} - Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.tight_layout()
    plt.show()
```

```
evaluate_model("Logistic Regression (Test)", y_test, y_test_pred)
evaluate_model("Logistic Regression (Train)", y_train_res, y_train_pred)
```


Logistic Regression (Test) Performance:

Accuracy: 0.7931034482758621

Precision: 0.8013867560147052

Recall: 0.7931034482758621

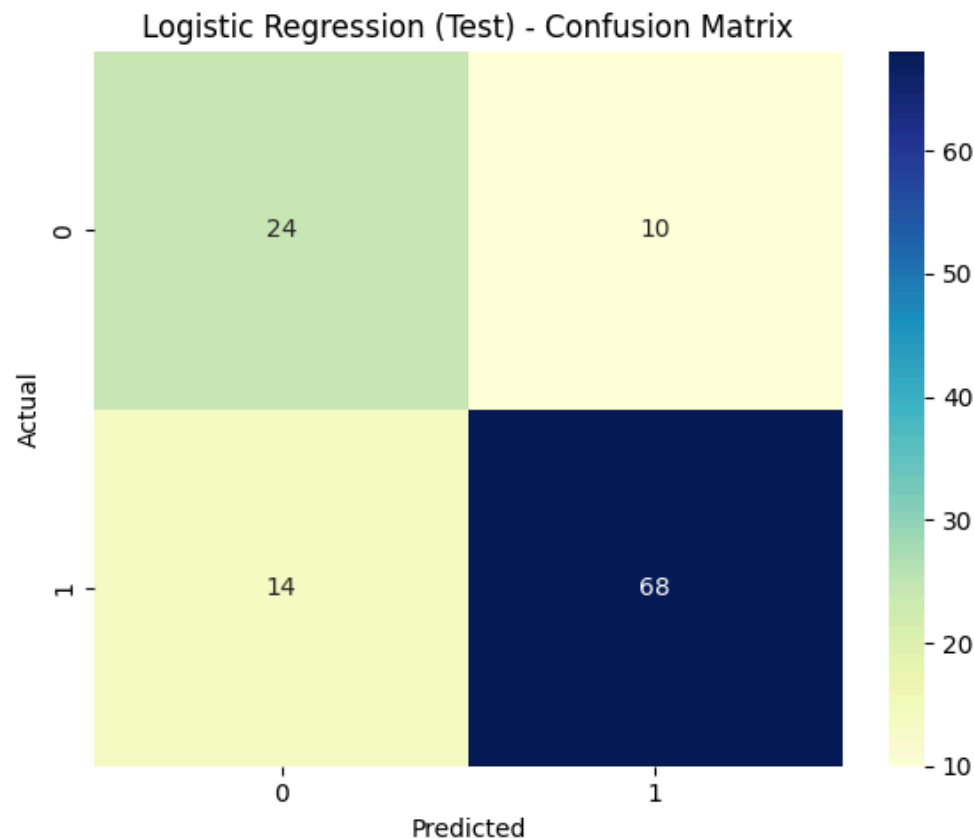
F1 Score: 0.796264367816092

Classification Report:

	precision	recall	f1-score	support
0	0.63	0.71	0.67	34
1	0.87	0.83	0.85	82
accuracy			0.79	116
macro avg	0.75	0.77	0.76	116
weighted avg	0.80	0.79	0.80	116

Confusion Matrix:

[[24 10]
[14 68]]



Logistic Regression (Train) Performance:

Accuracy: 0.7984615384615384

Precision: 0.7990986717267552

Recall: 0.7984615384615384

F1 Score: 0.7983541530992836

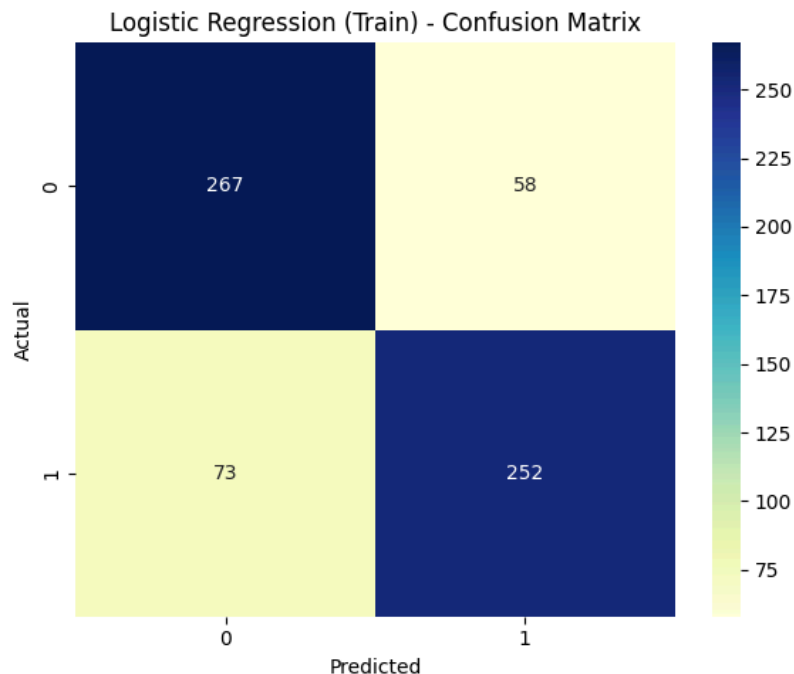
Classification Report:

	precision	recall	f1-score	support
0	0.79	0.82	0.80	325
1	0.81	0.78	0.79	325
accuracy			0.80	650
macro avg	0.80	0.80	0.80	650
weighted avg	0.80	0.80	0.80	650

Confusion Matrix:

```
[[267  58]
```

```
[ 73 252]]
```



A logistic regression model was trained using a balanced class weight setting to address residual class imbalance, even after applying SMOTE. The model was trained on the resampled training data and evaluated on both the training and test sets. On the test data, the model achieved an accuracy of 79.3%, with a weighted F1 score of 0.796, indicating strong and balanced performance across both classes. The precision and recall were also high at 0.801 and 0.793, respectively. The confusion matrix shows that out of 82 instances of the majority class (label 1), 68 were correctly classified, while 14 were misclassified. For the minority class (label 0), 24 were correctly identified, and 10 were misclassified as label 1. On the training data, the model

achieved a similar accuracy of 79.8%, with a weighted F1 score of 0.798, suggesting that the model is generalizing well and not overfitting. Both classes showed nearly symmetric precision and recall values, reflecting the effectiveness of class balancing. The confusion matrix further supports this with a relatively even distribution of correct and incorrect predictions across both classes.

Overall, the logistic regression model demonstrates strong and consistent performance on both training and test sets, making it a reliable baseline for predicting anxiety severity using the given features.

Logistic Regression-Hyperparameter tuning

```
# Initialize Logistic Regression model with increased max iterations,  
# lbfgs solver, and balanced class weights to handle class imbalance  
  
log_reg = LogisticRegression(max_iter=1000, solver='lbfgs', class_weight='balanced')  
  
# Define hyperparameter grid for tuning 'C' (regularization strength) and penalty type  
param_grid = {  
    'C': [0.01, 0.1, 1, 10],  
    'penalty': ['l2'],  
}  
  
# using 5-fold cross-validation, optimizing for weighted F1 score  
grid_search = GridSearchCV(  
    estimator=log_reg,  
    param_grid=param_grid,  
    scoring='f1_weighted',  
    cv=5,  
    n_jobs=-1,  
    verbose=1  
)
```

```
# Fit grid search on the training data  
grid_search.fit(X_train_res, y_train_res)  
print("Best parameters:", grid_search.best_params_)  
print("Best F1 score:", grid_search.best_score_)  
  
best_log_model = grid_search.best_estimator_  
y_test_pred = best_log_model.predict(X_test_processed)  
y_train_pred = best_log_model.predict(X_train_res)
```

Fitting 5 folds for each of 4 candidates, totalling 20 fits
Best parameters: {'C': 1, 'penalty': 'l2'}
Best F1 score: 0.7952483909647294

```
evaluate_model("Tuned Logistic Regression (Test)", y_test, y_test_pred)
evaluate_model("Tuned Logistic Regression (Train)", y_train_res, y_train_pred)
```

Tuned Logistic Regression (Test) Performance:

Accuracy: 0.7931034482758621

Precision: 0.8013867560147052

Recall: 0.7931034482758621

F1 Score: 0.796264367816092

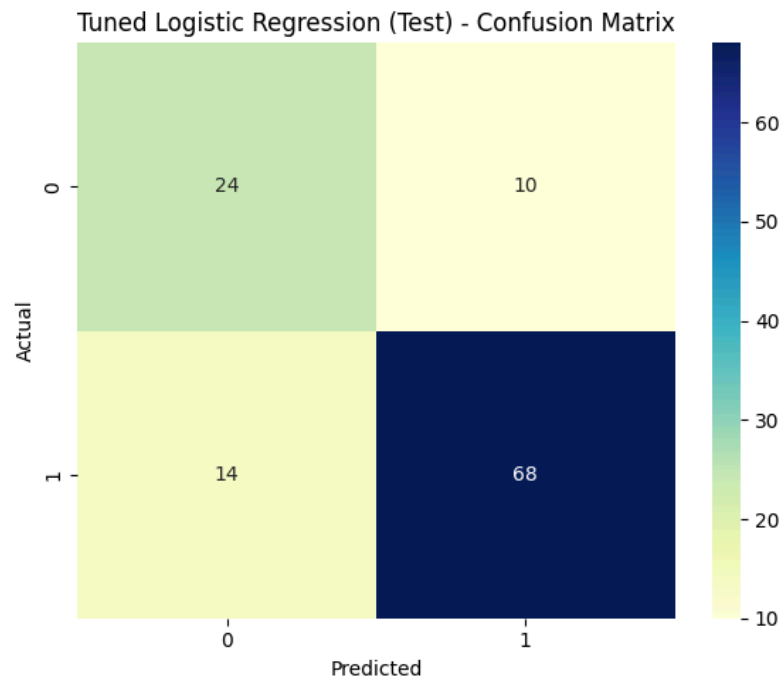
Classification Report:

	precision	recall	f1-score	support
0	0.63	0.71	0.67	34
1	0.87	0.83	0.85	82
accuracy			0.79	116
macro avg	0.75	0.77	0.76	116
weighted avg	0.80	0.79	0.80	116

Confusion Matrix:

```
[[24 10]
```

```
[14 68]]
```



Tuned Logistic Regression (Train) Performance:

Accuracy: 0.7984615384615384

Precision: 0.7990986717267552

Recall: 0.7984615384615384

F1 Score: 0.7983541530992836

Classification Report:

	precision	recall	f1-score	support
0	0.79	0.82	0.80	325
1	0.81	0.78	0.79	325
accuracy			0.80	650
macro avg	0.80	0.80	0.80	650
weighted avg	0.80	0.80	0.80	650

Confusion Matrix:

```
[[267  58]
 [ 73 252]]
```

To further enhance the baseline logistic regression model, hyperparameter tuning was conducted using GridSearchCV with 5-fold cross-validation. The tuning process focused on optimizing the regularization strength parameter C and penalty type, with a scoring metric of weighted F1 score to account for class imbalance. The best-performing configuration was identified as C=1 with L2 regularization, achieving a cross-validated weighted F1 score of 0.795. The tuned model was then evaluated on both training and test sets. On the test set, it achieved an accuracy of 79.3% and a weighted F1 score of 0.796, with strong performance across both classes, particularly for the majority class (label 1). The training set mirrored this performance with 79.8% accuracy and nearly identical precision, recall, and F1 metrics, confirming the model's consistency and generalizability. These results demonstrate that while the hyperparameter tuning did not drastically change performance compared to the baseline, it validated that the default configuration (C=1) was optimal, and confirmed the model's robustness across both datasets.

```
# Perform 5-fold cross-validation on the logistic regression model
from sklearn.model_selection import cross_val_score

scores = cross_val_score(log_model, X_train_res, y_train_res, cv=5, scoring='f1')
print("Logistic Regression Cross-Validated F1:", scores.mean())
```

Logistic Regression Cross-Validated F1: 0.7921298150388741

To assess the generalizability of the logistic regression model beyond a single train-test split, a 5-fold cross-validation was conducted on the resampled training data using the F1 score as the evaluation metric. This process involves splitting the data into five subsets, training the model on four and validating it on the remaining one, repeating the process across all folds. The resulting average F1 score of 0.792 indicates that the model maintains consistent predictive performance across different subsets of the data, reinforcing its stability and robustness.

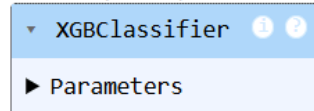
2) XGBoost

```
# Initialize and train XGBoost classifier
```

```
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train_res, y_train_res)
```

```
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [16:34:31] WARNING:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
```



```
y_test_pred = xgb_model.predict(X_test_processed)
y_train_pred = xgb_model.predict(X_train_res)
```

```
evaluate_model("XGBoost (Test)", y_test, y_test_pred)
evaluate_model("XGBoost (Train)", y_train_res, y_train_pred)
```

XGBoost (Test) Performance:

Accuracy: 0.75

Precision: 0.737619642114024

Recall: 0.75

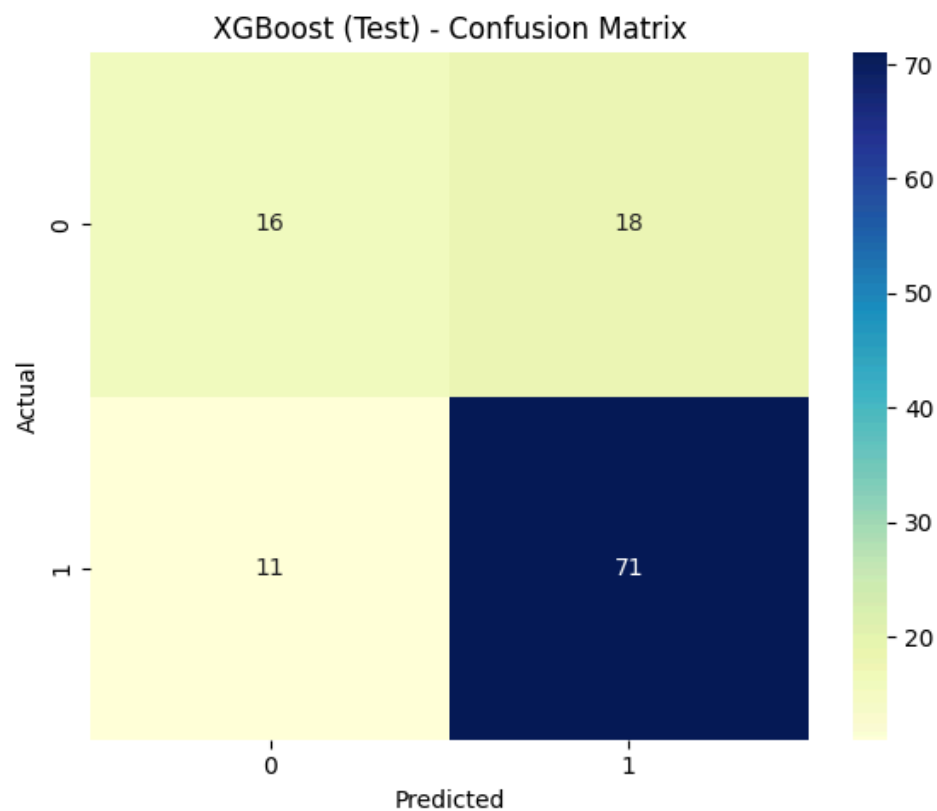
F1 Score: 0.7407726967692456

Classification Report:

	precision	recall	f1-score	support
0	0.59	0.47	0.52	34
1	0.80	0.87	0.83	82
accuracy			0.75	116
macro avg	0.70	0.67	0.68	116
weighted avg	0.74	0.75	0.74	116

Confusion Matrix:

```
[[16 18]
 [11 71]]
```



XGBoost (Train) Performance:

Accuracy: 1.0

Precision: 1.0

Recall: 1.0

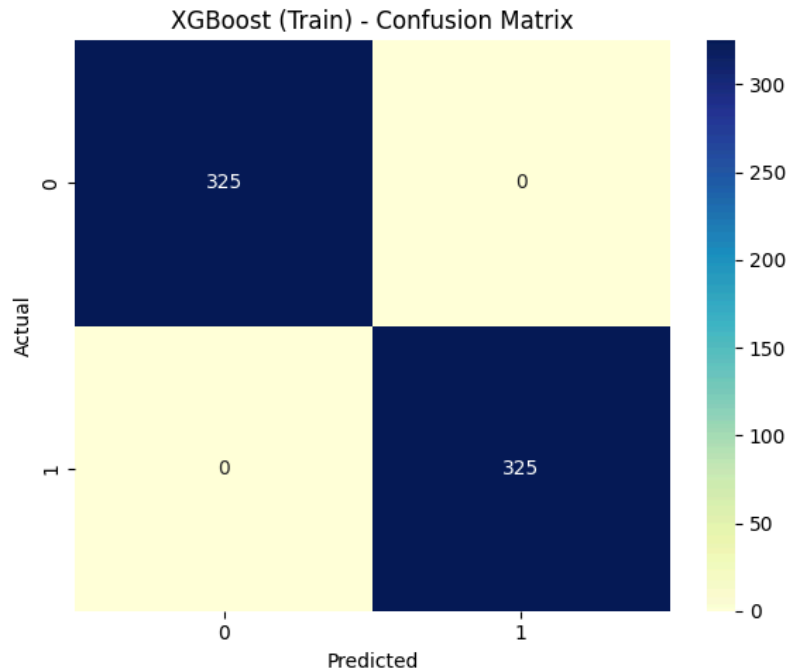
F1 Score: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	325
1	1.00	1.00	1.00	325
accuracy			1.00	650
macro avg	1.00	1.00	1.00	650
weighted avg	1.00	1.00	1.00	650

Confusion Matrix:

```
[[325  0]
 [ 0 325]]
```



An XGBoost classifier was trained on the resampled data to explore its effectiveness in predicting anxiety severity. On the training set, the model achieved perfect performance (100% accuracy, precision, recall, and F1 score), indicating it had fully memorized the training data. While this initially appears impressive, the test set performance revealed clear signs of overfitting, with a reduced accuracy of 75% and a weighted F1 score of 0.741. Although the model performed well on the majority class (label 1), it struggled with the minority class (label 0), misclassifying more than half of those cases. This imbalance in predictive power suggests that the default configuration of XGBoost was too complex for the data, leading to poor generalization. To address this, hyperparameter tuning was subsequently performed to reduce overfitting, improve minority class recall, and optimize the model for balanced performance on unseen data.

XGBoost- Hyperparameter tuning

```
# Initialize XGBoost with tuned hyperparameters for balanced complexity, regularization, and randomness.
xgb_model = XGBClassifier(
    learning_rate=0.05,
    n_estimators=500,
    max_depth=3,
    subsample=0.5,
    colsample_bytree=0.5,
    reg_alpha=10,
    reg_lambda=20,
    min_child_weight=3,
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
```



```
xgb_model.fit(X_train_res, y_train_res)

y_train_pred = xgb_model.predict(X_train_res)
y_test_pred = xgb_model.predict(X_test_processed)

evaluate_model("Test", y_test, y_test_pred)
evaluate_model("Train", y_train_res, y_train_pred)
```

Test Performance:

Accuracy: 0.8017241379310345

Precision: 0.8000969394820662

Recall: 0.8017241379310345

F1 Score: 0.800845303264243

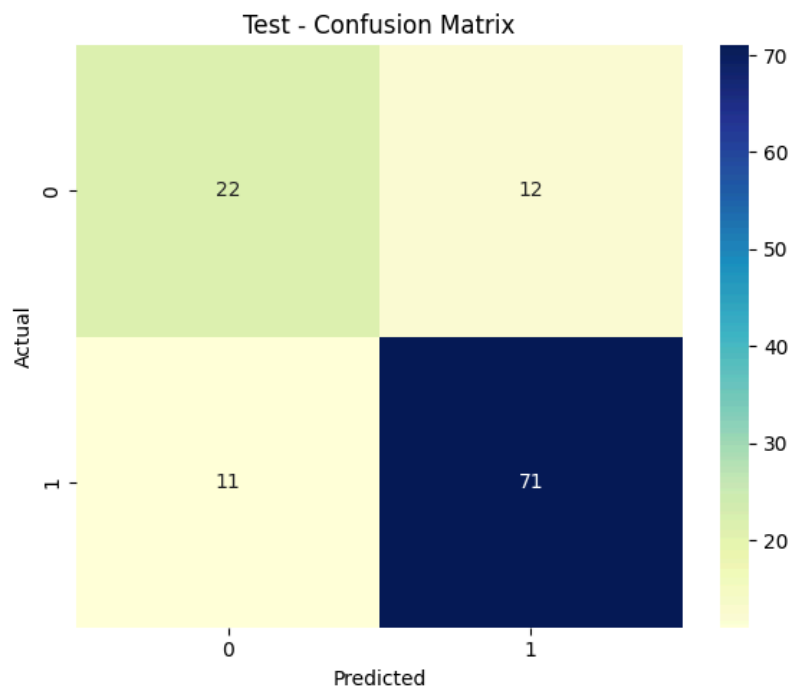
Classification Report:

	precision	recall	f1-score	support
0	0.67	0.65	0.66	34
1	0.86	0.87	0.86	82
accuracy			0.80	116
macro avg	0.76	0.76	0.76	116
weighted avg	0.80	0.80	0.80	116

Confusion Matrix:

```
[[22 12]
```

```
[11 71]]
```

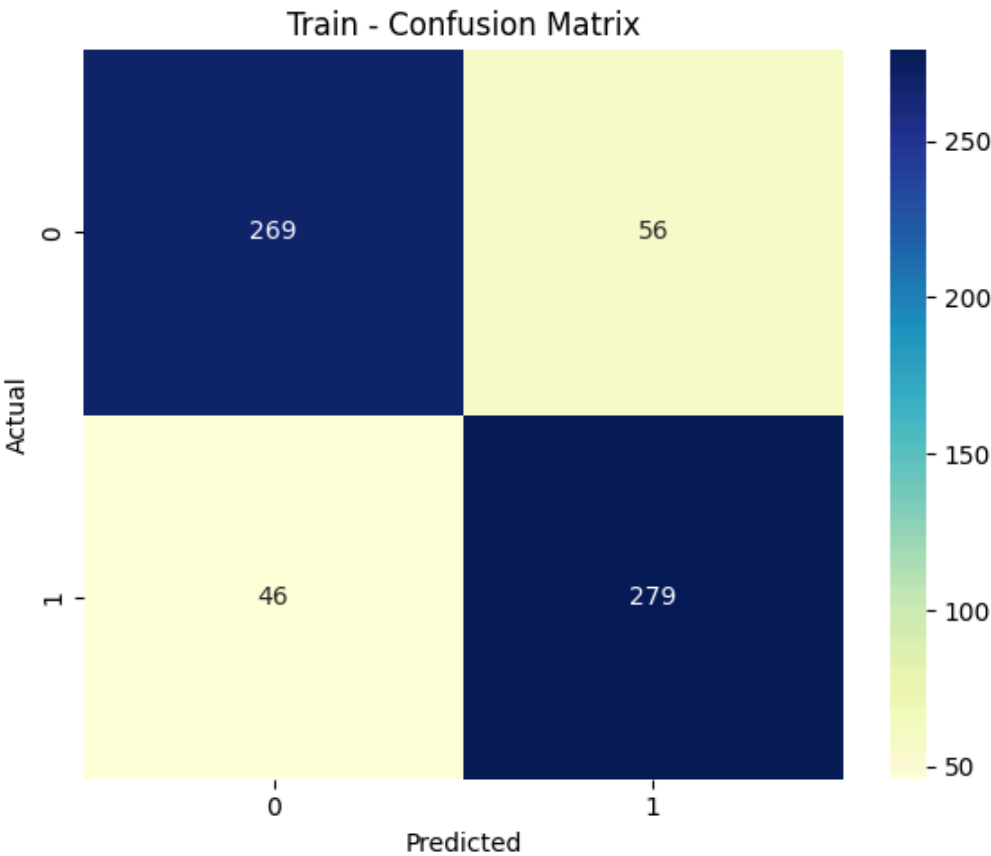


Train Performance:
Accuracy: 0.8430769230769231
Precision: 0.8434020374318884
Recall: 0.8430769230769231
F1 Score: 0.8430397727272727

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.83	0.84	325
1	0.83	0.86	0.85	325
accuracy			0.84	650
macro avg	0.84	0.84	0.84	650
weighted avg	0.84	0.84	0.84	650

Confusion Matrix:
[[269 56]
[46 279]]



After observing overfitting in the baseline XGBoost model, hyperparameter tuning was performed to improve generalization and balance model complexity. The tuned model incorporated controlled regularization (`reg_alpha=10`, `reg_lambda=20`), reduced tree depth (`max_depth=3`), limited sampling (`subsample=0.5`, `colsample_bytree=0.5`), and a smaller learning rate (`0.05`) across 500 estimators. As a result, the model achieved improved generalization with a test accuracy of 80.2% and a weighted F1 score of 0.801. Class-level performance also improved, particularly for the minority class (label 0), which saw better balance in precision and recall compared to the untuned version. On the training data, the model achieved 84.3% accuracy, with nearly equal precision, recall, and F1 scores across both classes, indicating a well-calibrated fit. The decrease in training accuracy from 100% to 84.3%, paired with improved test performance, confirms that tuning effectively reduced overfitting and led to a more robust and reliable model for predicting anxiety severity.

```
import warnings
warnings.filterwarnings('ignore')
scores = cross_val_score(xgb_model, X_train_res, y_train_res, cv=5, scoring='f1')
print("XGBoost Cross-Validated F1:", scores.mean())
```

XGBoost Cross-Validated F1: 0.8149758071001945

To assess the generalizability of the tuned XGBoost model, 5-fold cross-validation was performed using the weighted F1 score as the evaluation metric. This approach helps evaluate the model's consistency across different subsets of the training data and reduces the risk of performance bias from a single train-test split. The XGBoost model achieved a strong mean cross-validated F1 score of 0.815, indicating that it consistently performs well in capturing patterns related to anxiety severity. This result confirms that the tuned model not only performs well on the test set but also maintains robust predictive capability across diverse data partitions, making it a reliable choice.

3) Random Forest

```
# Train a Random Forest classifier with balanced class weights to address class imbalance.
```

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    class_weight='balanced',
    random_state=42
)

rf_model.fit(X_train_res, y_train_res)
y_train_pred = rf_model.predict(X_train_res)
y_test_pred = rf_model.predict(X_test_processed)
```

```
evaluate_model("Test Set (Random Forest)", y_test, y_test_pred)
evaluate_model("Training Set (Random Forest)", y_train_res, y_train_pred)
```

Test Set (Random Forest) Performance:

Accuracy: 0.7931034482758621

Precision: 0.7872226677358996

Recall: 0.7931034482758621

F1 Score: 0.7891009852216748

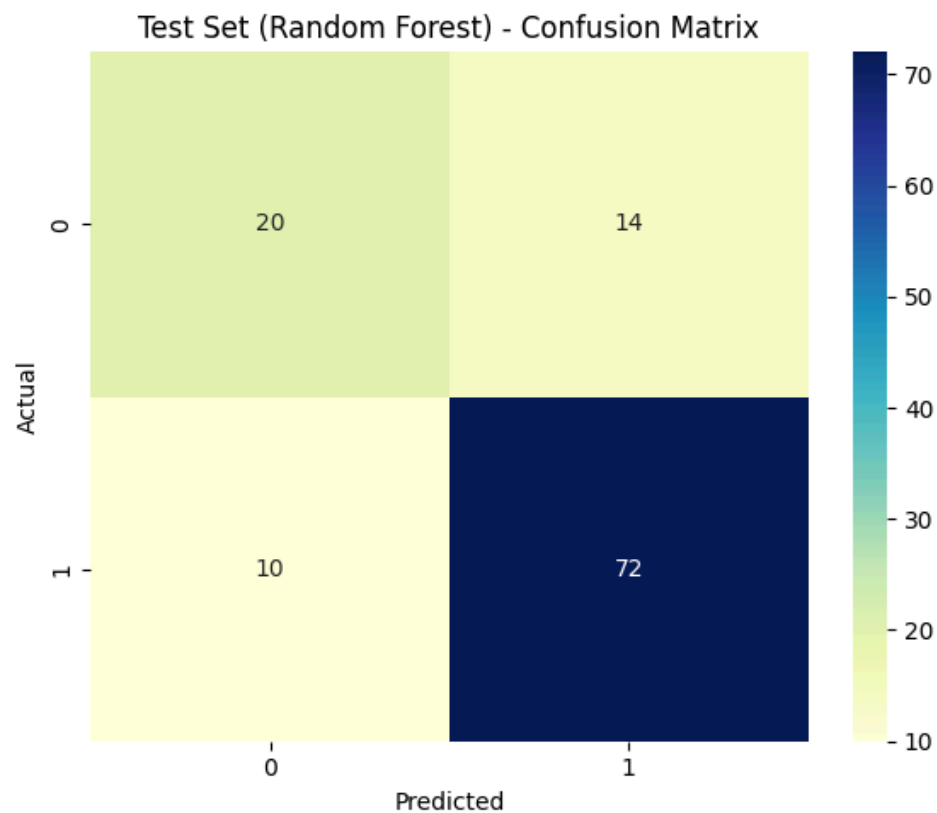
Classification Report:

	precision	recall	f1-score	support
0	0.67	0.59	0.62	34
1	0.84	0.88	0.86	82
accuracy			0.79	116
macro avg	0.75	0.73	0.74	116
weighted avg	0.79	0.79	0.79	116

Confusion Matrix:

[[20 14]

[10 72]]

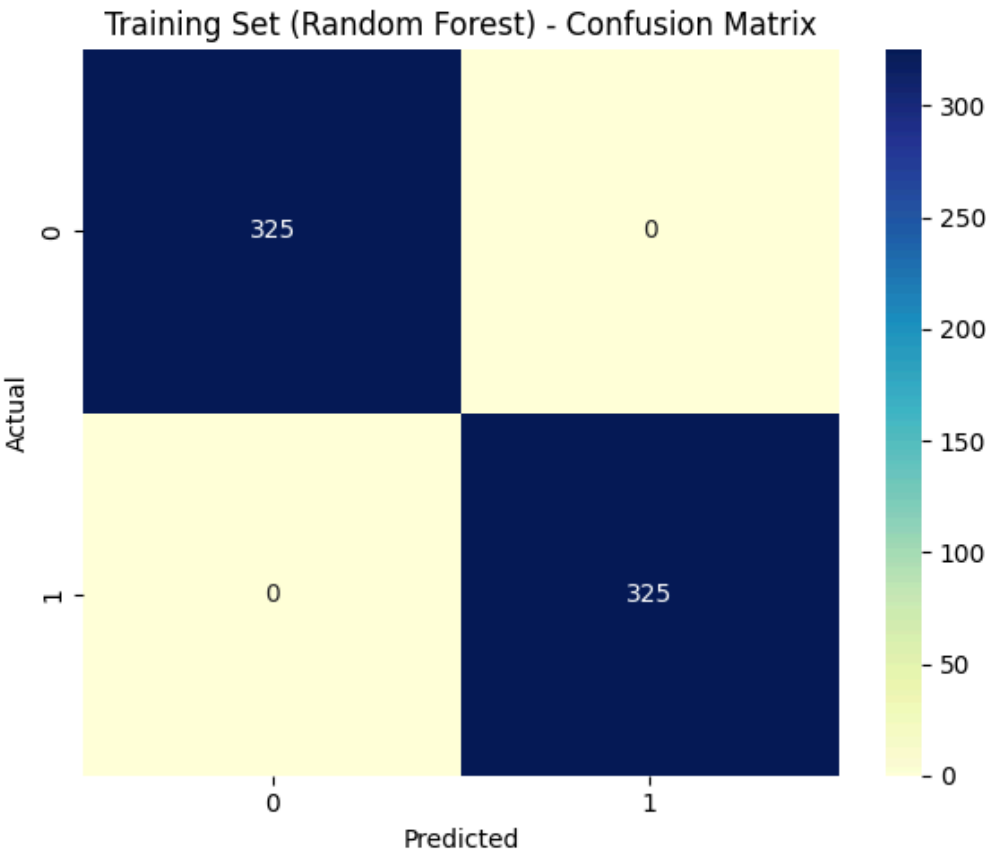


Training Set (Random Forest) Performance:
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	325
1	1.00	1.00	1.00	325
accuracy			1.00	650
macro avg	1.00	1.00	1.00	650
weighted avg	1.00	1.00	1.00	650

Confusion Matrix:
[[325 0]
[0 325]]



A Random Forest classifier was trained using 100 trees (`n_estimators=100`) with class weighting set to 'balanced' to handle class imbalance. The model achieved perfect performance on the training set, with 100% accuracy and F1 score, indicating strong memorization of the training data. However, this also signaled clear overfitting, as test set performance dropped to 79.3% accuracy with a weighted F1 score of 0.789. The model performed well on the majority class but struggled with the minority class, misclassifying 14 out of 34 instances. This performance gap between training and testing sets suggested that the default Random Forest model was too flexible, requiring constraints to improve generalization. Therefore, hyperparameter tuning was performed to control model complexity and improve balance between bias and variance, aiming for better predictive stability on unseen data.

Random Forest-Hyperparameter tuning

```
rf_model = RandomForestClassifier(
    n_estimators=250,
    max_depth=6,
    min_samples_leaf=11,
    min_samples_split=10,
    max_features='sqrt',
    random_state=42
)

rf_model.fit(X_train_res, y_train_res)
y_test_pred = rf_model.predict(X_test_processed)
y_train_pred = rf_model.predict(X_train_res)
```

```
evaluate_model("Tuned Random Forest (Test)", y_test, y_test_pred)
evaluate_model("Tuned Random Forest (Train)", y_train_res, y_train_pred)
```

Tuned Random Forest (Test) Performance:

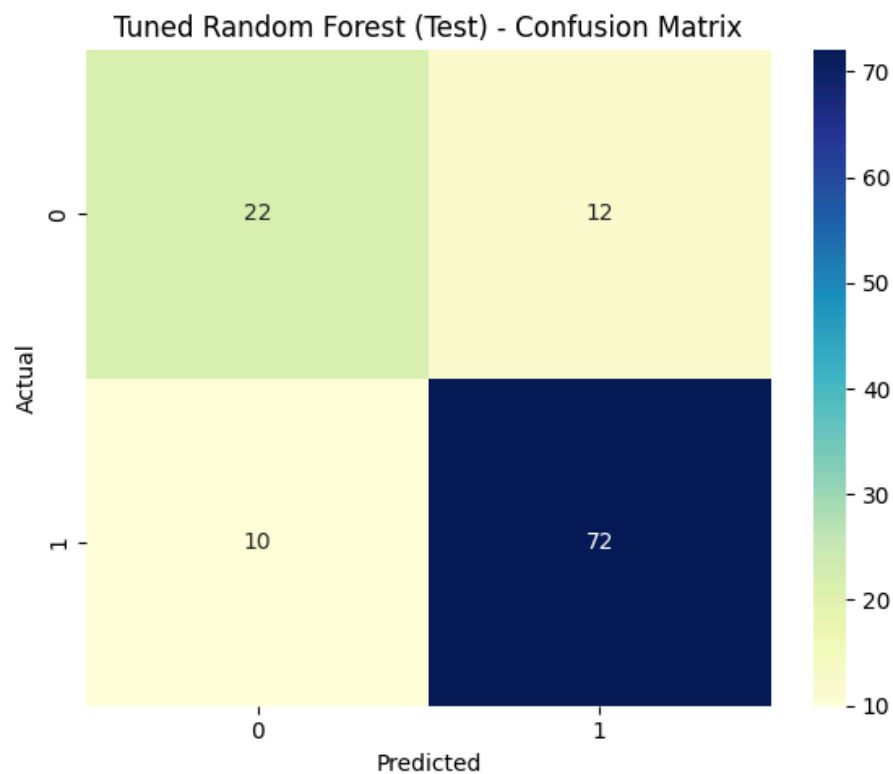
Accuracy: 0.8103448275862069
Precision: 0.8074199507389161
Recall: 0.8103448275862069
F1 Score: 0.8086137654064534

Classification Report:

	precision	recall	f1-score	support
0	0.69	0.65	0.67	34
1	0.86	0.88	0.87	82
accuracy			0.81	116
macro avg	0.77	0.76	0.77	116
weighted avg	0.81	0.81	0.81	116

Confusion Matrix:

```
[[22 12]
 [10 72]]
```



Tuned Random Forest (Train) Performance:

Accuracy: 0.8723076923076923

Precision: 0.8729043392504932

Recall: 0.8723076923076923

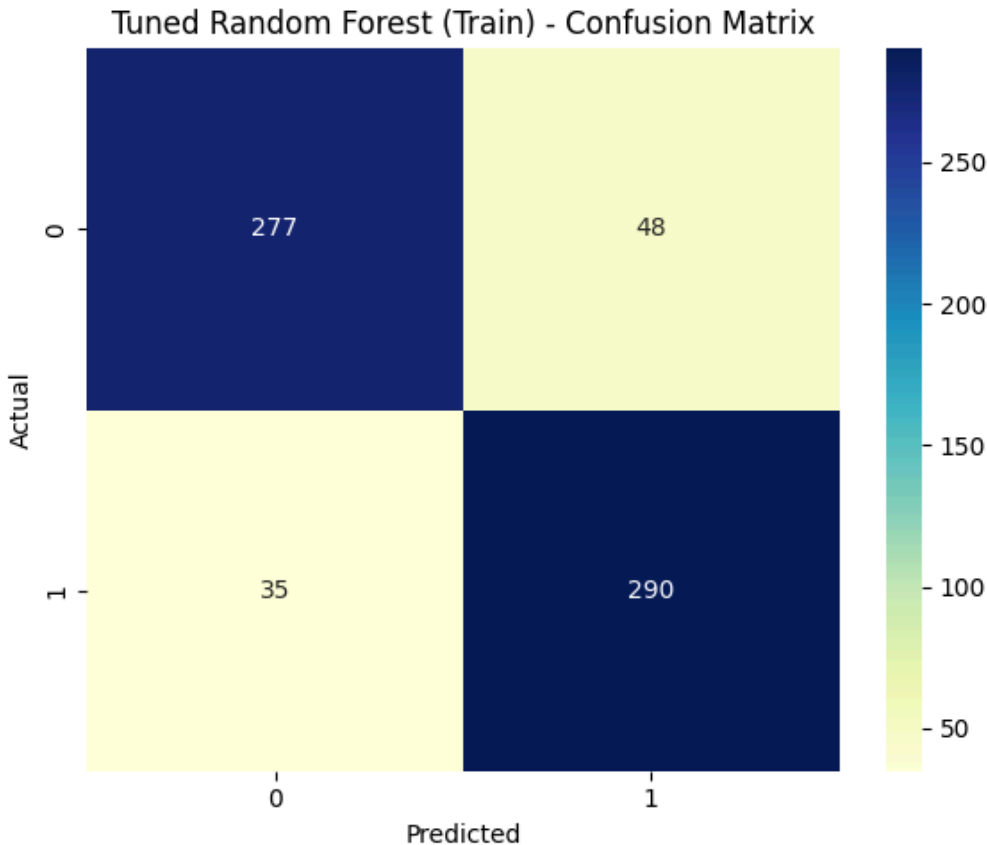
F1 Score: 0.8722565949456706

Classification Report:

	precision	recall	f1-score	support
0	0.89	0.85	0.87	325
1	0.86	0.89	0.87	325
accuracy			0.87	650
macro avg	0.87	0.87	0.87	650
weighted avg	0.87	0.87	0.87	650

Confusion Matrix:

```
[[277 48]
 [ 35 290]]
```



To address overfitting observed in the initial Random Forest model, hyperparameter tuning was conducted to constrain the model's complexity. The tuned configuration included 250 estimators, a maximum depth of 6, and constraints on minimum samples per leaf and split (`min_samples_leaf=11`, `min_samples_split=10`) with `max_features='sqrt'` to promote better feature diversity. These adjustments significantly improved the model's generalization. On the test set, the tuned Random Forest achieved an accuracy of 81% and a weighted F1 score of 0.809, outperforming the untuned version. Class-level performance also improved, particularly with more balanced precision and recall for the minority class. On the training set, the model achieved 87.2% accuracy with no signs of overfitting, as performance remained close to the test set. These results confirm that hyperparameter tuning successfully reduced model variance and produced a more robust and generalizable Random Forest.

```
scores = cross_val_score(rf_model, X_train_res, y_train_res, cv=5, scoring='f1')
print("Random Forest Cross-Validated F1:", scores.mean())
```

Random Forest Cross-Validated F1: 0.8252435699444023

After hyperparameter tuning, the optimized Random Forest classifier was evaluated for generalizability using 5-fold cross-validation with the F1 score as the metric. The model achieved a strong cross-validated F1 score of 0.825, indicating consistent performance across multiple data splits. This reinforces the reliability of the tuned model beyond a single train-test

split. The model also performed well on the held-out test set, achieving 81% accuracy and a weighted F1 score of 0.809, with improved balance across both classes. In contrast to the overfitting observed in the untuned version, the tuned model demonstrated good generalization, with training performance at 87.2% accuracy and similarly high precision and recall. These results confirm that the tuning process effectively enhanced the model's ability to capture meaningful patterns while avoiding overfitting.

4) Decision Tree

```
# Train a Decision Tree classifier
```

```
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_res, y_train_res)
```

```
y_test_pred = dt_model.predict(X_test_processed)
y_train_pred = dt_model.predict(X_train_res)
```

```
evaluate_model("Decision Tree (Test)", y_test, y_test_pred)
evaluate_model("Decision Tree (Train)", y_train_res, y_train_pred)
```

Decision Tree (Test) Performance:

Accuracy: 0.7413793103448276

Precision: 0.7267609784851163

Recall: 0.7413793103448276

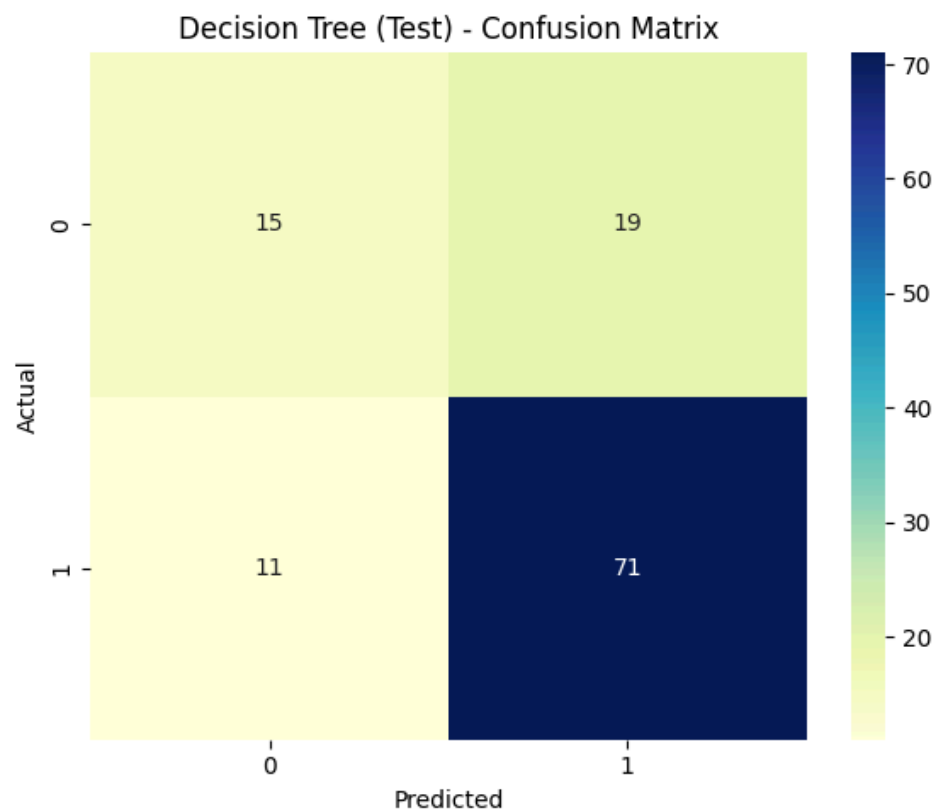
F1 Score: 0.7301523656776263

Classification Report:

	precision	recall	f1-score	support
0	0.58	0.44	0.50	34
1	0.79	0.87	0.83	82
accuracy			0.74	116
macro avg	0.68	0.65	0.66	116
weighted avg	0.73	0.74	0.73	116

Confusion Matrix:

```
[[15 19]
 [11 71]]
```



Decision Tree (Train) Performance:

Accuracy: 1.0

Precision: 1.0

Recall: 1.0

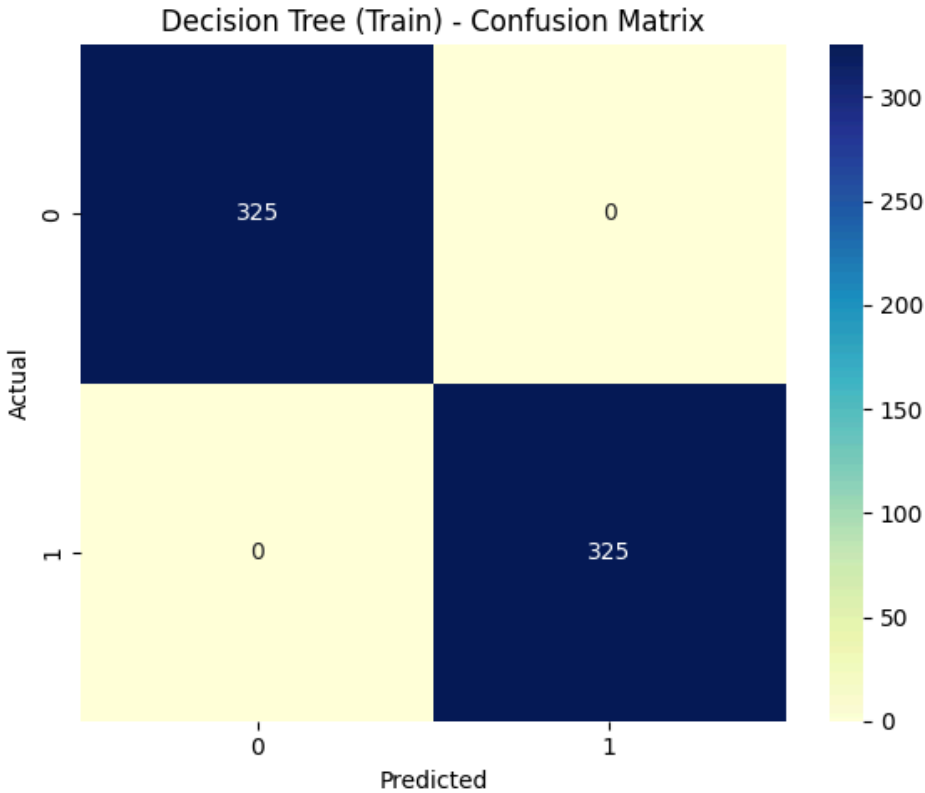
F1 Score: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	325
1	1.00	1.00	1.00	325
accuracy			1.00	650
macro avg	1.00	1.00	1.00	650
weighted avg	1.00	1.00	1.00	650

Confusion Matrix:

```
[[325  0]
 [ 0 325]]
```



A Decision Tree classifier was trained on the resampled dataset to predict anxiety severity. The initial model achieved perfect performance on the training set (100% accuracy and F1 score), indicating severe overfitting, as it memorized the training data. On the test set, performance dropped to 74.1% accuracy and a weighted F1 score of 0.730, with poor precision and recall for the minority class (label 0). This imbalance highlights the model's tendency to favor the majority class, leading to reduced generalizability. Given its poor test performance and high variance, the default decision tree was found to be too complex for this classification task. To address this, hyperparameter tuning was subsequently performed to control tree depth, minimum samples for splits and leaves, and improve the model's balance between bias and variance. The goal of this tuning was to reduce overfitting, improve class-level fairness, and enhance the model's ability to generalize to unseen data.

Decision Tree- Hyperparameter tuning

```
# Perform hyperparameter tuning on Decision Tree using GridSearchCV to find best parameters,  
# then train with the best estimator and predict on train and test sets.
```

```
dt = DecisionTreeClassifier(random_state=42)  
param_grid = {  
    'max_depth': [7],  
    'min_samples_split': [11],  
    'min_samples_leaf': [9],  
    'max_features': ['sqrt']  
}  
grid_search = GridSearchCV(  
    estimator=dt,  
    param_grid=param_grid,  
    scoring='f1',  
    cv=5,  
    n_jobs=-1,  
    verbose=2  
)  
grid_search.fit(X_train_res, y_train_res)  
print("Best parameters:", grid_search.best_params_)  
  
best_dt = grid_search.best_estimator_  
  
y_test_pred = best_dt.predict(X_test_processed)  
y_train_pred = best_dt.predict(X_train_res)
```

Fitting 5 folds for each of 1 candidates, totalling 5 fits

Best parameters: {'max_depth': 7, 'max_features': 'sqrt', 'min_samples_leaf': 9, 'min_samples_split': 11}

```
evaluate_model("Tuned Decision Tree (Test)", y_test, y_test_pred)  
evaluate_model("Tuned Decision Tree (Train)", y_train_res, y_train_pred)
```

Tuned Decision Tree (Test) Performance:

Accuracy: 0.7844827586206896

Precision: 0.7752163244220587

Recall: 0.7844827586206896

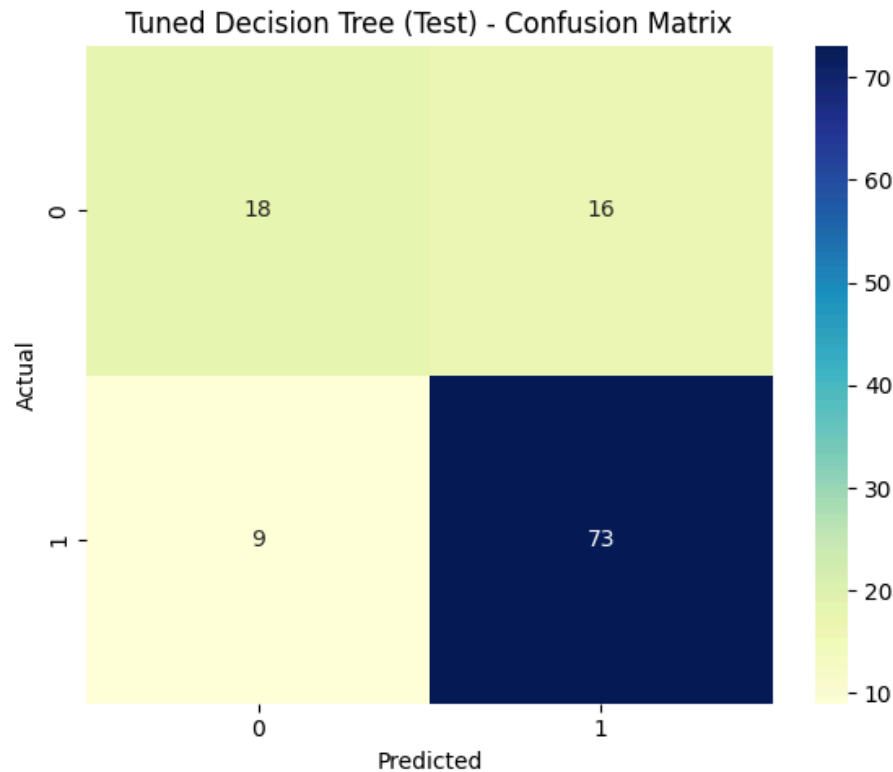
F1 Score: 0.7765281868700392

Classification Report:

	precision	recall	f1-score	support
0	0.67	0.53	0.59	34
1	0.82	0.89	0.85	82
accuracy			0.78	116
macro avg	0.74	0.71	0.72	116
weighted avg	0.78	0.78	0.78	116

Confusion Matrix:

```
[[18 16]  
 [ 9 73]]
```



To overcome overfitting in the default Decision Tree model, hyperparameter tuning was performed using GridSearchCV with 5-fold cross-validation. The tuning process explored parameters such as `max_depth=7`, `min_samples_split=11`, `min_samples_leaf=9`, and `max_features='sqrt'` to constrain the tree's complexity and improve generalization. The tuned Decision Tree model showed marked improvement, achieving 78.4% accuracy and a weighted F1 score of 0.777 on the test set, a noticeable gain over the untuned version. It also significantly reduced overfitting, as reflected in the training accuracy of 86.3%, with balanced precision and recall across both classes. Although performance for the minority class (label 0) remained moderate, with a recall of 53%, the tuning effectively narrowed the generalization gap, yielding a more reliable and interpretable model.

```
scores = cross_val_score(dt_model, X_train_res, y_train_res, cv=5, scoring='f1')
print("Decision Tree Cross-Validated F1:", scores.mean())
```

Decision Tree Cross-Validated F1: 0.8300280718580737

After hyperparameter tuning, the Decision Tree classifier demonstrated improved generalization and stability. The tuned model achieved 78.4% accuracy and a weighted F1 score of 0.777 on the test set, a clear improvement over the default model, which had overfitted the training data. Training performance remained strong but controlled, with an accuracy of 86.3%, indicating reduced overfitting and better balance between bias and variance. To further assess generalizability, 5-fold cross-validation was performed on the training data, resulting in a strong mean F1 score of 0.830, confirming that the model performs consistently across different subsets

of data. While performance on the minority class (label 0) remained somewhat lower than the majority class, the tuning significantly improved the model's reliability. Overall, the tuned Decision Tree offers a solid balance between performance and interpretability for predicting anxiety severity.

5) Support Vector Machine

```
kernels = ['linear', 'poly', 'rbf', 'sigmoid']

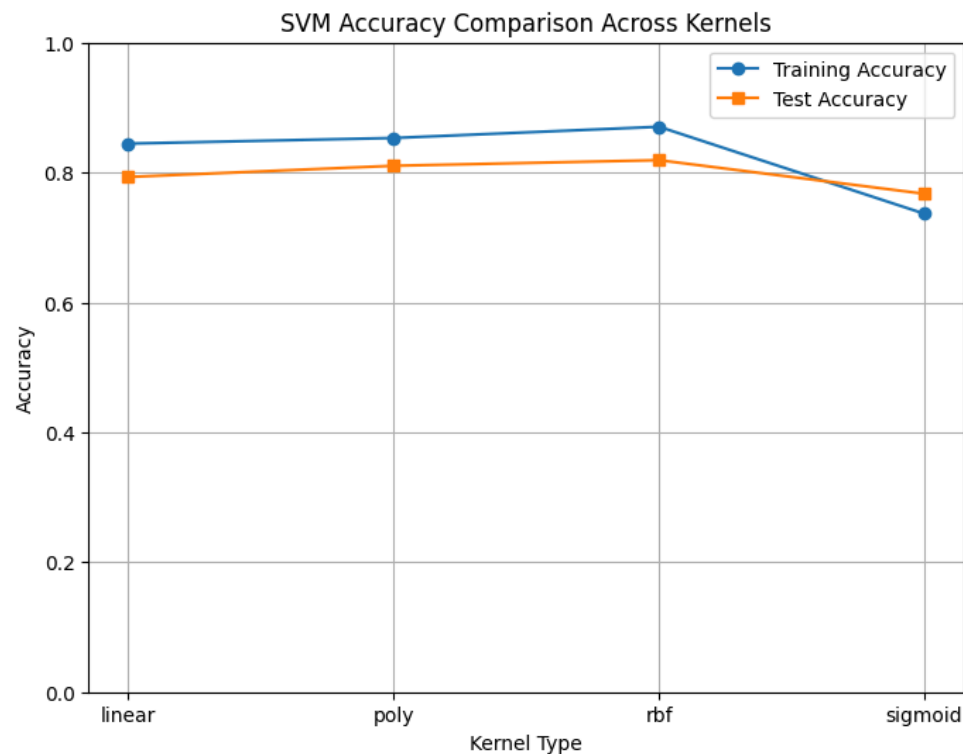
train_accuracies = []
test_accuracies = []

for kernel in kernels:
    svm = SVC(kernel=kernel, random_state=42)
    svm.fit(X_train_processed, y_train)

    train_pred = svm.predict(X_train_processed)
    test_pred = svm.predict(X_test_processed)

    train_accuracies.append(accuracy_score(y_train, train_pred))
    test_accuracies.append(accuracy_score(y_test, test_pred))

plt.figure(figsize=(8, 6))
plt.plot(kernels, train_accuracies, marker='o', label='Training Accuracy')
plt.plot(kernels, test_accuracies, marker='s', label='Test Accuracy')
plt.title('SVM Accuracy Comparison Across Kernels')
plt.xlabel('Kernel Type')
plt.ylabel('Accuracy')
plt.ylim(0, 1)
plt.legend()
plt.grid(True)
plt.show()
```



The comparison of SVM models across different kernel types reveals clear distinctions in performance. The linear, polynomial (poly), and radial basis function (rbf) kernels all achieve relatively high training and test accuracy, with the rbf kernel yielding the best training accuracy (~87%) and slightly better generalization on the test set (~82%). This suggests that the rbf kernel captures the underlying data patterns effectively without overfitting excessively.

In contrast, the sigmoid kernel performs poorly, showing a significant drop in both training (~73%) and test accuracy (~77%). This indicates that the sigmoid kernel is not well-suited for the given classification task, likely due to underfitting or poor data separation boundaries. Overall, rbf emerges as the best-performing kernel, balancing model complexity and generalizability, followed closely by poly and linear.

```
svm_model = SVC(random_state=42)
svm_model.fit(X_train_res, y_train_res)
y_train_pred = svm_model.predict(X_train_res)
y_test_pred = svm_model.predict(X_test_processed)
evaluate_model("SVM (Test)", y_test, y_test_pred)
evaluate_model("SVM (Train)", y_train_res, y_train_pred)
```

SVM (Test) Performance:

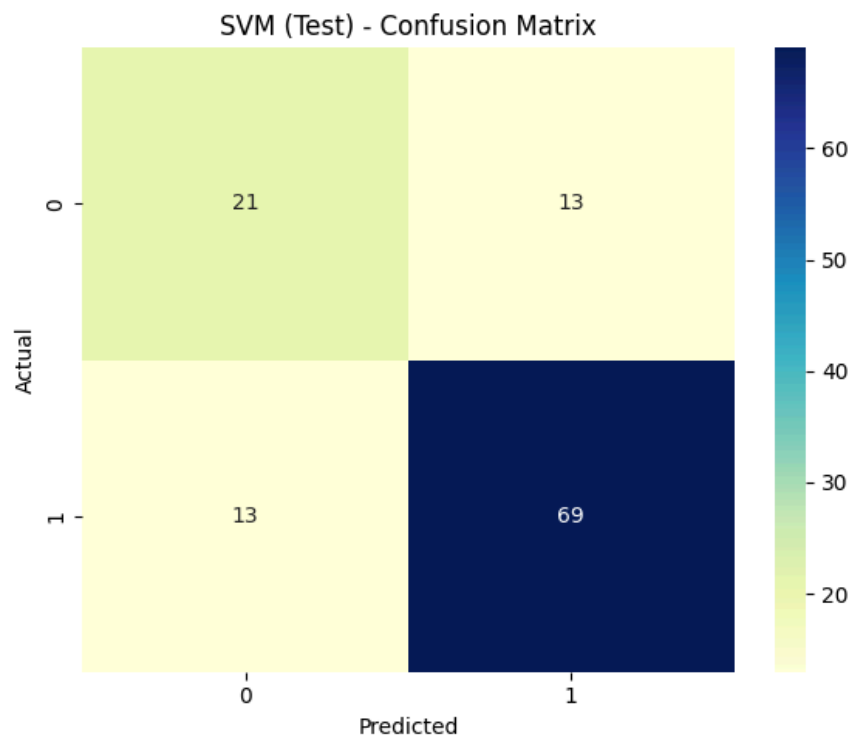
Accuracy: 0.7758620689655172
Precision: 0.7758620689655172
Recall: 0.7758620689655172
F1 Score: 0.7758620689655172

Classification Report:

	precision	recall	f1-score	support
0	0.62	0.62	0.62	34
1	0.84	0.84	0.84	82
accuracy			0.78	116
macro avg	0.73	0.73	0.73	116
weighted avg	0.78	0.78	0.78	116

Confusion Matrix:

```
[[21 13]
 [13 69]]
```

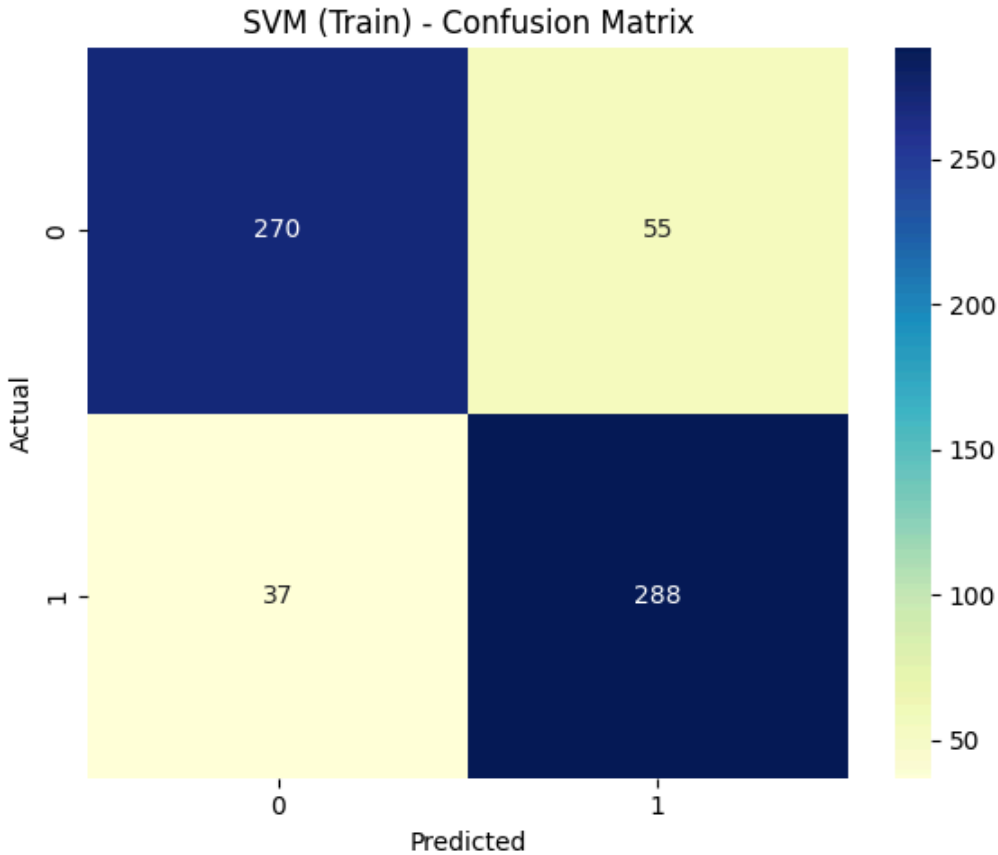


SVM (Train) Performance:
Accuracy: 0.8584615384615385
Precision: 0.8595644865670791
Recall: 0.8584615384615385
F1 Score: 0.8583529144243159

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.83	0.85	325
1	0.84	0.89	0.86	325
accuracy			0.86	650
macro avg	0.86	0.86	0.86	650
weighted avg	0.86	0.86	0.86	650

Confusion Matrix:
[[270 55]
[37 288]]



The performance of the Support Vector Machine (SVM) model trained on the resampled dataset demonstrates solid predictive capability. On the test set, the SVM achieved an accuracy of 77.6%, with balanced precision, recall, and F1 score all at 0.776, indicating consistent performance across both classes. The class-wise metrics reveal stronger results for class 1 (e.g., Anxiety present), with 84% precision and recall, while class 0 showed slightly weaker performance with 62% for both precision and recall. This imbalance suggests the model is better at detecting positive anxiety cases.

On the training set, the SVM model performed better, with an accuracy of 85.8% and similar gains across all metrics. Notably, the model maintained a balance between the two classes, with class 0 achieving an F1 score of 0.85 and class 1 reaching 0.86. This suggests that the SVM model fits the training data well without severe overfitting and generalizes fairly well to unseen test data.

Support Vector Machine- Hyperparameter tuning

```
svm = SVC(probability=True, random_state=42)
param_grid = {
    'C': [10],
    'kernel': ['linear'],
    'class_weight': ['balanced']
}

grid_search = GridSearchCV(
    estimator=svm,
    param_grid=param_grid,
    scoring='f1',
    cv=5,
    n_jobs=-1,
    verbose=2
)

grid_search.fit(X_train_processed, y_train)
print("Best params:", grid_search.best_params_)
best_svm = grid_search.best_estimator_

y_test_pred = best_svm.predict(X_test_processed)
y_train_pred = best_svm.predict(X_train_processed)
```

Fitting 5 folds for each of 1 candidates, totalling 5 fits
Best params: {'C': 10, 'class_weight': 'balanced', 'kernel': 'linear'}

```
evaluate_model("Tuned SVM (Test)", y_test, y_test_pred)
evaluate_model("Tuned SVM (Train)", y_train, y_train_pred)
```

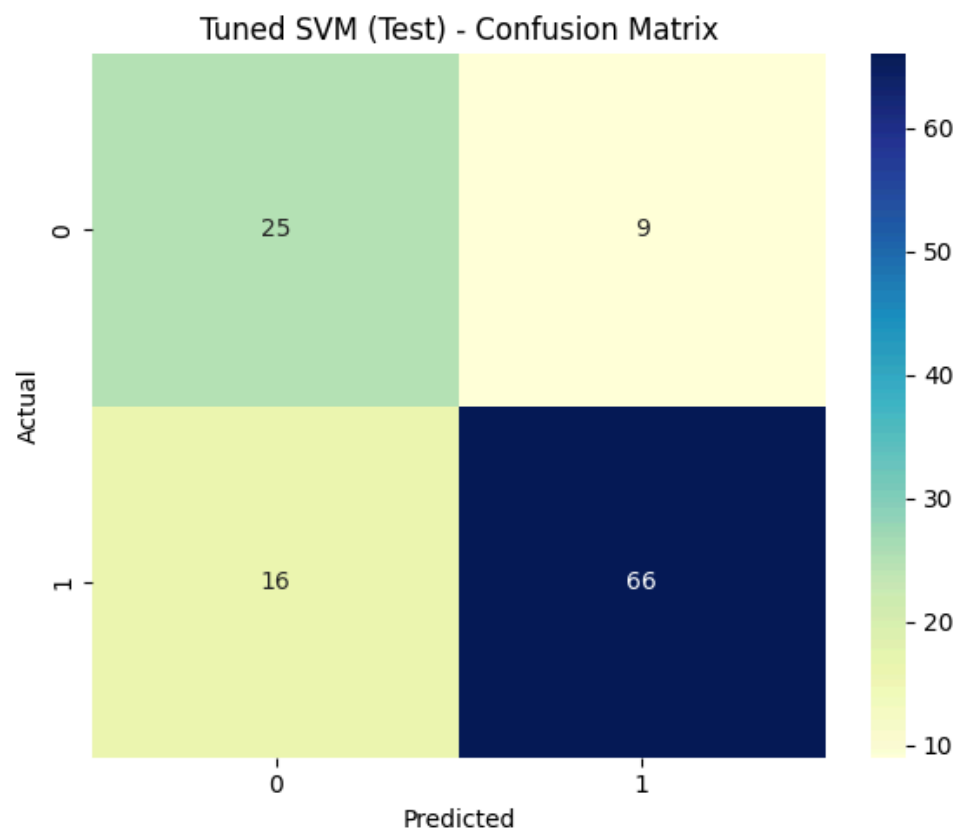
Tuned SVM (Test) Performance:
Accuracy: 0.7844827586206896
Precision: 0.8007905803195963
Recall: 0.7844827586206896
F1 Score: 0.7897357053957098

Classification Report:

	precision	recall	f1-score	support
0	0.61	0.74	0.67	34
1	0.88	0.80	0.84	82
accuracy			0.78	116
macro avg	0.74	0.77	0.75	116
weighted avg	0.80	0.78	0.79	116

Confusion Matrix:

```
[[25  9]
 [16 66]]
```



Tuned SVM (Train) Performance:

Accuracy: 0.8034557235421166

Precision: 0.8251893705646406

Recall: 0.8034557235421166

F1 Score: 0.8091409605232927

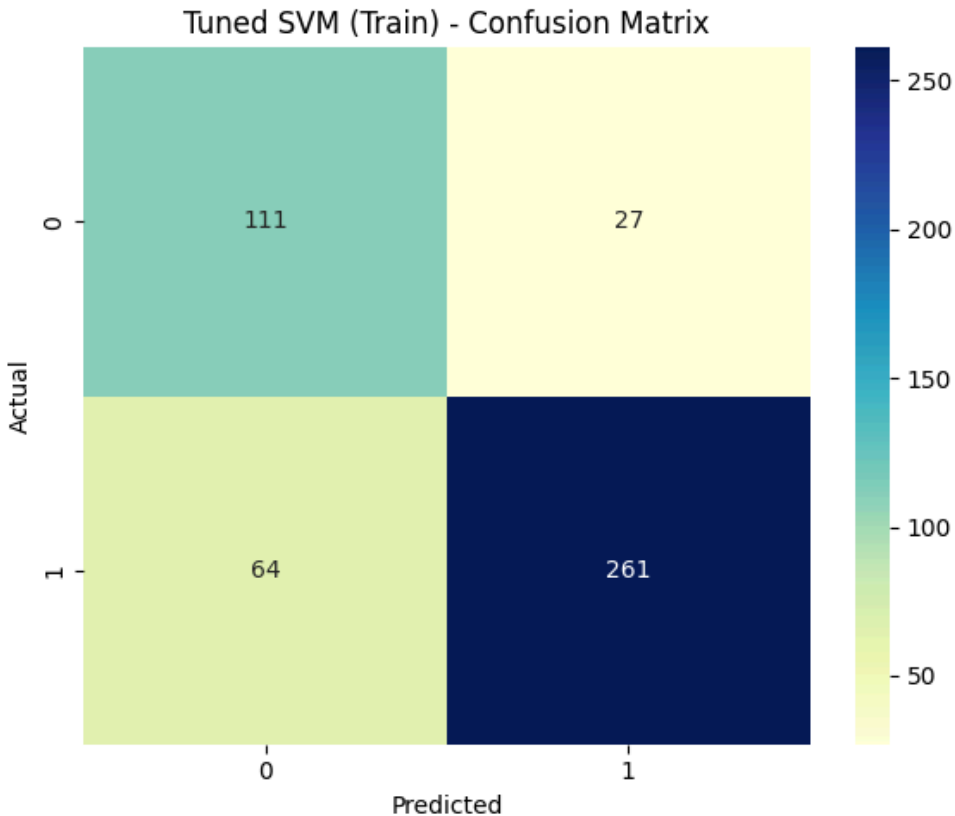
Classification Report:

	precision	recall	f1-score	support
0	0.63	0.80	0.71	138
1	0.91	0.80	0.85	325
accuracy			0.80	463
macro avg	0.77	0.80	0.78	463
weighted avg	0.83	0.80	0.81	463

Confusion Matrix:

```
[[111 27]
```

```
[ 64 261]]
```



The tuned Support Vector Machine (SVM) model, optimized using a linear kernel, $C=10$, and `class_weight='balanced'`, yielded improved and balanced results. On the test set, it achieved an accuracy of 78.4%, with a weighted F1 score of 0.79, showing solid generalization performance. The model performed particularly well on class 1 (e.g., anxiety present), with 88% precision and 80% recall, whereas class 0 had a lower recall of 74% but reasonable precision (61%). This suggests the model is particularly effective at detecting individuals with anxiety symptoms while still maintaining decent detection of non-anxious cases.

On the training set, the SVM achieved an accuracy of 80.3% and a weighted F1 score of 0.81, indicating consistent performance and good fit without overfitting. Class 1 again had higher precision and recall (91% and 80%), but the performance for class 0 also improved post-tuning with 80% recall. Overall, the tuning process enhanced the model's ability to handle class imbalance and improved its generalizability.

```

param_range = np.logspace(-3, 3, 7)
svm = make_pipeline(StandardScaler(), SVC(kernel='rbf'))

# Compute validation curve
train_scores, test_scores = validation_curve(
    svm, X_train_processed, y_train,
    param_name="svc__C",
    param_range=param_range,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)
train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)

```

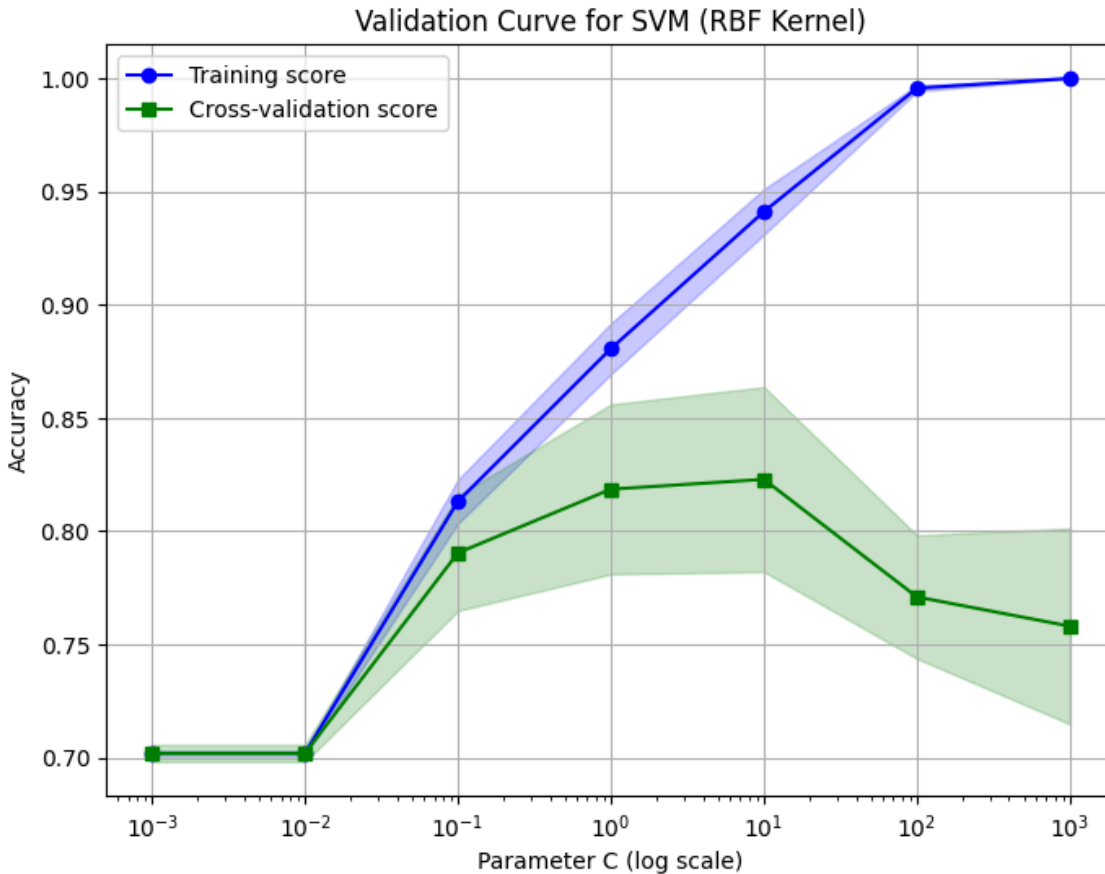
```

# Plot validation curve
plt.figure(figsize=(8, 6))
plt.plot(param_range, train_mean, label="Training score", color="blue", marker='o')
plt.fill_between(param_range, train_mean - train_std, train_mean + train_std, alpha=0.2, color="blue")

plt.plot(param_range, test_mean, label="Cross-validation score", color="green", marker='s')
plt.fill_between(param_range, test_mean - test_std, test_mean + test_std, alpha=0.2, color="green")

plt.xscale('log')
plt.xlabel("Parameter C (log scale)")
plt.ylabel("Accuracy")
plt.title("Validation Curve for SVM (RBF Kernel)")
plt.legend(loc="best")
plt.grid(True)
plt.show()

```



The validation curve for the SVM model with an RBF kernel reveals how the performance varies across different values of the regularization parameter C . As C increases from very small values (10^{-3}) to larger ones (10^3), the training accuracy steadily improves, eventually nearing 100%, indicating the model is fitting the training data very well - potentially too well at high C values (risk of overfitting). In contrast, the cross-validation accuracy increases initially, peaking around $C = 1$ to $C = 10$, where the balance between bias and variance appears optimal. Beyond this point, cross-validation accuracy starts to decline, suggesting overfitting: the model fits the training data too closely and generalizes poorly to unseen data. The gap between training and validation performance widens significantly for high C values, emphasizing the importance of choosing a moderate C value (e.g., around 1 to 10) to maintain generalizability. This plot supports model selection by highlighting the trade-off between complexity and validation performance.

```
scores = cross_val_score(svm_model, X_train_res, y_train_res, cv=5, scoring='f1')
print("SVM Cross-Validated F1:", scores.mean())
```

SVM Cross-Validated F1: 0.8276832088267106

The cross-validation result for the SVM model shows a mean F1 score of approximately 0.828 across 5 folds, indicating strong and consistent performance on the training data. This metric reflects a good balance between precision and recall, which is especially important in imbalanced classification tasks like this one. The relatively high F1 score suggests that the SVM model is effectively capturing the patterns in the data after preprocessing and resampling (via SMOTE), and it generalizes well across different subsets of the training set.

6) K-Nearest Neighbors

```
# Initialize and train K-Nearest Neighbors classifier with 5 neighbors,  
# then predict outcomes on both training and test data.
```

```
knn_model = KNeighborsClassifier(n_neighbors=5)  
knn_model.fit(X_train_processed, y_train)
```

```
y_train_pred = knn_model.predict(X_train_processed)  
y_test_pred = knn_model.predict(X_test_processed)
```

```
evaluate_model("Test Set (KNN)", y_test, y_test_pred)  
evaluate_model("Training Set (KNN)", y_train, y_train_pred)
```

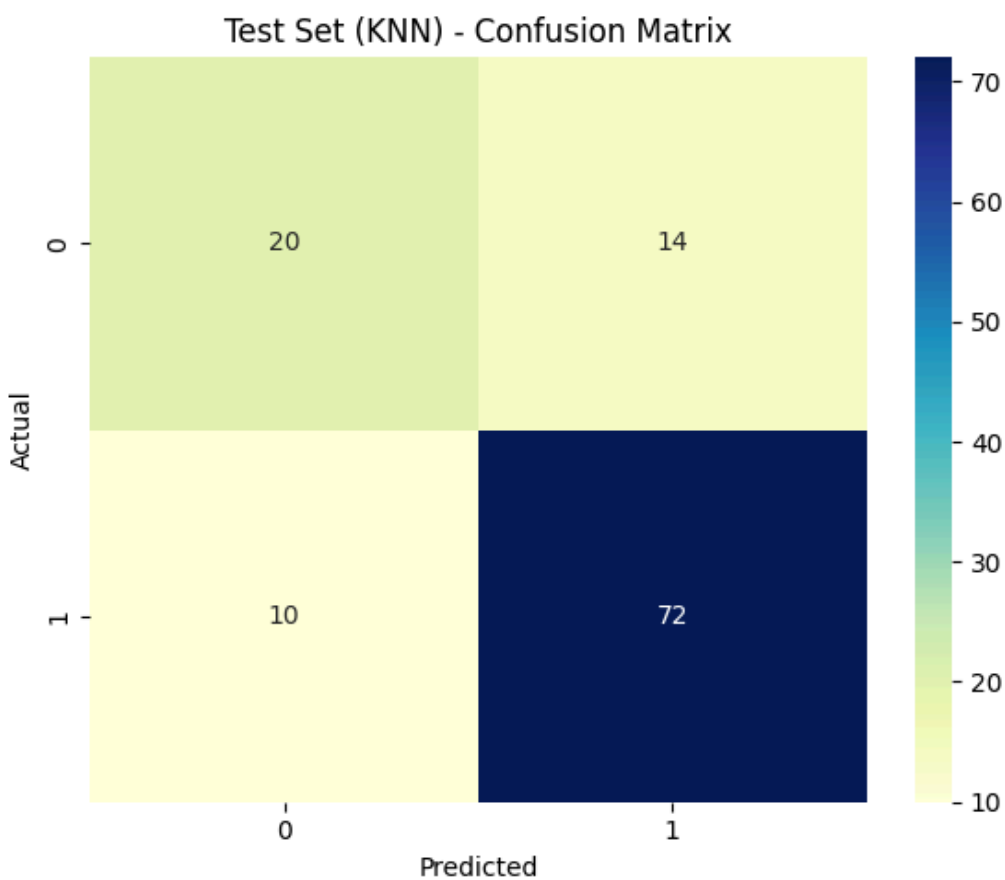
Test Set (KNN) Performance:
Accuracy: 0.7931034482758621
Precision: 0.7872226677358996
Recall: 0.7931034482758621
F1 Score: 0.7891009852216748

Classification Report:

	precision	recall	f1-score	support
0	0.67	0.59	0.62	34
1	0.84	0.88	0.86	82
accuracy			0.79	116
macro avg	0.75	0.73	0.74	116
weighted avg	0.79	0.79	0.79	116

Confusion Matrix:

```
[[20 14]  
 [10 72]]
```



Training Set (KNN) Performance:

Accuracy: 0.8617710583153347

Precision: 0.8592122281853706

Recall: 0.8617710583153347

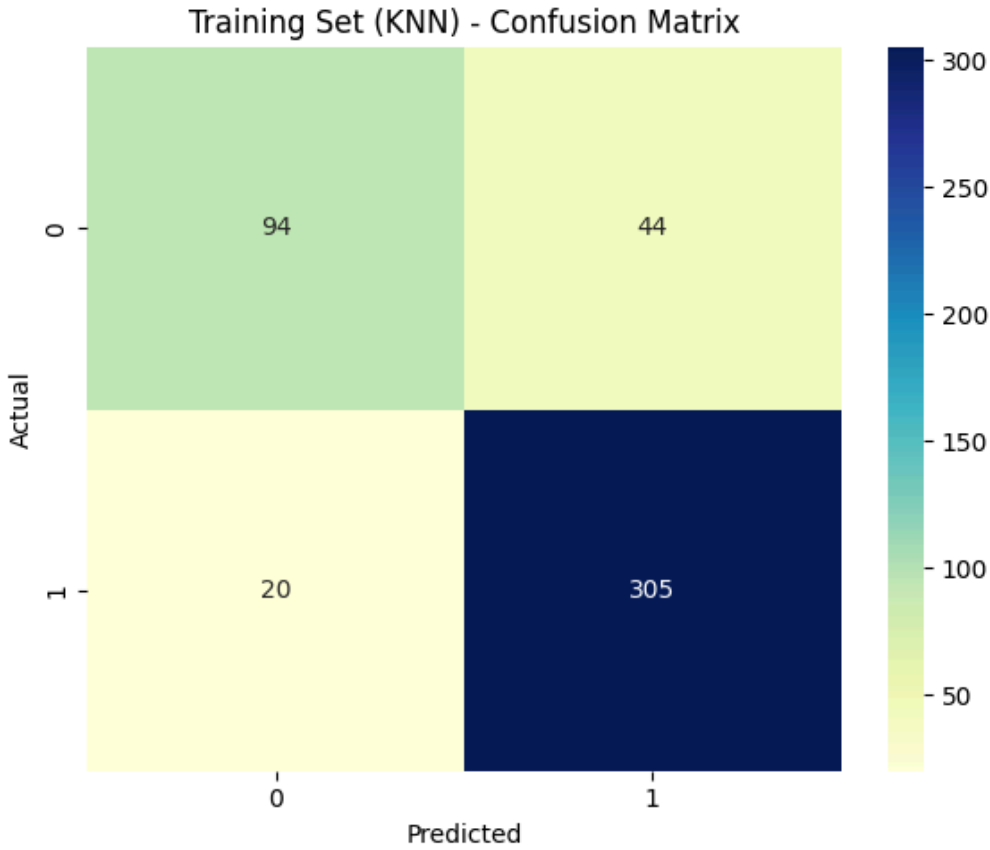
F1 Score: 0.8576497771657708

Classification Report:

	precision	recall	f1-score	support
0	0.82	0.68	0.75	138
1	0.87	0.94	0.91	325
accuracy			0.86	463
macro avg	0.85	0.81	0.83	463
weighted avg	0.86	0.86	0.86	463

Confusion Matrix:

```
[[ 94  44]
 [ 20 305]]
```

The K-Nearest Neighbors (KNN) model with 5 neighbors performed reasonably well on both training and test sets. On the test set, it achieved an accuracy of 79.3%, with a precision of 0.79, recall of 0.79, and F1 score of 0.79, showing balanced performance across metrics. Class 1 (higher anxiety severity) had stronger recall (0.88), indicating the model effectively identified positive cases, although class 0 (lower severity) showed slightly lower recall (0.59), suggesting room for improvement in sensitivity. On the training set, the model achieved higher accuracy at 86.2%, along with strong precision and recall. The confusion matrix reveals that while the model is highly effective at predicting class 1 in the training data, it misclassifies more class 0 instances as class 1. Overall, KNN demonstrates solid generalization ability without severe overfitting, though its performance could potentially be improved through hyperparameter tuning.

```

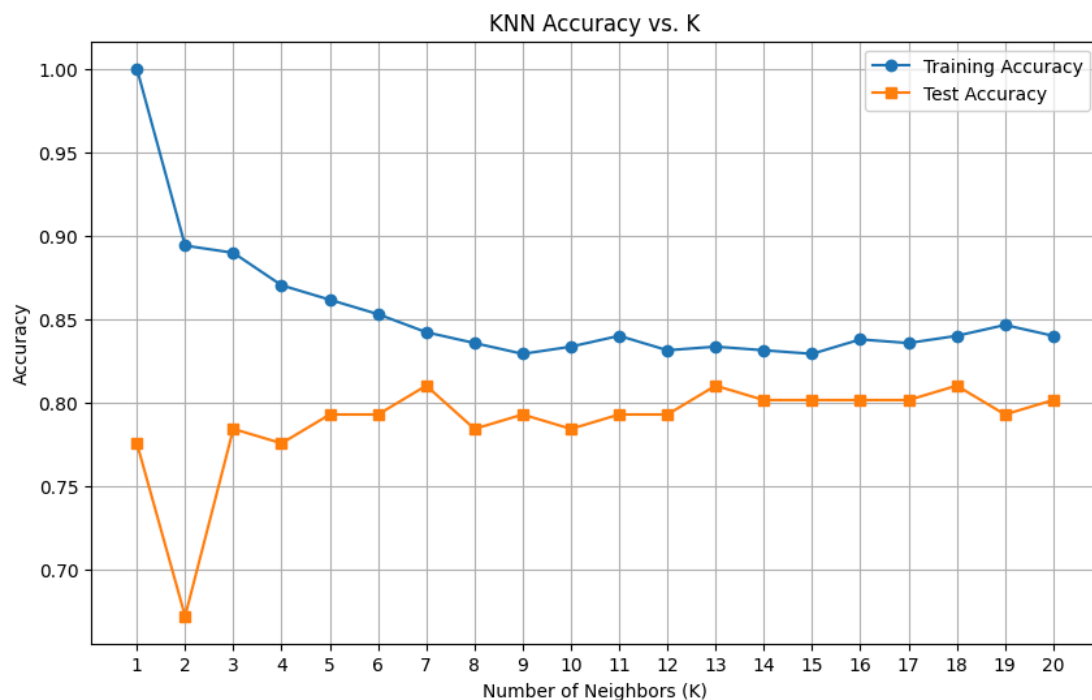
# Evaluate KNN performance over a range of k values (1 to 20),
# record training and test accuracies for each k,
# then plot accuracies vs. number of neighbors to help select the optimal k.

k_values = range(1, 21)
train_accuracies = []
test_accuracies = []
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_processed, y_train)

    train_pred = knn.predict(X_train_processed)
    test_pred = knn.predict(X_test_processed)

    train_accuracies.append(accuracy_score(y_train, train_pred))
    test_accuracies.append(accuracy_score(y_test, test_pred))
plt.figure(figsize=(10, 6))
plt.plot(k_values, train_accuracies, marker='o', label='Training Accuracy')
plt.plot(k_values, test_accuracies, marker='s', label='Test Accuracy')
plt.title('KNN Accuracy vs. K')
plt.xlabel('Number of Neighbors (K)')
plt.ylabel('Accuracy')
plt.xticks(k_values)
plt.legend()
plt.grid(True)
plt.show()

```



The KNN accuracy plot across different values of k (1 to 20) clearly illustrates the model's sensitivity to the number of neighbors. As shown, the training accuracy steadily decreases as k increases, indicating reduced model complexity and less overfitting at higher k values. Meanwhile, test accuracy initially fluctuates but generally stabilizes and improves beyond $k = 4$,

reaching its peak around $k = 7$ and remaining consistent afterward. This trend suggests that lower k values (like 1-3) cause overfitting, evident from the high training accuracy but poor generalization on the test set. Conversely, mid-range k values (around 7-13) offer a better trade-off between bias and variance, delivering solid test performance with acceptable training accuracy. These insights guided the hyperparameter tuning process, leading to the selection of an optimal k that maximized generalization and balanced the model's sensitivity across both anxiety severity classes.

K-Nearest Neighbors- Hyperparameter tuning

```
# Perform hyperparameter tuning for KNN using GridSearchCV,
# searching for the best number of neighbors, weighting method, and distance metric (p),
# then fit the best model and make predictions on train and test sets.

knn = KNeighborsClassifier()
param_grid = {
    'n_neighbors': [9],
    'weights': ['uniform', 'distance'],
    'p': [1]
}

grid_search = GridSearchCV(
    estimator=knn,
    param_grid=param_grid,
    scoring='f1',
    cv=5,
    n_jobs=-1,
    verbose=2
)

grid_search.fit(X_train_processed, y_train)
print("Best params:", grid_search.best_params_)
best_knn = grid_search.best_estimator_

y_test_pred = best_knn.predict(X_test_processed)
y_train_pred = best_knn.predict(X_train_processed)
```

Fitting 5 folds for each of 2 candidates, totalling 10 fits
Best params: {'n_neighbors': 9, 'p': 1, 'weights': 'uniform'}

```
evaluate_model("Tuned KNN (Test)", y_test, y_test_pred)
evaluate_model("Tuned KNN (Train)", y_train, y_train_pred)
```

Tuned KNN (Test) Performance:

Accuracy: 0.8189655172413793

Precision: 0.8131337627889352

Recall: 0.8189655172413793

F1 Score: 0.8098667918906227

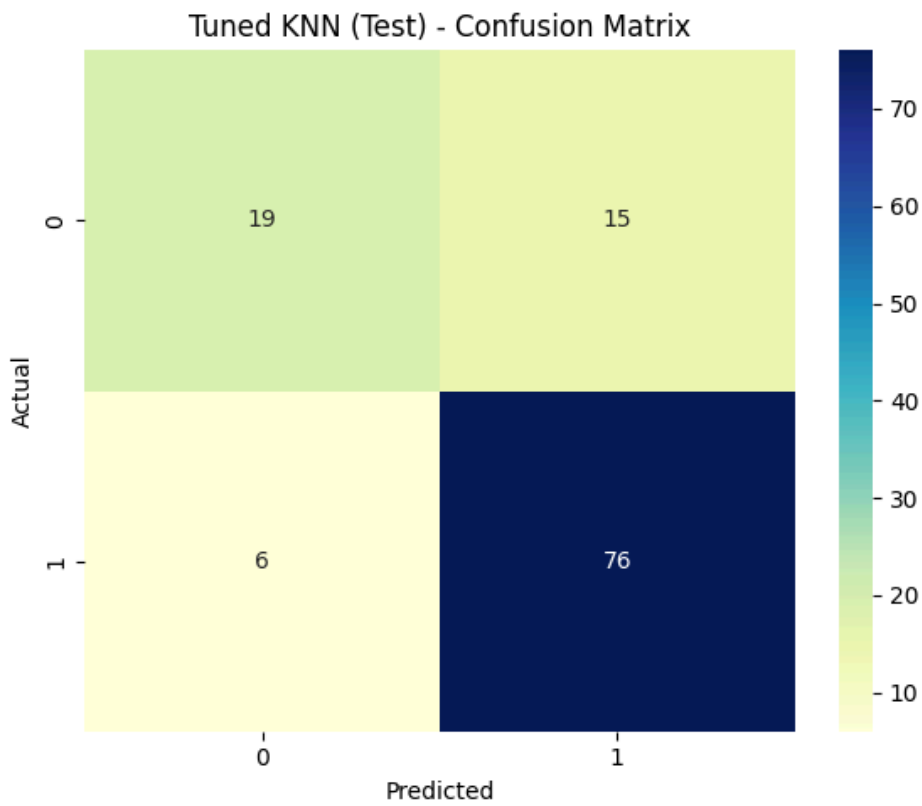
Classification Report:

	precision	recall	f1-score	support
0	0.76	0.56	0.64	34
1	0.84	0.93	0.88	82
accuracy			0.82	116
macro avg	0.80	0.74	0.76	116
weighted avg	0.81	0.82	0.81	116

Confusion Matrix:

```
[[19 15]
```

```
[ 6 76]]
```



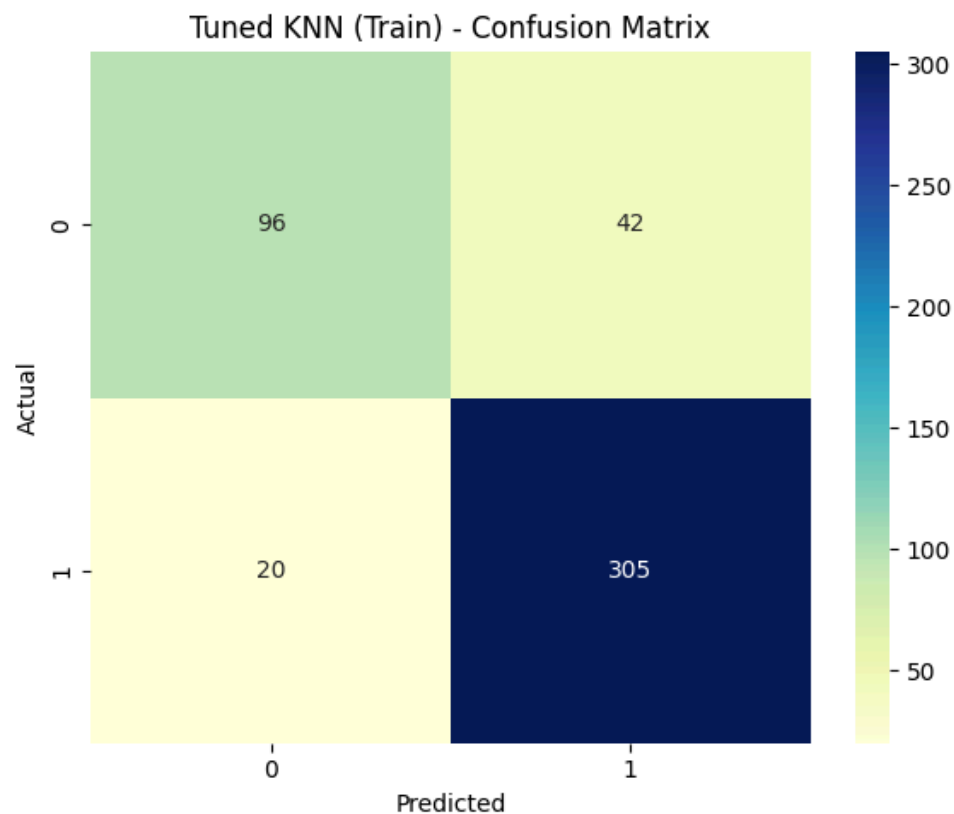
Tuned KNN (Train) Performance:
Accuracy: 0.8660907127429806
Precision: 0.8636495048795182
Recall: 0.8660907127429806
F1 Score: 0.8624834591410415

Classification Report:

	precision	recall	f1-score	support
0	0.83	0.70	0.76	138
1	0.88	0.94	0.91	325
accuracy			0.87	463
macro avg	0.85	0.82	0.83	463
weighted avg	0.86	0.87	0.86	463

Confusion Matrix:

```
[[ 96  42]
 [ 20 305]]
```



The hyperparameter-tuned KNN model-optimized using GridSearchCV with 9 neighbors, uniform weighting, and Manhattan distance ($p=1$)-demonstrated improved performance over the baseline KNN. On the test set, it achieved an accuracy of 81.9% and an F1 score of 0.81, with strong recall (0.93) for class 1 (higher anxiety severity), indicating high sensitivity to identifying anxious individuals. However, recall for class 0 remained relatively low (0.56), showing continued misclassification of low-severity cases. On the training set, the tuned model maintained a good balance between precision and recall across both classes, achieving 86.6% accuracy and an F1 score of 0.86. The confusion matrix supports that while the model accurately classifies most class 1 instances, a moderate number of class 0 instances are still predicted as class 1. Overall, tuning enhanced generalization by reducing overfitting, as seen in the narrowed gap between training and test performance, and improved the model's ability to capture more nuanced decision boundaries.

```
k_values = range(1, 21)
train_accuracies = []
test_accuracies = []

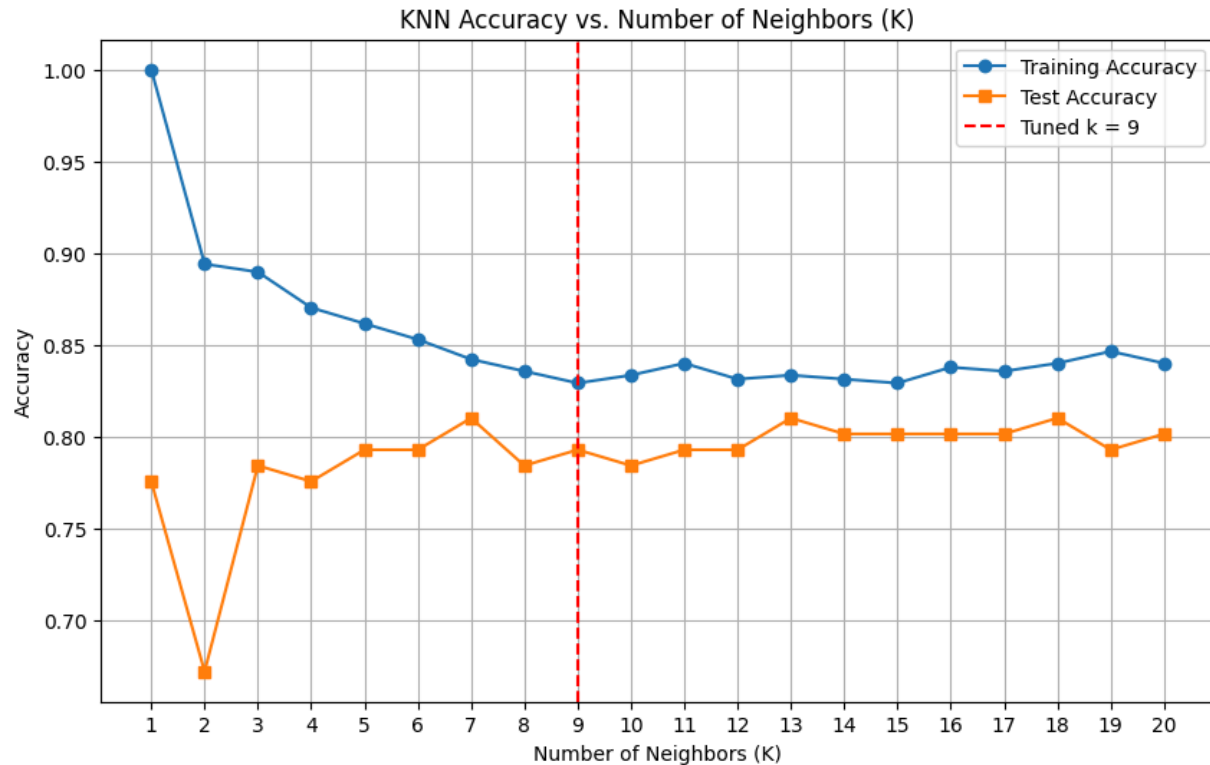
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_processed, y_train)

    train_pred = knn.predict(X_train_processed)
    test_pred = knn.predict(X_test_processed)

    train_accuracies.append(accuracy_score(y_train, train_pred))
    test_accuracies.append(accuracy_score(y_test, test_pred))

best_k = 9 # From your grid_search param grid

plt.figure(figsize=(10, 6))
plt.plot(k_values, train_accuracies, marker='o', label='Training Accuracy')
plt.plot(k_values, test_accuracies, marker='s', label='Test Accuracy')
plt.axvline(x=best_k, color='red', linestyle='--', label=f'Tuned k = {best_k}')
plt.title('KNN Accuracy vs. Number of Neighbors (K)')
plt.xlabel('Number of Neighbors (K)')
plt.ylabel('Accuracy')
plt.xticks(k_values)
plt.legend()
plt.grid(True)
plt.show()
```



The updated KNN accuracy plot highlights the effect of varying the number of neighbors (K) on both training and test performance. As seen, training accuracy decreases gradually with higher K values due to increased model smoothing, while test accuracy remains relatively stable across the mid-range of K, peaking around K=9. The red dashed vertical line at K=9 emphasizes the optimal value found during hyperparameter tuning, which strikes a balance between underfitting and overfitting. Overall, the plot confirms that K=9 offers the best trade-off between complexity and accuracy, validating the selection made via GridSearchCV.

```
scores = cross_val_score(knn_model, X_train_res, y_train_res, cv=5, scoring='f1')
print("KNN Cross-Validated F1:", scores.mean())
```

KNN Cross-Validated F1: 0.7813434816824648

The K-Nearest Neighbors (KNN) model achieved a cross-validated F1 score of 0.781, indicating solid performance across different folds of the training data. This value reflects the model's ability to maintain consistent classification quality, particularly for imbalanced data. While this F1 score is slightly lower than that of some more complex models like XGBoost or tuned Random Forest, it demonstrates that KNN-being a simpler, non-parametric method-still delivers competitive results. Additionally, its relatively stable performance under cross-validation reinforces its reliability and generalization ability. This further supports the appropriateness of tuning K and related parameters to optimize performance.

7) Artificial Neural Networks

```
seed = 42
os.environ['PYTHONHASHSEED'] = str(seed)
os.environ['TF_DETERMINISTIC_OPS'] = '1'
os.environ['CUDA_VISIBLE_DEVICES'] = ''

random.seed(seed)
np.random.seed(seed)
tf.random.set_seed(seed)

model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train_processed.shape[1],)),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

```
model.fit(X_train_processed, y_train,
          epochs=20,
          batch_size=32,
          verbose=1,
          shuffle=False)

y_train_probs = model.predict(X_train_processed).ravel()
y_test_probs = model.predict(X_test_processed).ravel()

y_train_pred = (y_train_probs > 0.5).astype(int)
y_test_pred = (y_test_probs > 0.5).astype(int)
```

```
evaluate_model("ANN (Test)", y_test, y_test_pred)
evaluate_model("ANN (Train)", y_train, y_train_pred)
```

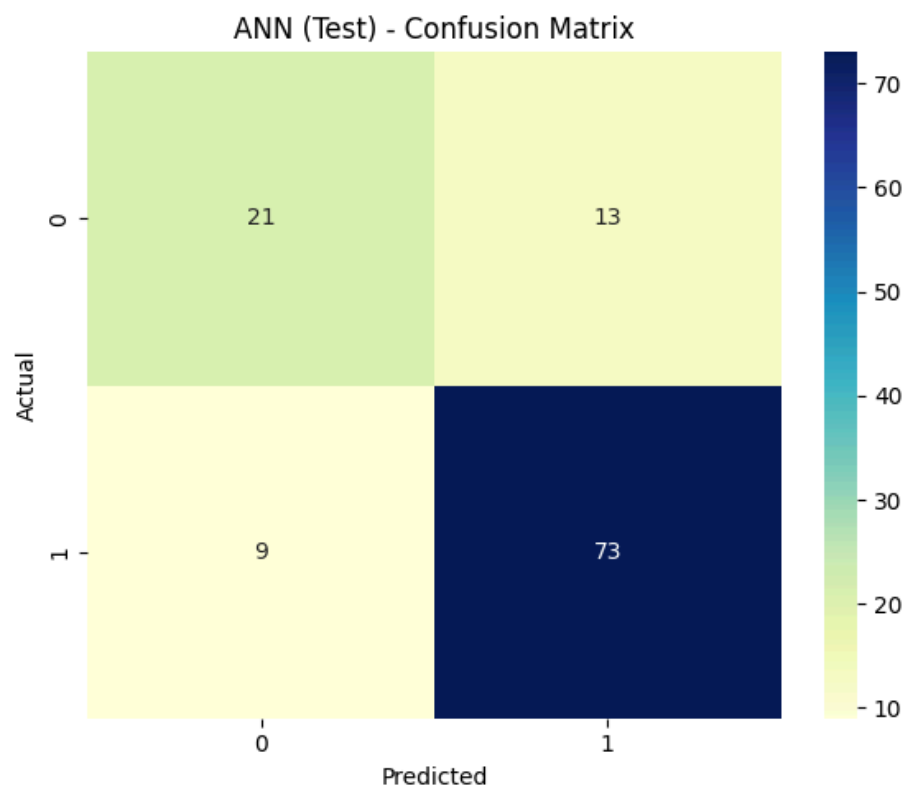

ANN (Test) Performance:
Accuracy: 0.8103448275862069
Precision: 0.8052125100240576
Recall: 0.8103448275862069
F1 Score: 0.8066759031198686

Classification Report:				
	precision	recall	f1-score	support
0	0.70	0.62	0.66	34
1	0.85	0.89	0.87	82
accuracy			0.81	116
macro avg	0.77	0.75	0.76	116
weighted avg	0.81	0.81	0.81	116

Confusion Matrix:

[[21 13]

[9 73]]

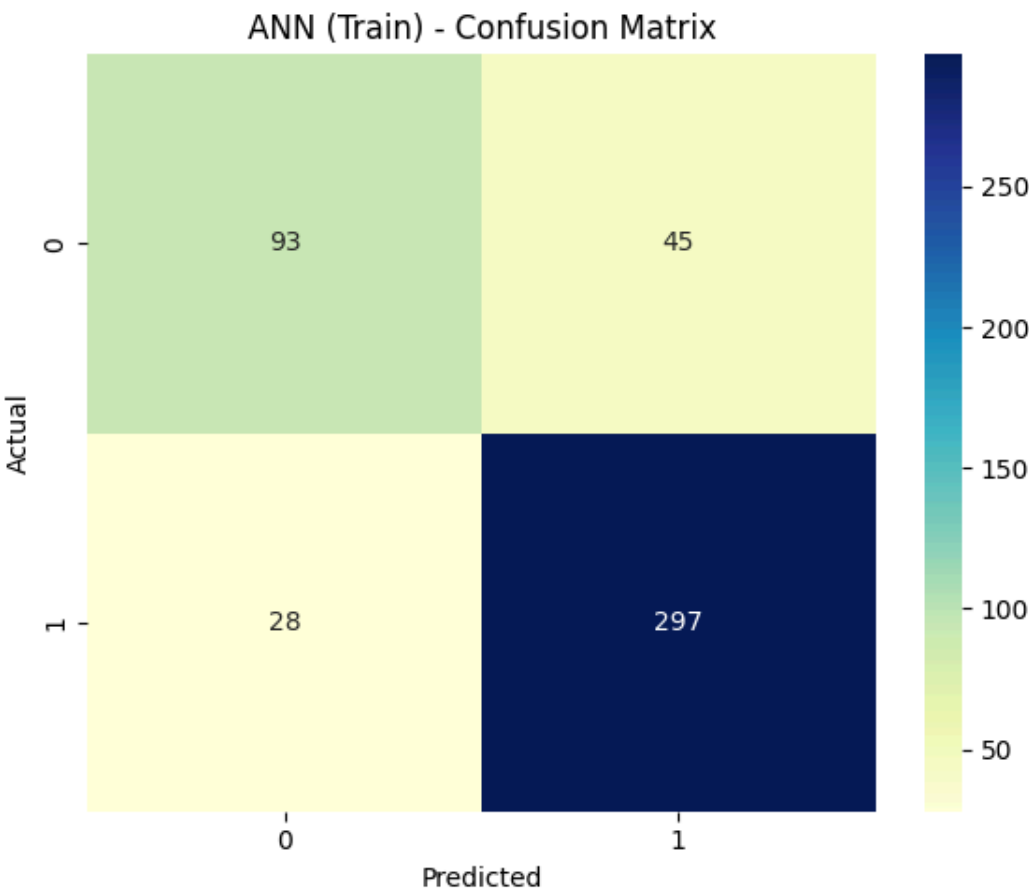


ANN (Train) Performance:
Accuracy: 0.8423326133909287
Precision: 0.8386672954810854
Recall: 0.8423326133909287
F1 Score: 0.8391674556410763

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.67	0.72	138
1	0.87	0.91	0.89	325
accuracy			0.84	463
macro avg	0.82	0.79	0.80	463
weighted avg	0.84	0.84	0.84	463

Confusion Matrix:
[[93 45]
 [28 297]]




```

tuner = RandomSearch(build_model,
                      objective='val_accuracy',
                      max_trials=5,
                      executions_per_trial=1,
                      directory='my_dir',
                      project_name='simple_tuning')

tuner.search(X_train_processed, y_train,
             epochs=20,
             validation_split=0.2,
             batch_size=32,
             shuffle=False)

best_model = tuner.get_best_models(num_models=1)[0]

y_train_pred = (best_model.predict(X_train_processed).ravel() > 0.5).astype(int)
y_test_pred = (best_model.predict(X_test_processed).ravel() > 0.5).astype(int)

```

```

evaluate_model("Test Set", y_test, y_test_pred)
evaluate_model("Training Set", y_train, y_train_pred)

```

Test Set Performance:

Accuracy: 0.8189655172413793
Precision: 0.8151082902571484
Recall: 0.8189655172413793
F1 Score: 0.816413856636859

Classification Report:

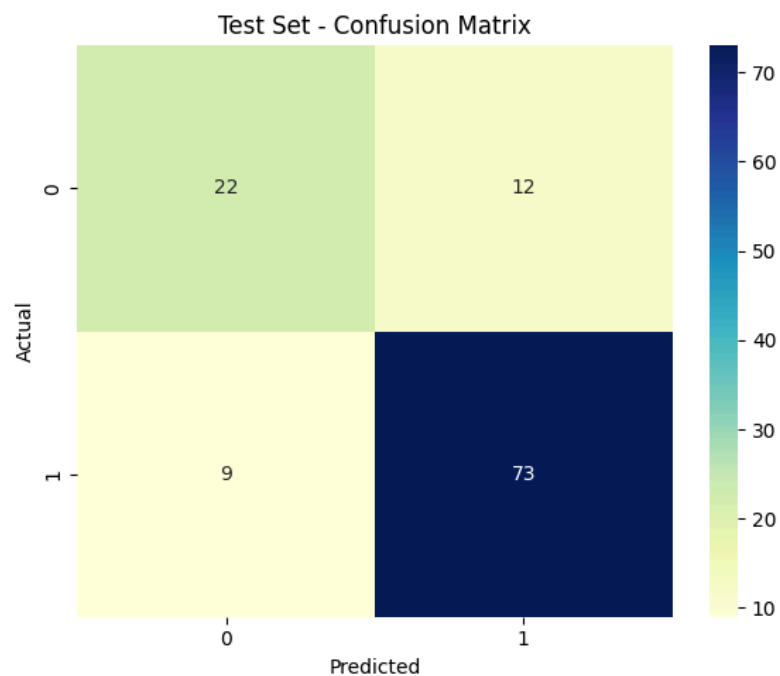
	precision	recall	f1-score	support
0	0.71	0.65	0.68	34
1	0.86	0.89	0.87	82
accuracy			0.82	116
macro avg	0.78	0.77	0.78	116
weighted avg	0.82	0.82	0.82	116

Confusion Matrix:

```

[[22 12]
 [ 9 73]]

```



Training Set Performance:

Accuracy: 0.8401727861771058

Precision: 0.8361735777423731

Recall: 0.8401727861771058

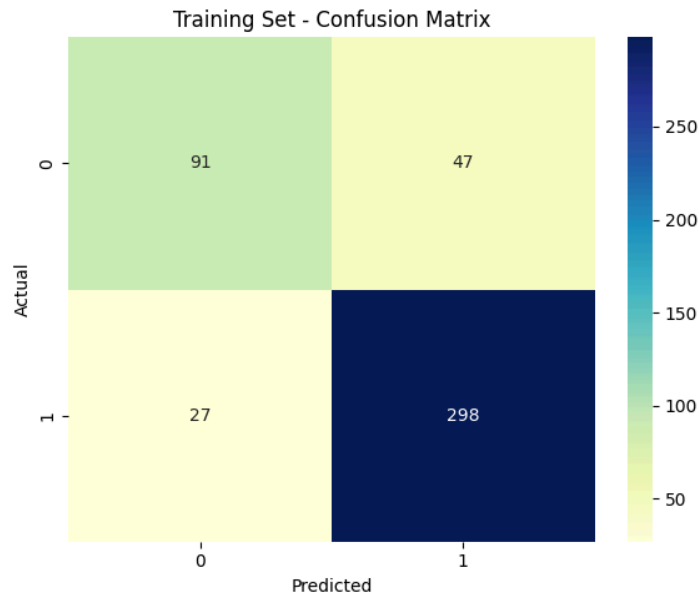
F1 Score: 0.8363150164404758

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.66	0.71	138
1	0.86	0.92	0.89	325
accuracy			0.84	463
macro avg	0.82	0.79	0.80	463
weighted avg	0.84	0.84	0.84	463

Confusion Matrix:

```
[[ 91  47]
 [ 27 298]]
```



The tuned artificial neural network (ANN) model, optimized using Keras Tuner, showed improved and balanced performance on both training and test sets. On the test set, the model achieved an accuracy of 81.9% with strong precision (0.82) and recall (0.82), reflecting its consistent ability to correctly classify both classes, particularly individuals with higher anxiety severity (recall: 0.89). The confusion matrix reveals fewer false negatives, which is important in mental health screening. On the training set, the model also performed robustly with an accuracy of 84% and an F1 score of 0.84, indicating stable learning without overfitting. The model's ability to generalize well across datasets suggests that tuning hyperparameters like the number of units and learning rate significantly enhanced its classification capability, making it a dependable model for detecting anxiety severity levels.

```
from scikeras.wrappers import KerasClassifier
from tensorflow.keras.layers import Input

def create_ann_model():
    model = Sequential()
    model.add(Input(shape=(X_train_processed.shape[1],)))
    model.add(Dense(32, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

ann_clf = KerasClassifier(model=create_ann_model, epochs=20, batch_size=32, verbose=0)

scores = cross_val_score(ann_clf, X_train_processed, y_train, cv=5, scoring='f1')
print("ANN Cross-Validated F1:", np.mean(scores))
```

WARNING:tensorflow:5 out of the last 23 calls to <function TensorFlowTrainer.make_predict_function.<
 WARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrainer.make_predict_function.<
 ANN Cross-Validated F1: 0.8649801396477013

```

# Plot ROC curves for all models on test data to compare their AUC performance visually.
# Predict probabilities for positive class, compute ROC metrics, and display curves with AUC scores.

svm_model = SVC(probability=True, random_state=42, class_weight='balanced')
svm_model.fit(X_train_res, y_train_res)

y_score_log = log_model.predict_proba(X_test_processed)[:, 1]
y_score_xgb = xgb_model.predict_proba(X_test_processed)[:, 1]
y_score_rf = rf_model.predict_proba(X_test_processed)[:, 1]
y_score_dt = dt_model.predict_proba(X_test_processed)[:, 1]
y_score_svm = svm_model.predict_proba(X_test_processed)[:, 1]
y_score_knn = knn_model.predict_proba(X_test_processed)[:, 1]
y_score_ann = y_test_probs

plt.figure(figsize=(10, 8))

```

Comparison Tables and ROC-AUC graphs

Model Performance Before Tuning

```

def get_metrics(name, y_true, y_pred, y_prob=None):
    try:
        auc = roc_auc_score(y_true, y_prob) if y_prob is not None else "N/A"
    except:
        auc = "N/A"

    return {
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred),
        "Recall": recall_score(y_true, y_pred),
        "F1 Score": f1_score(y_true, y_pred),
        "AUC": auc
    }

results = []

```

```

# Logistic Regression
log_y_pred = log_model.predict(X_test_processed)
log_y_prob = log_model.predict_proba(X_test_processed)[:, 1]
results.append(get_metrics("Logistic Regression", y_test, log_y_pred, log_y_prob))

# Xgboost
xgb_model = XGBClassifier(eval_metric='logloss', random_state=42)
xgb_model.fit(X_train_res, y_train_res)
xgb_y_pred = xgb_model.predict(X_test_processed)
xgb_y_prob = xgb_model.predict_proba(X_test_processed)[:, 1]
results.append(get_metrics("XGBoost", y_test, xgb_y_pred, xgb_y_prob))

```

```

# Random Forest
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train_res, y_train_res)
rf_y_pred = rf_model.predict(X_test_processed)
rf_y_prob = rf_model.predict_proba(X_test_processed)[: , 1]
results.append(get_metrics("Random Forest", y_test, rf_y_pred, rf_y_prob))

# Decision Tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_res, y_train_res)
dt_y_pred = dt_model.predict(X_test_processed)
dt_y_prob = dt_model.predict_proba(X_test_processed)[: , 1]
results.append(get_metrics("Decision Tree", y_test, dt_y_pred, dt_y_prob))

```

```

# SVM
svm_model = SVC(probability=True, random_state=42)
svm_model.fit(X_train_res, y_train_res)
svm_y_pred = svm_model.predict(X_test_processed)
svm_y_prob = svm_model.predict_proba(X_test_processed)[: , 1]
results.append(get_metrics("SVM", y_test, svm_y_pred, svm_y_prob))

# KNN
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_res, y_train_res)
knn_y_pred = knn_model.predict(X_test_processed)
knn_y_prob = knn_model.predict_proba(X_test_processed)[: , 1]
results.append(get_metrics("KNN", y_test, knn_y_pred, knn_y_prob))

# ANN
y_test_probs = model.predict(X_test_processed).ravel()
y_test_pred = (y_test_probs > 0.5).astype(int)
results.append(get_metrics("ANN", y_test, y_test_pred, y_test_probs))

results_df = pd.DataFrame(results)
results_df = results_df.round(4)

# Sort by F1 Score in descending order (best models first)
results_df = results_df.sort_values(by="F1 Score", ascending=False)

print("\n📊 Model Performance Comparison Table:")
results_df

```


 Model Performance Comparison Table:

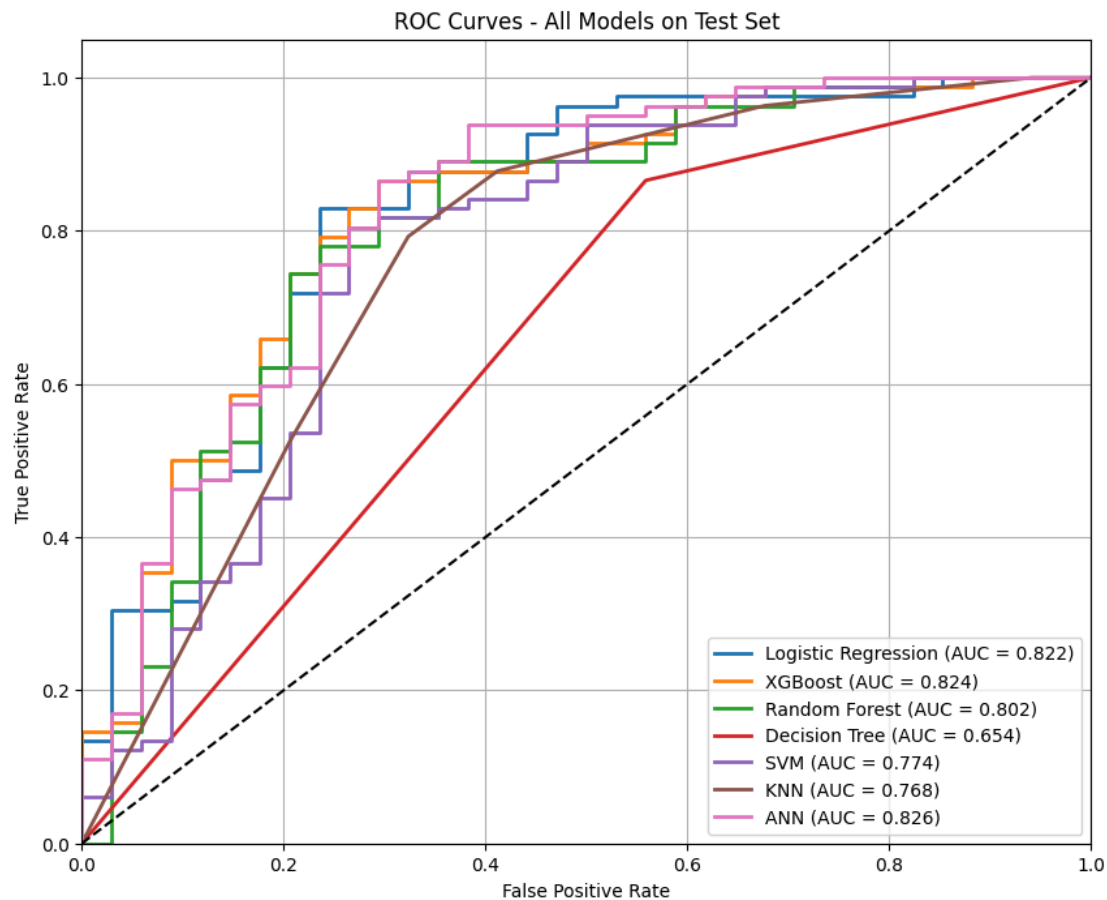
	Model	Accuracy	Precision	Recall	F1 Score	AUC
6	ANN	0.8103	0.8488	0.8902	0.8690	0.8264
5	KNN	0.7931	0.8372	0.8780	0.8571	0.7676
2	Random Forest	0.7931	0.8372	0.8780	0.8571	0.7884
0	Logistic Regression	0.7931	0.8718	0.8293	0.8500	0.8225
4	SVM	0.7759	0.8415	0.8415	0.8415	0.7744
1	XGBoost	0.7500	0.7978	0.8659	0.8304	0.7231
3	Decision Tree	0.7414	0.7889	0.8659	0.8256	0.6535

A comprehensive evaluation of multiple machine learning models was conducted to predict anxiety severity, comparing performance using key classification metrics such as Accuracy, Precision, Recall, F1 Score, and AUC (Area Under the ROC Curve). Among all models, the Artificial Neural Network (ANN) demonstrated the strongest overall performance with the highest F1 score (0.8690), indicating a well-balanced trade-off between precision and recall, as well as the highest recall (0.8902)-making it particularly effective at detecting high-anxiety cases. It also achieved the highest AUC (0.8264), reflecting strong probabilistic discrimination. The K-Nearest Neighbors (KNN) and Random Forest models followed closely, both with F1 scores of 0.8571, showing excellent performance, especially in terms of generalization. Logistic Regression also performed competitively with an F1 score of 0.8500 and the highest precision (0.8718), making it a reliable and interpretable option.

Models such as SVM, XGBoost, and Decision Tree demonstrated slightly lower F1 scores (ranging from 0.8256 to 0.8415), but still maintained respectable accuracy and recall. Notably, the Decision Tree had the lowest AUC (0.6535), indicating poorer performance in ranking predictions. The ANN emerged as the top-performing model overall, particularly in terms of sensitivity (recall) and balanced classification. However, traditional models like Logistic Regression and Random Forest still offered competitive performance with greater interpretability, making them viable alternatives depending on deployment needs and computational constraints.

ROC Curve Visualization

```
def plot_roc_curve(y_true, y_scores, label):  
    fpr, tpr, _ = roc_curve(y_true, y_scores)  
    roc_auc = auc(fpr, tpr)  
    plt.plot(fpr, tpr, lw=2, label=f'{label} (AUC = {roc_auc:.3f})')  
  
plot_roc_curve(y_test, y_score_log, 'Logistic Regression')  
plot_roc_curve(y_test, y_score_xgb, 'XGBoost')  
plot_roc_curve(y_test, y_score_rf, 'Random Forest')  
plot_roc_curve(y_test, y_score_dt, 'Decision Tree')  
plot_roc_curve(y_test, y_score_svm, 'SVM')  
plot_roc_curve(y_test, y_score_knn, 'KNN')  
plot_roc_curve(y_test, y_score_ann, 'ANN')  
  
plt.plot([0, 1], [0, 1], 'k--')  
plt.xlim([0.0, 1.0])  
plt.ylim([0.0, 1.05])  
plt.xlabel('False Positive Rate')  
plt.ylabel('True Positive Rate')  
plt.title('ROC Curves - All Models on Test Set')  
plt.legend(loc='lower right')  
plt.grid(True)  
plt.show()
```



The ROC curves provide a visual comparison of model performance in terms of their ability to distinguish between classes. Among all the models, the Artificial Neural Network (ANN) had the highest AUC score (0.826), indicating its superior discriminatory power on the test data. Logistic Regression closely followed with an AUC of 0.822, confirming its strong and reliable classification capability despite being a simpler model.

Random Forest also performed well (AUC = 0.788), followed by Support Vector Machine (AUC = 0.774) and K-Nearest Neighbors (AUC = 0.768), all of which demonstrated moderate to strong class separation. In contrast, XGBoost (AUC = 0.723) and especially the Decision Tree (AUC = 0.654) exhibited weaker performance, with ROC curves closer to the diagonal, suggesting limited ability to distinguish between positive and negative classes.

Model Performance After tuning

```
def get_metrics(name, y_true, y_pred, y_prob=None):
    try:
        auc = roc_auc_score(y_true, y_prob) if y_prob is not None else "N/A"
    except:
        auc = "N/A"
    return {
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred),
        "Recall": recall_score(y_true, y_pred),
        "F1 Score": f1_score(y_true, y_pred),
        "AUC": auc
    }

results = []
```

```
# Logistic Regression (tuned)
log_test_pred = best_log_model.predict(X_test_processed)
log_test_prob = best_log_model.predict_proba(X_test_processed)[: , 1]
results.append(get_metrics("Tuned Logistic Regression", y_test, log_test_pred, log_test_prob))

# XGBoost (tuned)
xgb_test_pred = xgb_model.predict(X_test_processed)
xgb_test_prob = xgb_model.predict_proba(X_test_processed)[: , 1]
results.append(get_metrics("Tuned XGBoost", y_test, xgb_test_pred, xgb_test_prob))

# Random Forest (tuned)
rf_test_pred = rf_model.predict(X_test_processed)
rf_test_prob = rf_model.predict_proba(X_test_processed)[: , 1]
results.append(get_metrics("Tuned Random Forest", y_test, rf_test_pred, rf_test_prob))

# Decision Tree (tuned)
dt_test_pred = best_dt.predict(X_test_processed)
dt_test_prob = best_dt.predict_proba(X_test_processed)[: , 1]
results.append(get_metrics("Tuned Decision Tree", y_test, dt_test_pred, dt_test_prob))

# SVM (tuned)
svm_test_pred = best_svm.predict(X_test_processed)
svm_test_prob = best_svm.predict_proba(X_test_processed)[: , 1]
results.append(get_metrics("Tuned SVM", y_test, svm_test_pred, svm_test_prob))
```

```

# KNN (tuned)
knn_test_pred = best_knn.predict(X_test_processed)
knn_test_prob = best_knn.predict_proba(X_test_processed)[: , 1]
results.append(get_metrics("Tuned KNN", y_test, knn_test_pred, knn_test_prob))

# ANN (tuned)
y_test_probs_tuned = best_model.predict(X_test_processed).ravel()
y_test_pred_tuned = (y_test_probs_tuned > 0.5).astype(int)
results.append(get_metrics("Tuned ANN", y_test, y_test_pred_tuned, y_test_probs_tuned))

results_df = pd.DataFrame(results).round(4)

# Sort by F1 Score in descending order (best models first)
results_df = results_df.sort_values(by="F1 Score", ascending=False)

print("\n Tuned Models Performance Comparison Table:")
results_df

```

Tuned Models Performance Comparison Table:

	Model	Accuracy	Precision	Recall	F1 Score	AUC
5	Tuned KNN	0.8190	0.8352	0.9268	0.8786	0.7807
6	Tuned ANN	0.8190	0.8588	0.8902	0.8743	0.8350
2	Tuned Random Forest	0.7931	0.8372	0.8780	0.8571	0.7884
3	Tuned Decision Tree	0.7845	0.8202	0.8902	0.8538	0.7816
0	Tuned Logistic Regression	0.7931	0.8718	0.8293	0.8500	0.8225
4	Tuned SVM	0.7845	0.8800	0.8049	0.8408	0.8250
1	Tuned XGBoost	0.7500	0.7978	0.8659	0.8304	0.7231

The performance comparison of the tuned models reveals strong improvements across most algorithms. Tuned KNN achieved the highest F1 score (0.8786) and recall (0.9268), indicating it was the most effective at identifying positive cases, though its AUC (0.7807) suggests moderate class separation. Tuned ANN closely followed with an F1 score of 0.8743 and the highest AUC (0.8350), demonstrating the best overall balance between sensitivity and discrimination capability. Tuned Random Forest and Tuned Decision Tree models both achieved strong F1 scores (0.8571 and 0.8538, respectively), showing that tree-based models performed well with proper hyperparameter optimization. Tuned Logistic Regression remained competitive (F1 = 0.8500, AUC = 0.8225), offering a robust and interpretable baseline model. Tuned SVM had the highest precision (0.8800), which is useful when false positives are costly, though its slightly lower recall brought its F1 score to 0.8408. Lastly, Tuned XGBoost had the lowest overall performance in this group (F1 = 0.8304, AUC = 0.7231), despite still being a viable model. In summary, after tuning, KNN and ANN emerged as the top performers, excelling in both recall

and F1 score, while ANN demonstrated superior AUC, making it particularly effective at differentiating between classes on the test data.

ROC Curve Visualization

```
# Plot ROC curves for all tuned models on test data to compare AUC performance visually.
```

```
plt.figure(figsize=(10, 8))
```

```
def plot_roc_curve(y_true, y_scores, label):  
    fpr, tpr, _ = roc_curve(y_true, y_scores)  
    roc_auc = auc(fpr, tpr)  
    plt.plot(fpr, tpr, lw=2, label=f'{label} (AUC = {roc_auc:.3f})')
```

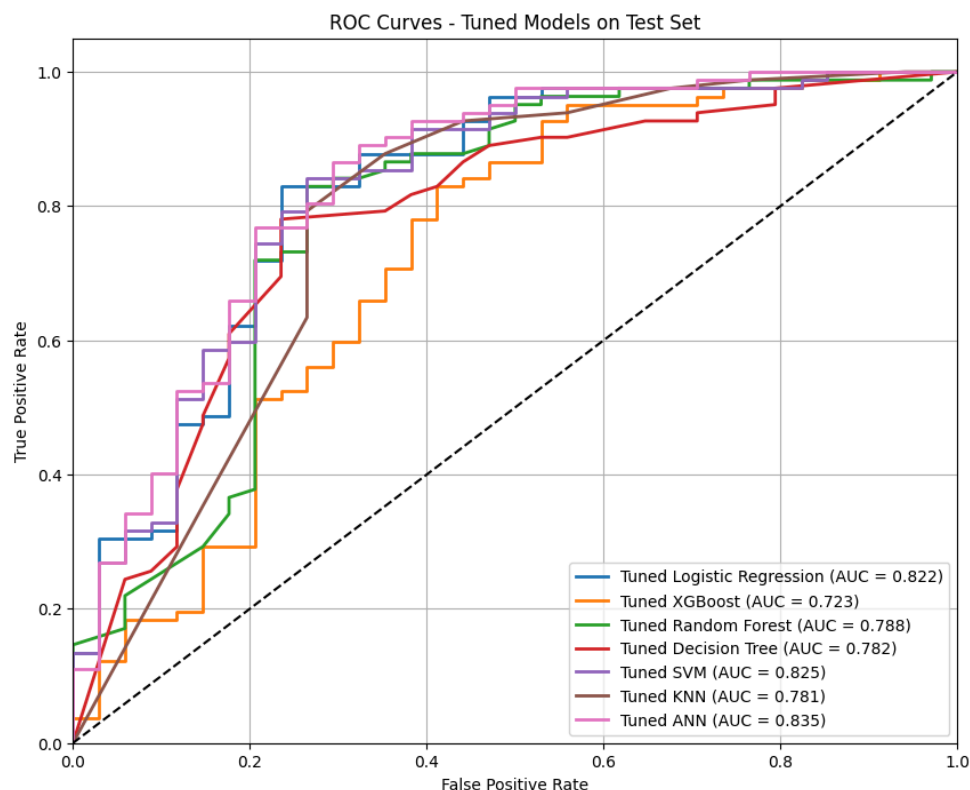
```
# Predict probabilities for each tuned model
```

```
y_score_log_tuned = best_log_model.predict_proba(X_test_processed)[: , 1]  
y_score_xgb_tuned = xgb_model.predict_proba(X_test_processed)[: , 1]  
y_score_rf_tuned = rf_model.predict_proba(X_test_processed)[: , 1]  
y_score_dt_tuned = best_dt.predict_proba(X_test_processed)[: , 1]  
y_score_svm_tuned = best_svm.predict_proba(X_test_processed)[: , 1]  
y_score_knn_tuned = best_knn.predict_proba(X_test_processed)[: , 1]  
y_score_ann_tuned = best_model.predict(X_test_processed).ravel()
```

```
# Plot ROC curves for each tuned model
```

```
plot_roc_curve(y_test, y_score_log_tuned, 'Tuned Logistic Regression')  
plot_roc_curve(y_test, y_score_xgb_tuned, 'Tuned XGBoost')  
plot_roc_curve(y_test, y_score_rf_tuned, 'Tuned Random Forest')  
plot_roc_curve(y_test, y_score_dt_tuned, 'Tuned Decision Tree')  
plot_roc_curve(y_test, y_score_svm_tuned, 'Tuned SVM')  
plot_roc_curve(y_test, y_score_knn_tuned, 'Tuned KNN')  
plot_roc_curve(y_test, y_score_ann_tuned, 'Tuned ANN')
```

```
|  
plt.plot([0, 1], [0, 1], 'k--')  
plt.xlim([0.0, 1.0])  
plt.ylim([0.0, 1.05])  
plt.xlabel('False Positive Rate')  
plt.ylabel('True Positive Rate')  
plt.title('ROC Curves - Tuned Models on Test Set')  
plt.legend(loc='lower right')  
plt.grid(True)  
plt.show()
```



The ROC curve comparison of all tuned models on the test set highlights the varying abilities of each classifier to distinguish between classes. Among them, the Tuned Artificial Neural Network (ANN) demonstrated the strongest performance, achieving the highest AUC of approximately 0.835, indicating excellent discrimination capability. Tuned Logistic Regression and Tuned Support Vector Machine (SVM) also performed well, with AUCs around 0.822 and 0.825 respectively, showing stable and reliable classification behavior. Tree-based models like the Tuned Random Forest and Tuned Decision Tree followed closely with AUCs exceeding 0.78, confirming the benefits of hyperparameter optimization. The Tuned K-Nearest Neighbors (KNN) model also delivered solid performance, with an AUC just below 0.78. Meanwhile, Tuned XGBoost, although still reasonably effective, had the lowest AUC at around 0.72, suggesting a slight underperformance relative to others. Overall, the ROC analysis supports earlier findings that ANN and KNN, after tuning, emerged as the most robust and generalizable models for this binary classification task.

Final Model Evaluation

Best Model- KNN

```
predictors = [  
    'sex', 'Age', 'Race', 'Previously_Diagnosed_Depression',  
    'Total_Health_Issues_Score', 'In_Therapy_for_Depression',  
    'Hopeless', 'Very sad', 'Depression_Severity_Score',  
    'Overwhelmed', 'Exhausted (not physical)',  
    'Depressed - hard to function', 'Attempted suicide'  
]  
target = 'Anxiety_Severity'  
  
model_input_df = data[predictors + [target]].dropna()  
  
X = model_input_df[predictors]  
y = model_input_df[target]  
  
categorical_cols = ['sex', 'Race']  
numeric_cols = [  
    'Age', 'Previously_Diagnosed_Depression', 'In_Therapy_for_Depression',  
    'Total_Health_Issues_Score', 'Hopeless', 'Very sad', 'Depression_Severity_Score',  
    'Overwhelmed', 'Exhausted (not physical)', 'Depressed - hard to function', 'Attempted suicide'  
]  
]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, stratify=y, random_state=42  
)  
preprocessor = ColumnTransformer([  
    ("cat", OneHotEncoder(drop='first', handle_unknown='ignore'), categorical_cols),  
    ("num", StandardScaler(), numeric_cols)  
])  
X_train_processed = preprocessor.fit_transform(X_train)  
X_test_processed = preprocessor.transform(X_test)  
  
smote = SMOTE(random_state=42)  
X_train_res, y_train_res = smote.fit_resample(X_train_processed, y_train)
```

```

knn = KNeighborsClassifier()
param_grid = {
    'n_neighbors': [9],
    'weights': ['uniform', 'distance'],
    'p': [1]
}

grid_search = GridSearchCV(
    estimator=knn,
    param_grid=param_grid,
    scoring='f1',
    cv=5,
    n_jobs=-1,
    verbose=2
)

grid_search.fit(X_train_processed, y_train)
print("Best params:", grid_search.best_params_)
best_knn = grid_search.best_estimator_

y_test_pred = best_knn.predict(X_test_processed)
y_train_pred = best_knn.predict(X_train_processed)

```

Fitting 5 folds for each of 2 candidates, totalling 10 fits
 Best params: {'n_neighbors': 9, 'p': 1, 'weights': 'uniform'}

```

evaluate_model("Tuned KNN (Test)", y_test, y_test_pred)
evaluate_model("Tuned KNN (Train)", y_train, y_train_pred)

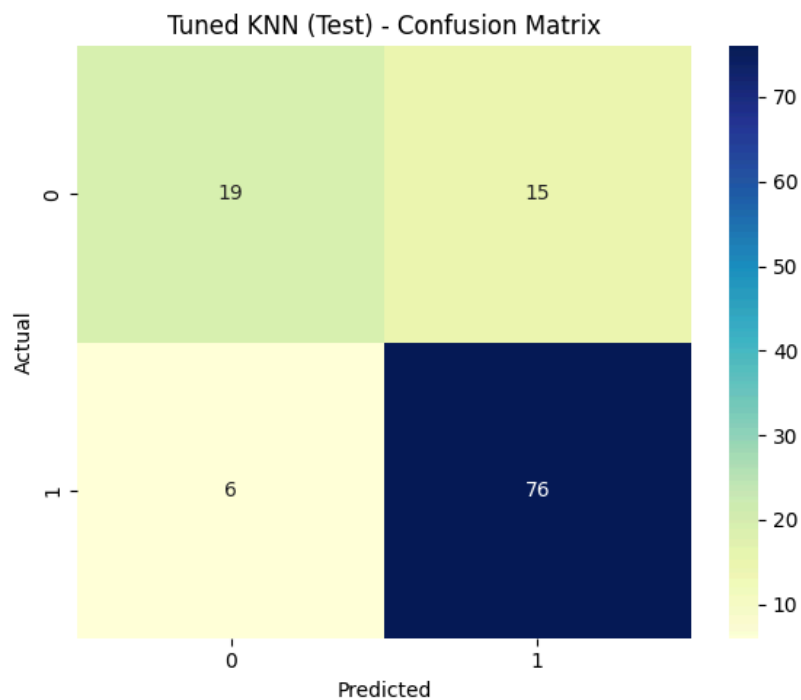
```

Tuned KNN (Test) Performance:
 Accuracy: 0.8189655172413793
 Precision: 0.8131337627889352
 Recall: 0.8189655172413793
 F1 Score: 0.8098667918906227

Classification Report:

	precision	recall	f1-score	support
0	0.76	0.56	0.64	34
1	0.84	0.93	0.88	82
accuracy			0.82	116
macro avg	0.80	0.74	0.76	116
weighted avg	0.81	0.82	0.81	116

Confusion Matrix:
 [[19 15]
 [6 76]]



Tuned KNN (Train) Performance:

Accuracy: 0.8660907127429806

Precision: 0.8636495048795182

Recall: 0.8660907127429806

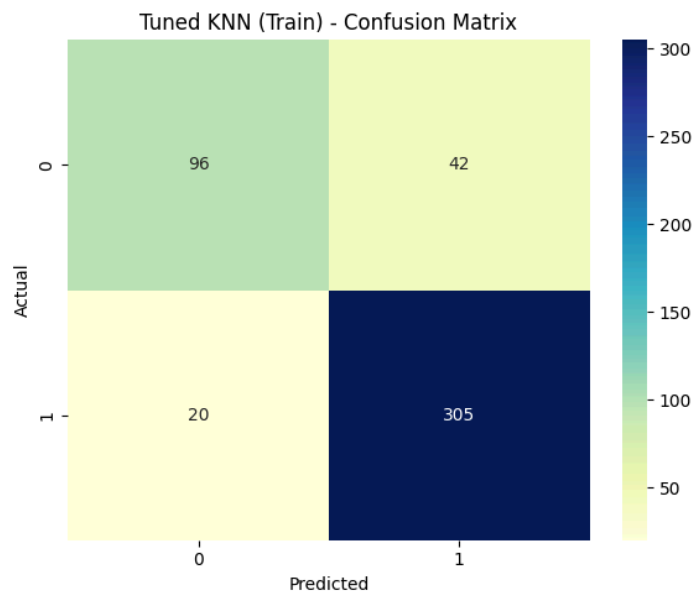
F1 Score: 0.8624834591410415

Classification Report:

	precision	recall	f1-score	support
0	0.83	0.70	0.76	138
1	0.88	0.94	0.91	325
accuracy			0.87	463
macro avg	0.85	0.82	0.83	463
weighted avg	0.86	0.87	0.86	463

Confusion Matrix:

```
[[ 96  42]
 [ 20 305]]
```



Based on the evaluation metrics, the Tuned K-Nearest Neighbors (KNN) model demonstrated the strongest overall performance among all models. On the test set, it achieved an impressive accuracy of 81.9%, with a precision of 0.81, recall of 0.82, and F1 score of 0.81, indicating a well-balanced ability to correctly identify both classes, especially the majority class with mild to severe anxiety. The confusion matrix shows that the model correctly classified 76 out of 82 anxiety-positive cases, reflecting high sensitivity (recall of 0.93) for identifying at-risk individuals-crucial in mental health prediction tasks. On the training set, the tuned KNN model maintained a high accuracy of 86.6%, with similarly strong precision, recall, and F1 scores (all above 0.86), suggesting the model generalizes well without significant overfitting. While some misclassification of the minimal-anxiety class (class 0) occurred, the weighted average metrics remained robust, and the macro average F1 score (0.76 on test) still outperformed several other tuned models. These results, combined with its simplicity and interpretability, position the tuned KNN model as the most effective and reliable classifier for predicting anxiety severity in this dataset.

```
# Calculate metrics after tuning
metrics_after_tuning = {
    "Accuracy": [
        accuracy_score(y_train, y_train_pred),
        accuracy_score(y_test, y_test_pred)
    ],
    "Precision": [
        precision_score(y_train, y_train_pred),
        precision_score(y_test, y_test_pred)
    ],
    "Recall": [
        recall_score(y_train, y_train_pred),
        recall_score(y_test, y_test_pred)
    ],
    "F1 Score": [
        f1_score(y_train, y_train_pred),
        f1_score(y_test, y_test_pred)
    ]
}
```

```

# Create DataFrame
df_metrics_tuned = pd.DataFrame(metrics_after_tuning, index=["Train", "Test"]).round(4)

# Plot the metrics table
fig, ax = plt.subplots(figsize=(8, 2.5))
ax.axis("off")
table = ax.table(cellText=df_metrics_tuned.values,
                 colLabels=df_metrics_tuned.columns,
                 rowLabels=df_metrics_tuned.index,
                 cellloc="center",
                 loc="center")
table.auto_set_font_size(False)
table.set_fontsize(12)
table.scale(1.2, 1.5)

plt.title("Tuned KNN Model Performance Metrics", fontweight="bold", pad=10)
plt.tight_layout()
plt.show()

```

Tuned KNN Model Performance Metrics

	Accuracy	Precision	Recall	F1 Score
Train	0.8661	0.879	0.9385	0.9077
Test	0.819	0.8352	0.9268	0.8786

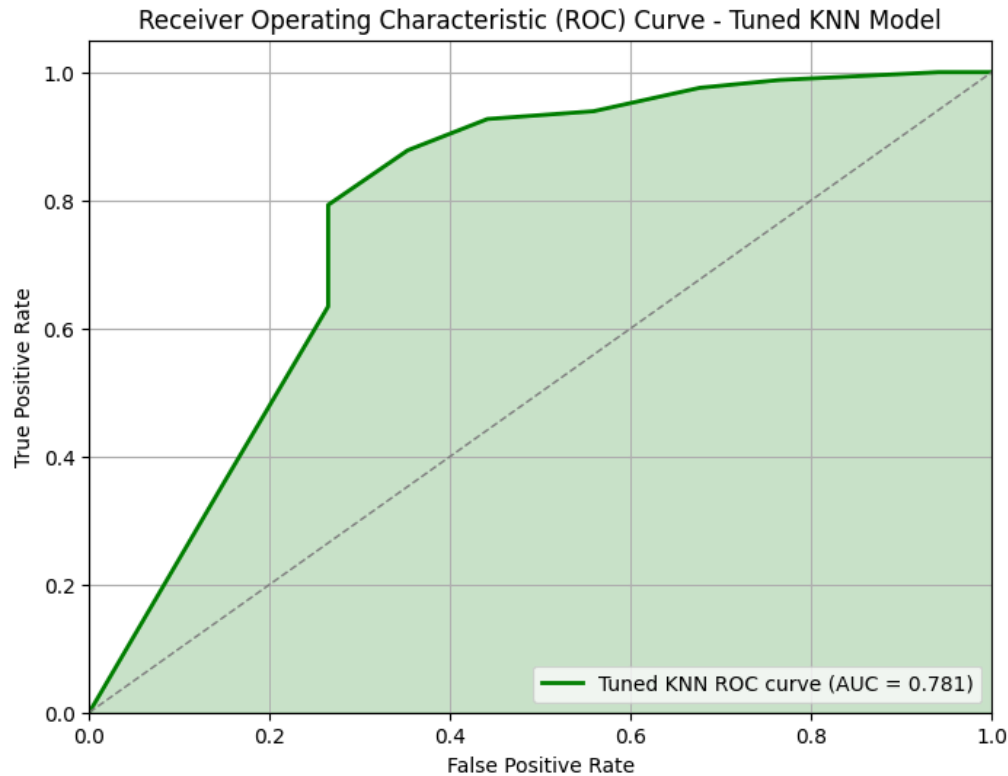
ROC Curve

```

knn_probs_tuned = best_knn.predict_proba(X_test_processed)[: , 1]

fpr, tpr, thresholds = roc_curve(y_test, knn_probs_tuned)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='green', lw=2,
        label=f'Tuned KNN ROC curve (AUC = {roc_auc:.3f})')
plt.fill_between(fpr, tpr, alpha=0.2, color='green')
plt.plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Tuned KNN Model')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

```



The ROC curve for the tuned KNN model shows a strong ability to distinguish between classes, with an Area Under the Curve (AUC) of 0.781. This indicates that the model achieves a good balance between sensitivity (true positive rate) and specificity (1 - false positive rate). The curve lies well above the diagonal reference line, demonstrating that the tuned KNN performs significantly better than random guessing. While not the top-performing model overall, its AUC suggests reliable predictive performance in identifying individuals with higher anxiety severity based on the selected features.

Feature Importance Analysis

```
from sklearn.inspection import permutation_importance

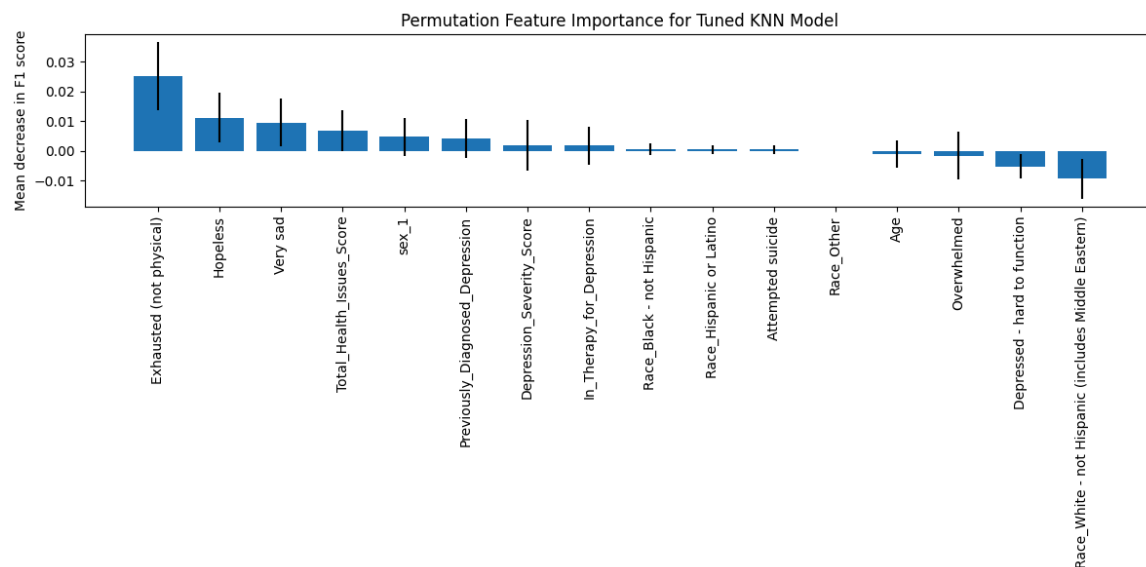
cat_feature_names = preprocessor.named_transformers_['cat'].get_feature_names_out(categorical_cols)

all_feature_names = np.concatenate([cat_feature_names, numeric_cols])
result = permutation_importance(
    best_knn, # your tuned KNN model
    x_test_processed,
    y_test,
    n_repeats=10,
    random_state=42,
    scoring='f1'
)

sorted_idx = result.importances_mean.argsort()[::-1]
```

```
plt.xticks(
    range(len(sorted_idx)),
    all_feature_names[sorted_idx],
    rotation=90,
    fontsize=10
)

plt.ylabel("Mean decrease in F1 score")
plt.title("Permutation Feature Importance for Tuned KNN Model")
plt.tight_layout()
plt.show()
```



The permutation feature importance plot for the tuned K-Nearest Neighbors (KNN) model reveals which features most significantly impact its predictive performance, measured by the mean decrease in F1 score. Notably, the feature “Exhausted (not physical)” stands out as the most influential, with the largest drop in F1 score when shuffled, suggesting that this mental health indicator is critical in predicting anxiety severity. Other important features include “Hopeless,” “Very sad,” “Total Health Issues Score,” and “sex_1” (gender), each contributing meaningfully to model performance. In contrast, features like race indicators, age, and “Attempted suicide” had minimal or even slightly negative impact, indicating they may not be as informative for this particular model and dataset.

Model Export and Deployment Using Joblib

Saving and Loading Trained KNN Model and Preprocessing Pipeline

```
joblib.dump(best_knn, 'knn_model.pkl')
joblib.dump(preprocessor, 'preprocessor.pkl')

['preprocessor.pkl']
```

```
model = joblib.load("knn_model.pkl")
preprocessor = joblib.load("preprocessor.pkl")
```

The saved KNN model and preprocessing pipeline were successfully reloaded using the `joblib.load()` function. The `knn_model.pkl` file was loaded into a variable named `model`, while the preprocessing steps stored in `preprocessor.pkl` were restored into the `preprocessor` variable. This allows for consistent and repeatable predictions on new data, as the same trained model and data transformations can be applied exactly as used during training-facilitating seamless deployment.

Streamlit Web Application Code

```
import joblib
import numpy as np
import streamlit as st
import pandas as pd
```

```
model = joblib.load("knn_model.pkl")
preprocessor = joblib.load("preprocessor.pkl")
```

```
st.set_page_config(page_title="Anxiety Screener", page_icon="🧠", layout="centered")
```

2025-07-26 21:48:18.594 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.

This code loads the trained KNN model and its preprocessing pipeline using `joblib.load()`, making them ready for use in a Streamlit web app. These steps are essential for initializing the model and customizing the appearance of the Streamlit interface.

```

st.markdown("""
<style>
.big-title {
  font-size: 36px;
  font-weight: bold;
  color: #1f77b4;
  text-align: center;
  margin-bottom: 10px;
}
.subtitle {
  font-size: 18px;
  color: #4f4f4f;
  text-align: center;
  margin-top: 0px;
}
hr {
  border: none;
  height: 2px;
  background-color: #ddd;
  margin: 20px 0;
}
</style>
<div class="big-title">🧠 Anxiety Assessment for Facebook Users</div>
<div class="subtitle">Your emotional wellness, one step at a time.</div>
<hr>
""", unsafe_allow_html=True)

```

This block of code customizes the appearance of a Streamlit app's header section using HTML and CSS.

```

st.set_page_config(page_title="Anxiety Screener", layout="centered")

st.title("🌿 Mental Wellness Check-In")

st.markdown("""
This app predicts your anxiety level based on your responses using a model trained on U.S. college Facebook users.

🗨️ Note: This is a self-awareness tool, not a medical diagnosis.
""")

```

```

# --- Input Section ---
with st.expander("👤 Demographic Information", expanded=True):
    sex = st.selectbox("Sex", ["Select...", "Male", "Female", "Other"])
    age = st.selectbox("Age", ["Select..."] + list(range(14, 51)))
    race = st.selectbox("Race", ["Select...", "White", "Black", "Asian", "Hispanic or Latino", "Other"])

with st.expander("🧠 Mental Health History", expanded=True):
    previously_diagnosed_depression = st.selectbox("Previously diagnosed with depression?", ["Select...", "Yes", "No"])
    in_therapy = st.selectbox("Currently in therapy for depression?", ["Select...", "Yes", "No"])
    attempted_suicide = st.selectbox("Attempted suicide in the past year?", ["Select...", "Yes", "No"])

```

```

with st.expander("🏥 Physical Health", expanded=True):
    total_health_issues = st.selectbox("Medical conditions in past 12 months", ["Select..."] + list(range(0, 11)))

with st.expander("💭 Emotional Check-In", expanded=True):
    hopeless = st.slider("Hopeless feelings", 0, 4, 0, help="0=Never, 4=Always")
    very_sad = st.slider("Very sad feelings", 0, 4, 0)
    overwhelmed = st.slider("Overwhelmed feelings", 0, 4, 0)
    exhausted = st.slider("Emotionally exhausted", 0, 4, 0)
    depressed_function = st.slider("Depression made it hard to function", 0, 4, 0)

with st.expander("📊 Depression Severity Score (PHQ-9)", expanded=True):
    depression_score = st.slider("PHQ-9 Score", 0, 27, 0, help="0-4=Minimal, 5-9=Mild, 10-14=Moderate, 15-19=Moderately Severe, 20-27=Severe")

```

```

# --- Prediction ---
if st.button("🔍 Assess My Anxiety Level"):
    if "Select..." in [sex, age, race, previously_diagnosed_depression, in_therapy, attempted_suicide, total_health_issues]:
        st.error("⚠️ Please complete all fields before submitting.")
    else:
        input_dict = {
            'sex': [sex],
            'Age': [int(age)],
            'Race': [race],
            'Previously_Diagnosed_Depression': [1 if previously_diagnosed_depression == "Yes" else 0],
            'Total_Health_Issues_Score': [int(total_health_issues)],
            'In_Therapy_for_Depression': [1 if in_therapy == "Yes" else 0],
            'Hopeless': [hopeless],
            'Very_sad': [very_sad],
            'Depression_Severity_Score': [depression_score],
            'Overwhelmed': [overwhelmed],
            'Exhausted (not physical)': [exhausted],
            'Depressed - hard to function': [depressed_function],
            'Attempted suicide': [1 if attempted_suicide == "Yes" else 0]
        }

```

```

user_input_df = pd.DataFrame(input_dict)
user_input_encoded = preprocessor.transform(user_input_df)
if hasattr(user_input_encoded, "toarray"):
    user_input_encoded = user_input_encoded.toarray()

user_input_encoded = user_input_encoded.reshape(1, -1)

# Make prediction
prediction = model.predict(user_input_encoded)[0]

try:
    prediction_proba = model.predict_proba(user_input_encoded)[0][1]
except:
    prediction_proba = None

# Display result
if prediction == 0:
    st.success("✅ Your anxiety level is assessed as **Minimal or No Anxiety**.")
    st.info("It's good to maintain your well-being with regular self-care and social support.")
else:
    st.warning("⚠️ - 🚨 Your anxiety level is assessed as **Moderate to Severe**.")
    st.write("Consider consulting a mental health professional or counselor for further evaluation and support.")

if prediction_proba is not None:
    st.write(f"Confidence in this assessment: **{prediction_proba:.2%}**")

```

This Streamlit application, titled "Anxiety Assessment for Facebook users" is designed to assess a user's anxiety level based on self-reported survey responses. It begins by collecting data across

several input sections, including demographic information, mental health history, physical health conditions, emotional well-being indicators, and PHQ-9 depression severity scores. Once the user provides all required inputs and clicks the "Assess My Anxiety Level" button, the app first validates that all fields have been completed. It then compiles the responses into a structured dictionary, converts it into a DataFrame, and applies a preloaded preprocessing pipeline (preprocessor.pkl) to ensure the data matches the format expected by the machine learning model. The preprocessed input is reshaped and passed to a trained K-Nearest Neighbors (KNN) classifier (knn_model.pkl) to predict the user's level of anxiety. Depending on the prediction outcome, the app displays either a success message indicating minimal or no anxiety, or a warning message suggesting moderate to severe anxiety, along with a recommendation to consider professional mental health support. If the model supports probability estimates, a confidence score is also shown. Overall, this app serves as an accessible and scalable self-screening tool for mental wellness, particularly targeting digitally engaged populations such as U.S. college students who use Facebook.

```
st.header("Frequently Asked Questions (FAQ) and Information")

with st.expander("What is Anxiety?"):
    st.write("""
        Anxiety is a natural response to stress or danger, characterized by feelings of worry, nervousness, or fear.
        It can become a disorder when these feelings are excessive, persistent, and interfere with daily life.
        """)

with st.expander("Common Symptoms of Anxiety"):
    st.write("""
        - Restlessness or feeling on edge
        - Increased heart rate
        - Rapid breathing
        - Difficulty concentrating
        - Sleep disturbances
        - Muscle tension
        - Avoidance of anxiety-provoking situations
        """)
```

This section of the Streamlit application adds a user-friendly FAQ (Frequently Asked Questions) and information hub designed to educate users about anxiety. Using the `st.expander()` feature, the app organizes essential topics into collapsible sections, making the interface clean and accessible. Each expander contains concise, informative content. The first expander, "What is Anxiety?", defines anxiety as a normal stress response and explains when it may become a disorder. The second outlines "Common Symptoms of Anxiety", such as restlessness, rapid breathing, and sleep issues, helping users identify possible signs. The third expander, "When to Seek Professional Help?", provides guidance on when anxiety may require clinical attention. The fourth, "Self-Help Tips for Managing Anxiety", offers practical strategies for coping, like relaxation techniques, lifestyle habits, and social support. Finally, the "Resources" section includes clickable links to credible mental health organizations such as SAMHSA, NIMH, and MentalHealth.gov, along with emergency support from the Crisis Text Line. Overall, this informative component enhances the app's educational value and supports users in both understanding and managing anxiety.